

분산처리



Computer & Ai

Department of Computer Engineering

왜 병렬 처리일까요? 왜 효율성일까요?

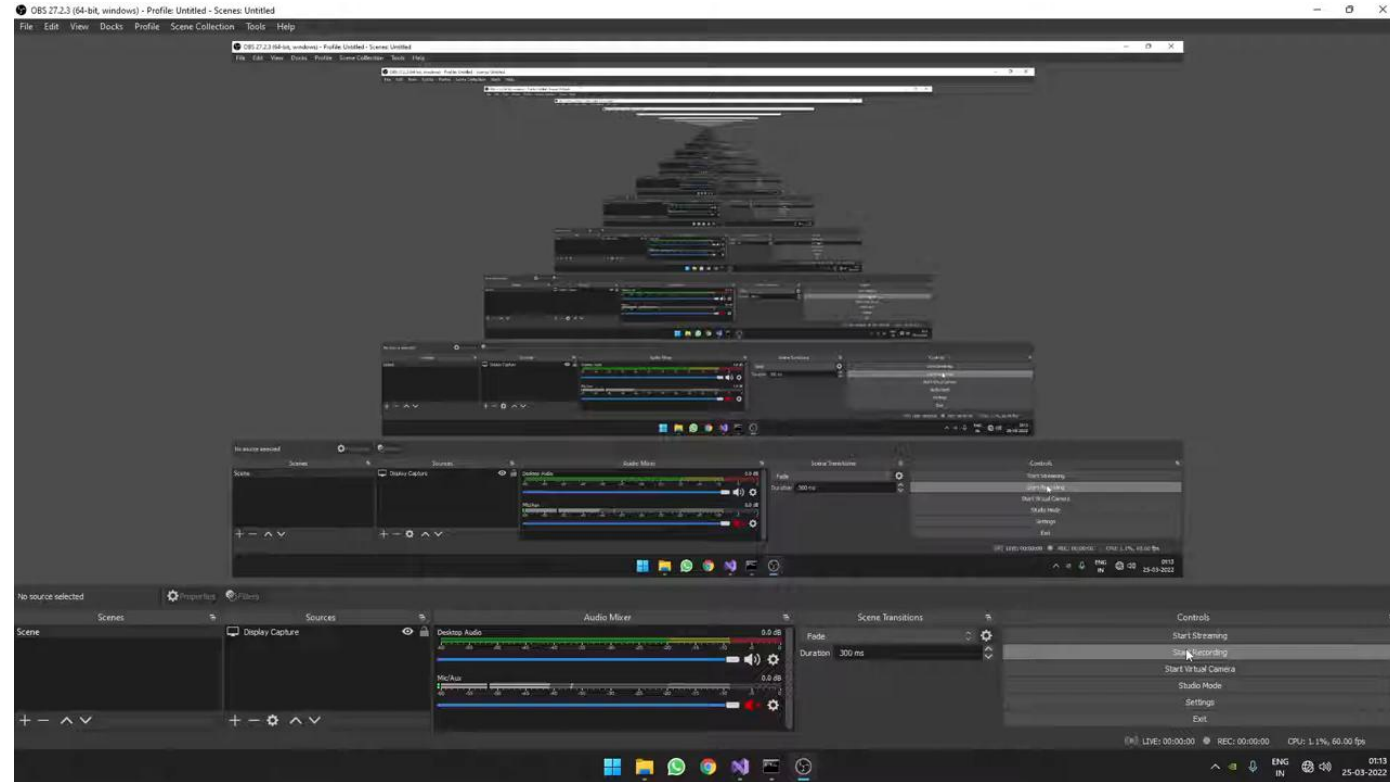
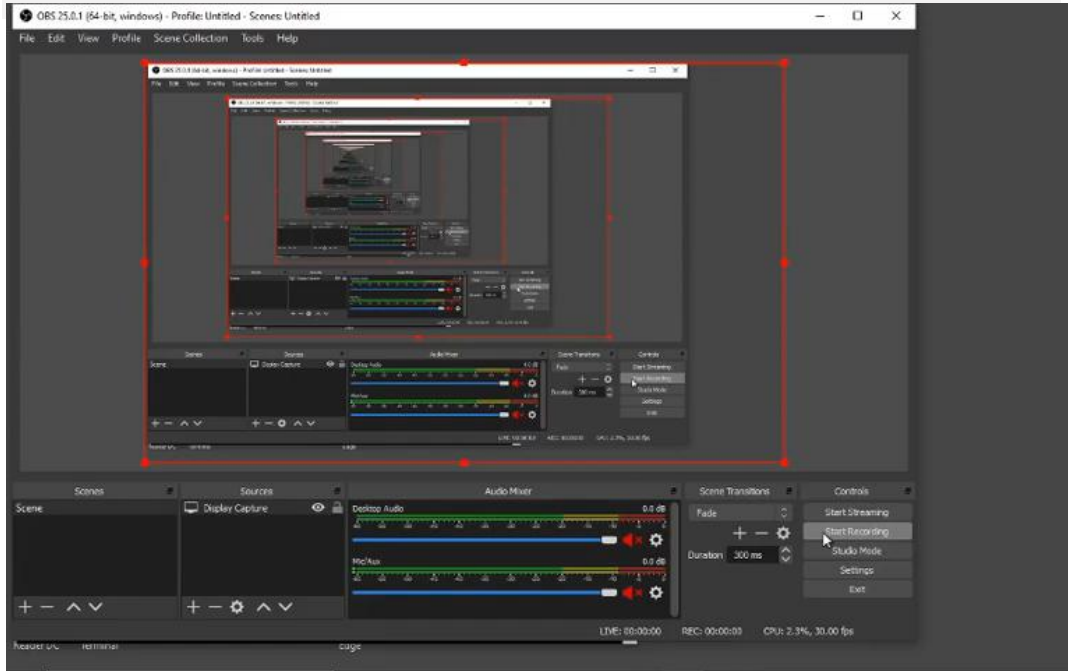
한 가지 공통된 정의

병렬 컴퓨터는 **처리 요소의 모음**으로 의 집합으로, 문제를 **빠르게** 해결하기 위해 협력합니다.

이를 얻기 위해 여러 처리
요소를 사용할 것

성능과 효율성을
중요하게 고려함

Mandelbrot Fractal



한 번에 많은 프로세싱 elements 를 돌려서 성능 향상을 얻는 것이 병렬 프로그램의 목적.

$$\text{speedup(using P processors)} = \frac{\text{execution time (using 1 processor)}}{\text{execution time (using P processors)}}$$

- 위와 같은 문제를 설정하고 speedup 을 최대로 하는 것이 목표.
- 프로세스의 갯수 P 와 speedup 이 비례하다면 이를 scalable 하다고 함.
- 통신은 최대 속도 향상을 달성하는 데 한계가 됨
→ 따라서 communication 시간을 줄이면 성능이 자연스레 오른다.
- 작업의 불균형도 최대 속도 향상을 달성하는 데 한계가 됨
→ task 를 잘 배분하면 성능이 오른다.

강의 주제1:

병렬 프로그램 설계 및 작성... 그 규모!

- 병렬적 사고(Parallel thinking)

1. 안전하게 병렬로 수행 할 수 있는 조각(크기)으로 작업 분해.
2. 프로세서에 작업 할당하기
3. 속도 향상을 제한하지 않도록 프로세서 간의 통신/동기화 관리

- 위의 작업을 수행하기 위한 추상화/메커니즘.

- 널리 사용되는 병렬 프로그래밍 언어로 코드 작성 (CUDA, MPI)

강의 주제2:

병렬 컴퓨터 하드웨어 구현: 병렬 컴퓨터의 작동 방식.

- 추상화를 효율적으로 구현하는 데 사용되는 메커니즘

- - 구현의 성능 특성
- - 설계 트레이드 오프: 성능과 편의성, 비용 비교

- 하드웨어에 대해 알아야 하는 이유는 무엇인가요?

- - 기계의 특성이 정말 중요하기 때문.
(통신 속도 문제 등)
- - 효율성과 성능이 중요하기 때문.
(결국 병렬 프로그램을 작성해야 하기 때문에!)

효율성에 대한 생각!

joonojoono.tistory.com/30

- FAST != EFFICIENT
- 병렬 컴퓨터에서 프로그램이 더 빠르게 실행된다고 해서 하드웨어를 효율적으로 사용하고 있다는 의미는 아님.
 - 프로세서가 10개인 컴퓨터에서 2배의 속도 향상이 좋은 결과일까요?
- 프로그래머의 관점: 제공된 기계의 기능을 활용해야 합니다.
- HW 설계자의 관점: 시스템에 넣을 적절한 기능 선택(성능/비용? 비용 = 실리콘 면적?, 전력? 등)

Why parallelism?

역사적 맥락: 병렬 처리를 회피한 이유는 무엇인가요?

- 싱글 스레드 CPU 성능 2배 증가 ~ 18개월마다 증가
- 시사점: 코드 병렬화 작업은 종종 시간 대비 가치가 없었습니다.
 - - 소프트웨어 개발자가 아무것도 하지 않아도 내년에는 코드가 더 빨라집니다. 우와!

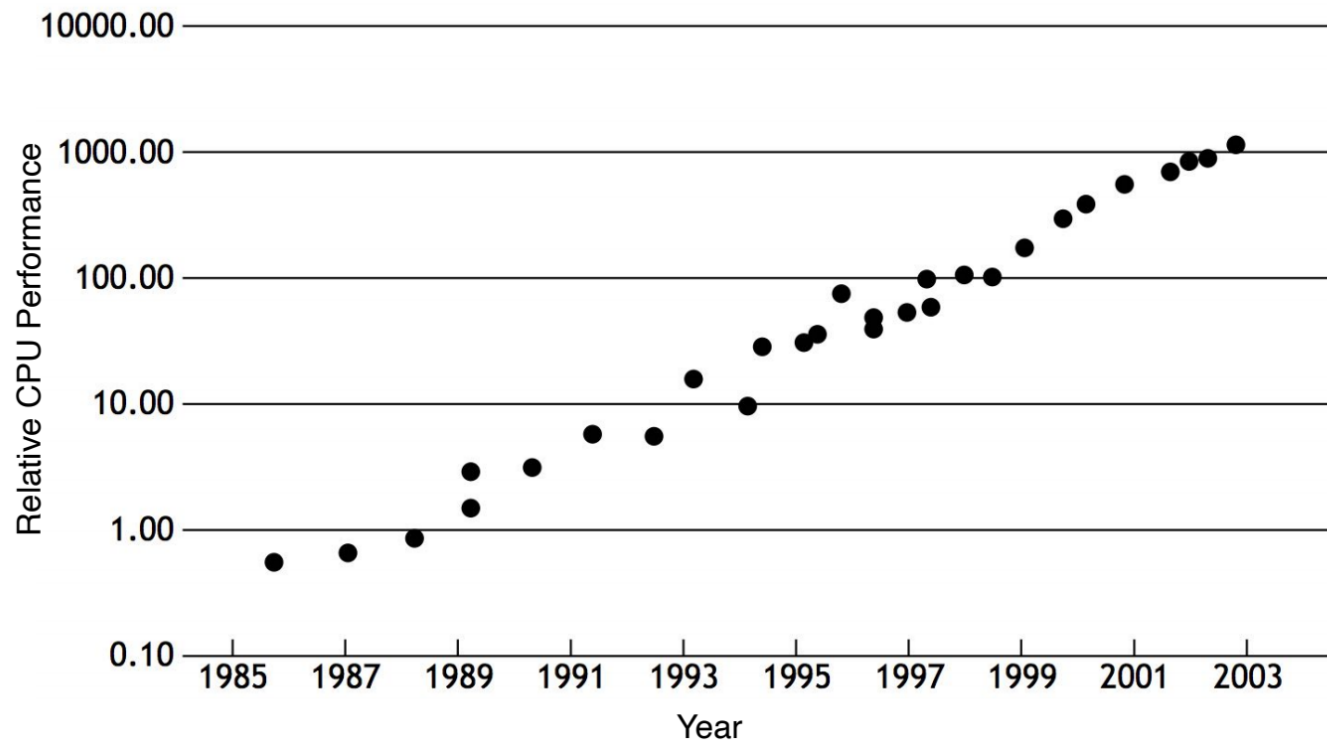


Image credit: Olukutun and Hammond, ACM Queue 2005

역사적 맥락: 병렬 처리를 회피한 이유는 무엇인가요?

- 15년 전까지만 해도 프로세서 성능 개선의 두 가지 중요한 이유는 다음과 같습니다.
 1. 명령어 수준의 병렬 처리 (instruction-level parallelism)(슈퍼스칼라 실행) 활용
 2. CPU 클럭 주파수 증가

Instruction level parallelism (ILP) example

- ILP
 - 한 번에 병렬 처리할 수 있는 instructions 을 구분하기 위한 level 값

$$a = x*x + y*y + z*z$$

Consider the following five instruction program:

Assume register $R0 = x$, $R1 = y$, $R2 = z$

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

- 왼쪽과 같은 예제가 있다고 하자.
- 총 5개의 instructions 로 이루어져있음.

This program has five instructions, so it will take five clocks to execute, correct?

Can we do better?

What is a computer program?

프로그램이란 무엇인가요? (프로세서의 관점에서)

- 프로그램은 프로세서 명령어의 리스트일 뿐

```
int main(int argc, char** argv) {  
  
    int x = 1;  
  
    for (int i=0; i<10; i++) {  
        x = x + x;  
    }  
  
    printf("%d\n", x);  
  
    return 0;  
}
```



Compile
code



```
_main:  
100000f10:    pushq    %rbp  
100000f11:    movq     %rsp, %rbp  
100000f14:    subq     $32, %rsp  
100000f18:    movl     $0, -4(%rbp)  
100000f1f:    movl     %edi, -8(%rbp)  
100000f22:    movq     %rsi, -16(%rbp)  
100000f26:    movl     $1, -20(%rbp)  
100000f2d:    movl     $0, -24(%rbp)  
100000f34:    cmpl     $10, -24(%rbp)  
100000f38:    jge      23 <_main+0x45>  
100000f3e:    movl     -20(%rbp), %eax  
100000f41:    addl     -20(%rbp), %eax  
100000f44:    movl     %eax, -20(%rbp)  
100000f47:    movl     -24(%rbp), %eax  
100000f4a:    addl     $1, %eax  
100000f4d:    movl     %eax, -24(%rbp)  
100000f50:    jmp      -33 <_main+0x24>  
100000f55:    leaq     58(%rip), %rdi  
100000f5c:    movl     -20(%rbp), %esi  
100000f5f:    movb     $0, %al  
100000f61:    callq    14  
100000f66:    xorl     %esi, %esi  
100000f68:    movl     %eax, -28(%rbp)  
100000f6b:    movl     %esi, %eax  
100000f6d:    addq     $32, %rsp  
100000f71:    popq     %rbp  
100000f72:    rets
```

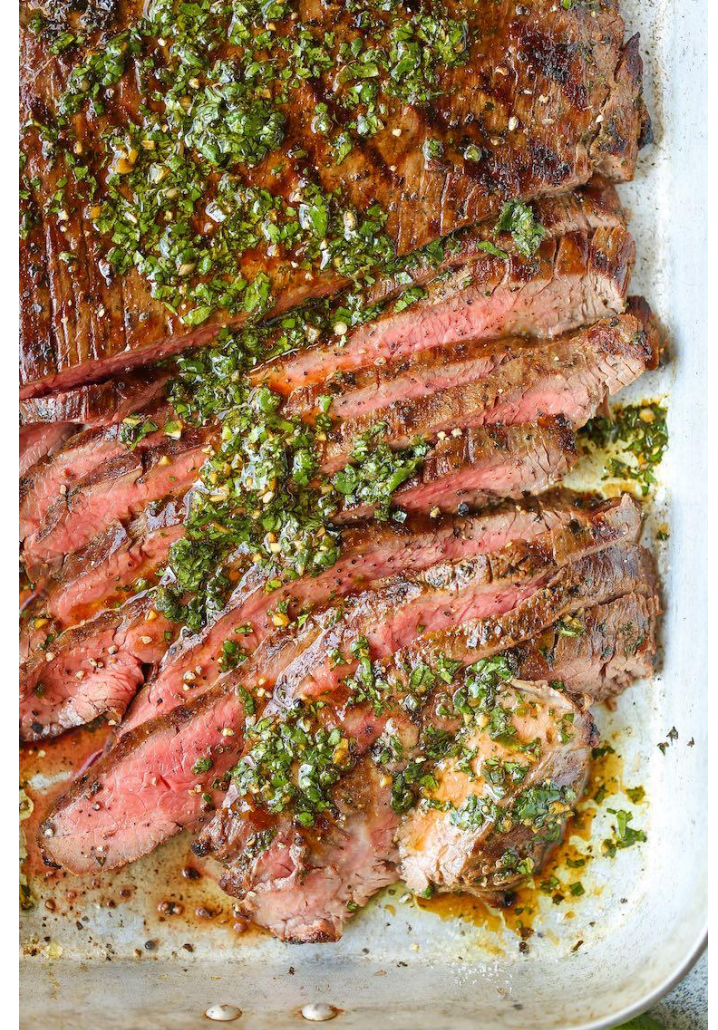

좋아하는 요리 레시피의 설명과 비슷

- 프로그램은 프로세서 명령어의 리스트일 뿐

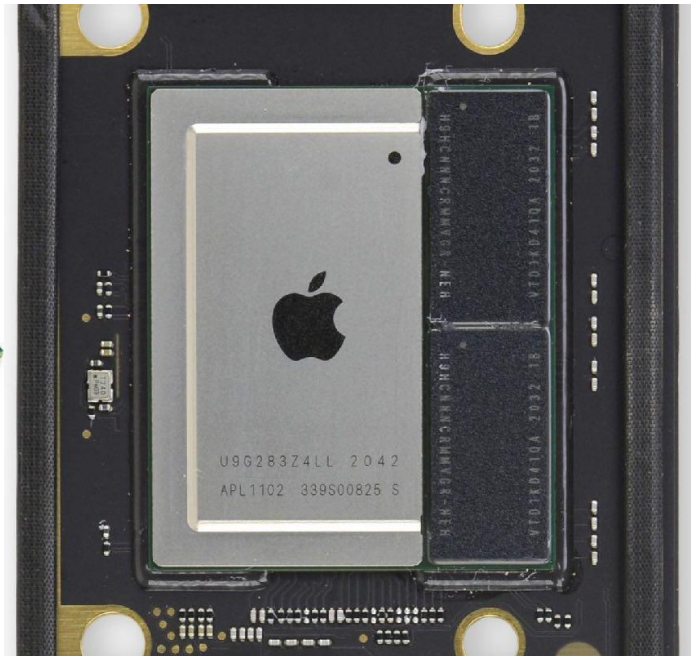
카르네 아사다

Instructions

1. In a large mixing bowl combine orange juice, olive oil, cilantro, lime juice, lemon juice, white wine vinegar, cumin, salt and pepper, jalapeno, and garlic; whisk until well combined.
2. Reserve $\frac{1}{3}$ cup of the marinade; cover the rest and refrigerate.
3. Combine remaining marinade and steak in a large resealable freezer bag; seal and refrigerate for at least 2 hours, or overnight.
4. Preheat grill to HIGH heat.
5. Remove steak from marinade and lightly pat dry with paper towels.
6. Add steak to the preheated grill and cook for another 6 to 8 minutes per side, or until desired doneness. **Note that flank steak tastes best when cooked to rare or medium rare because it's a lean cut of steak.**
7. Remove from heat and let rest for 10 minutes. Thinly slice steak against the grain, garnish with reserved cilantro mixture, and serve.

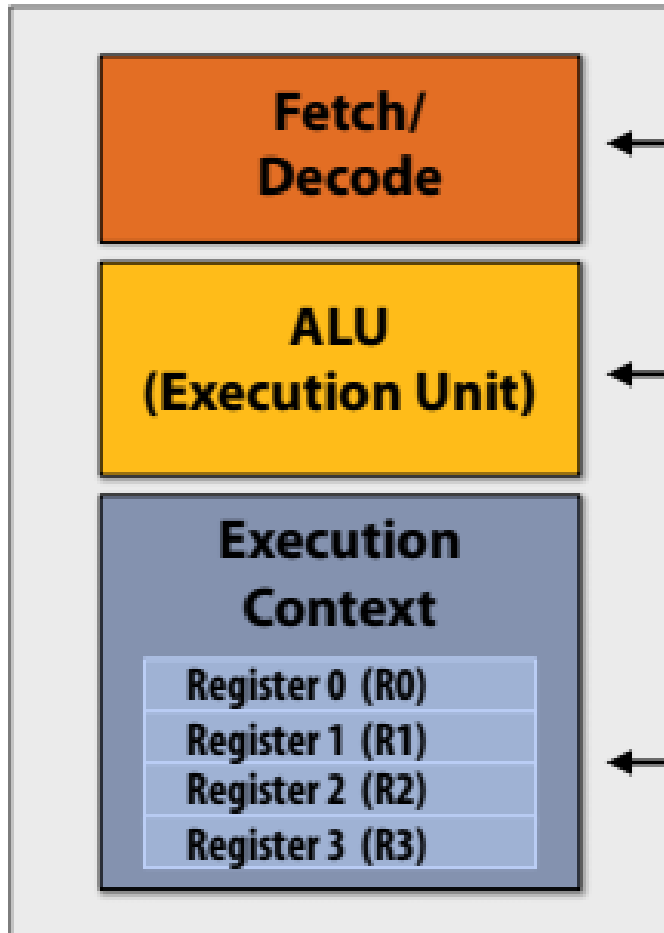


프로세서의 역할은 무엇인가요?



프로세서는 명령어를 실행

Very Simple Processor



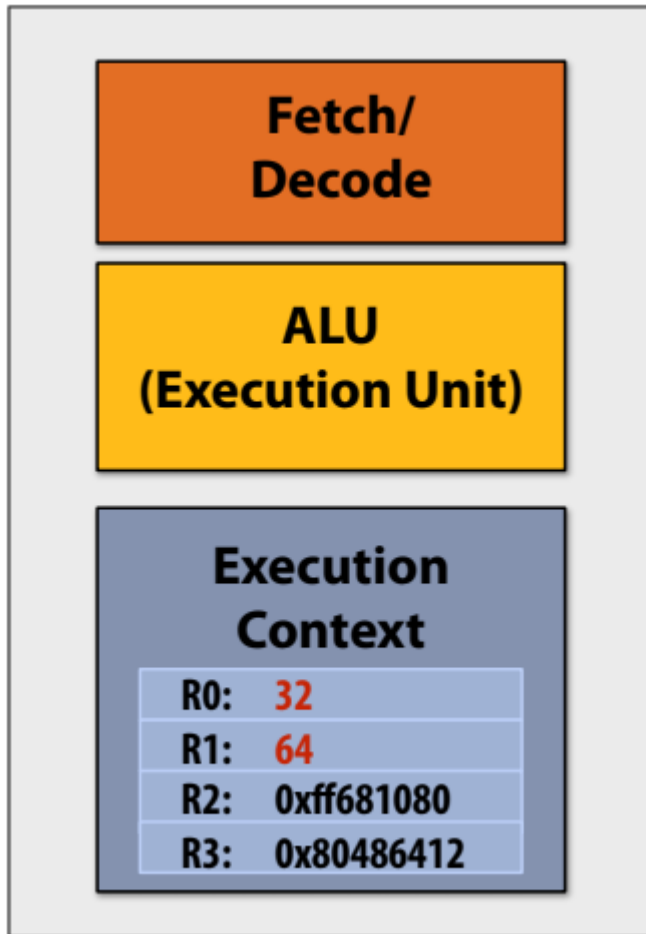
- 다음에 실행할 명령어를 결정.

- 실행 단위: 명령어로 기술된 작업을 수행하며, 프로세서 레지스터 또는 컴퓨터 메모리의 값을 수정할 수 있음

- 레지스터: 프로그램 상태 유지: 연산에 입력 및 출력으로 사용되는 변수의 값을 저장.

한 가지 예시: 숫자 두 개 더하기

Very Simple Processor



Step 1:

- 프로세서가 메모리에서 다음 프로그램 명령을 가져옵니다("프로세서가 다음에 수행해야 할 작업 파악").

add R0 ← R0, R1

"레지스터 R0의 내용을 레지스터 R1의 내용에 더하고 그 결과를 레지스터 R0에 넣어주세요."

Step 2:

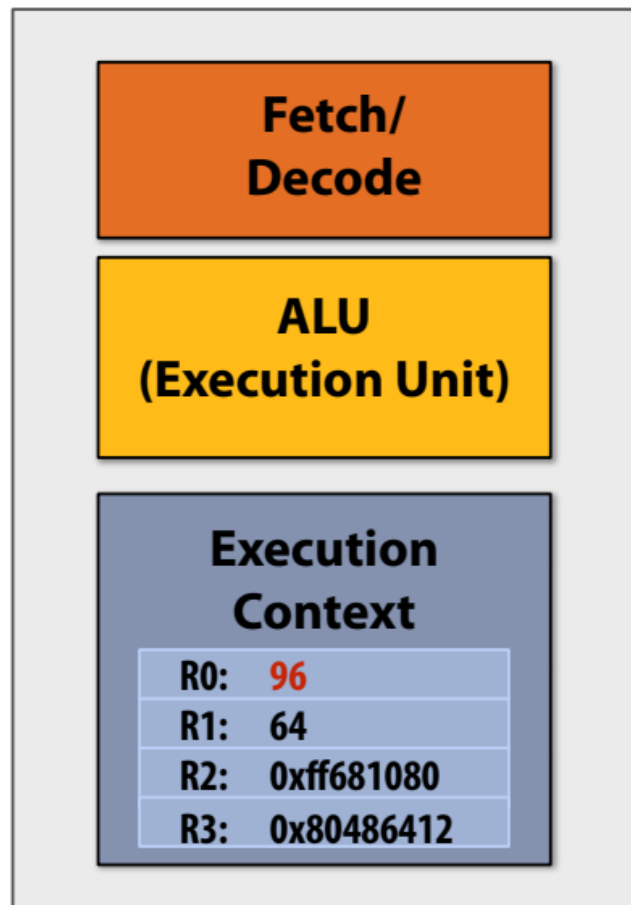
- 레지스터에서 연산 입력을 가져옵니다.
- 실행 유닛에 입력된 R0의 내용: 32
- 실행 유닛에 입력된 R1의 내용: 64

Step 3:

- 더하기 작업을 수행
- 실행 단위가 산술을 수행하면 결과는 96

한 가지 예시: 숫자 두 개 더하기

Very Simple Processor



Step 1:

- 프로세서가 메모리에서 다음 프로그램 명령을 가져옵니다("프로세서가 다음에 수행해야 할 작업 파악").

add R0 ← R0, R1

"레지스터 R0의 내용을 레지스터 R1의 내용에 더하고 그 결과를 레지스터 R0에 넣어주세요."

Step 2:

- 레지스터에서 연산 입력을 가져옵니다.
- 실행 유닛에 입력된 R0의 내용: 32
- 실행 유닛에 입력된 R1의 내용: 64

Step 3:

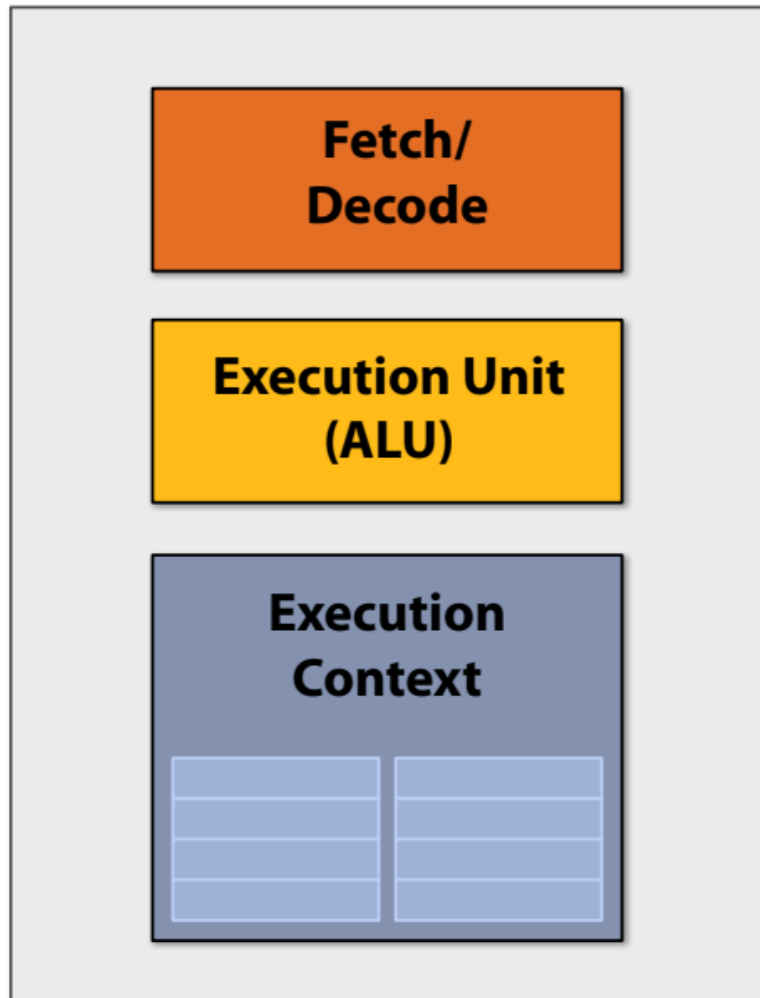
- 더하기 작업을 수행
- 실행 단위가 산술을 수행하면 결과는

Step 4:

- 결과 96 을 저장하고 R0에 Register(등록).

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

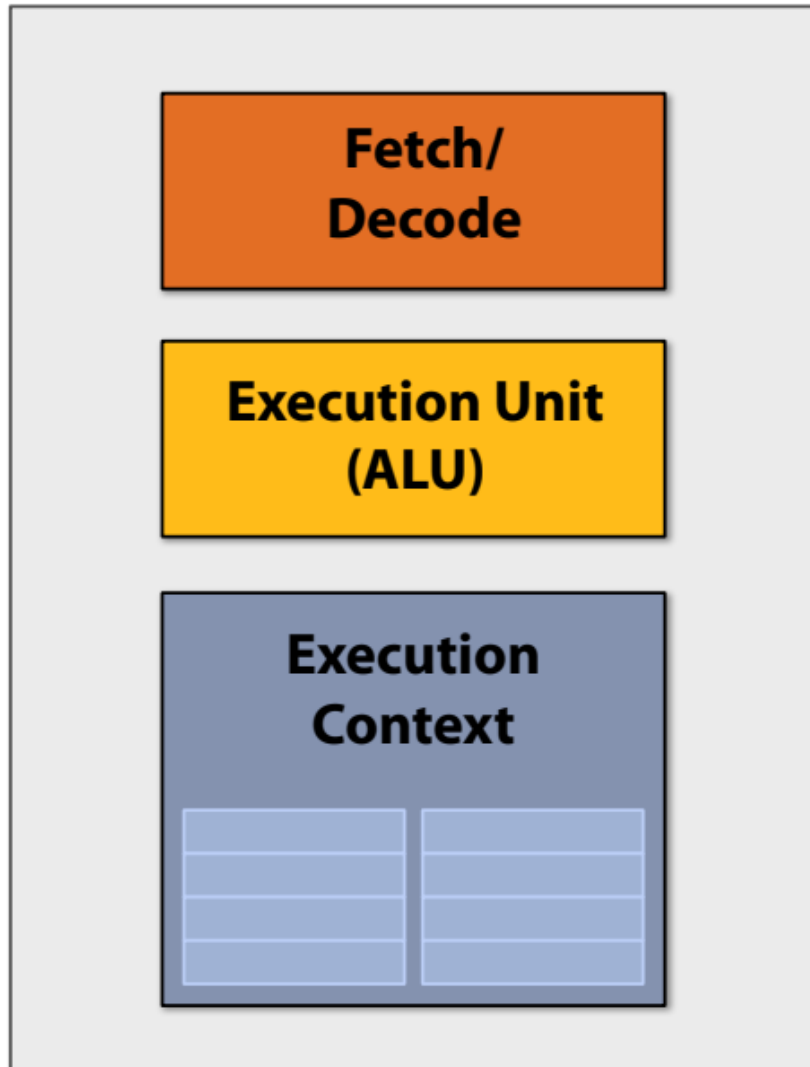
```
...
```

```
...
```

```
st    addr[r2], r0
```

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld  r0, addr[r1]
```

```
mul r1, r0, r0
```

```
mul r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

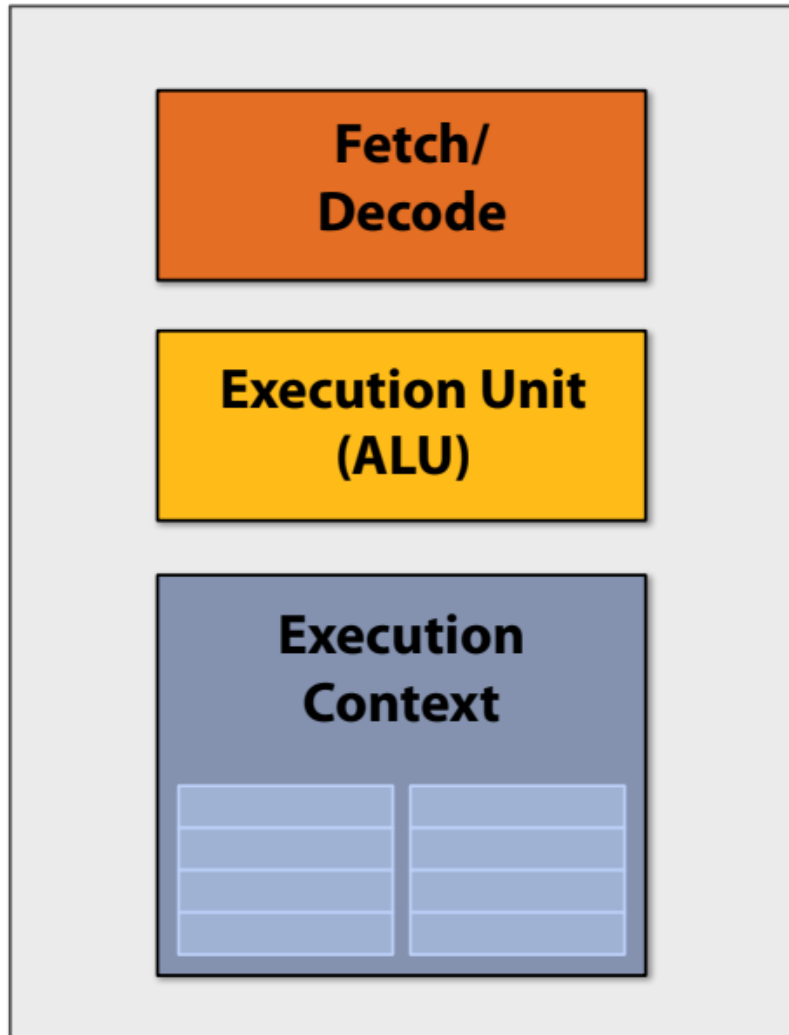
```
...
```

```
...
```

```
st  addr[r2], r0
```

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

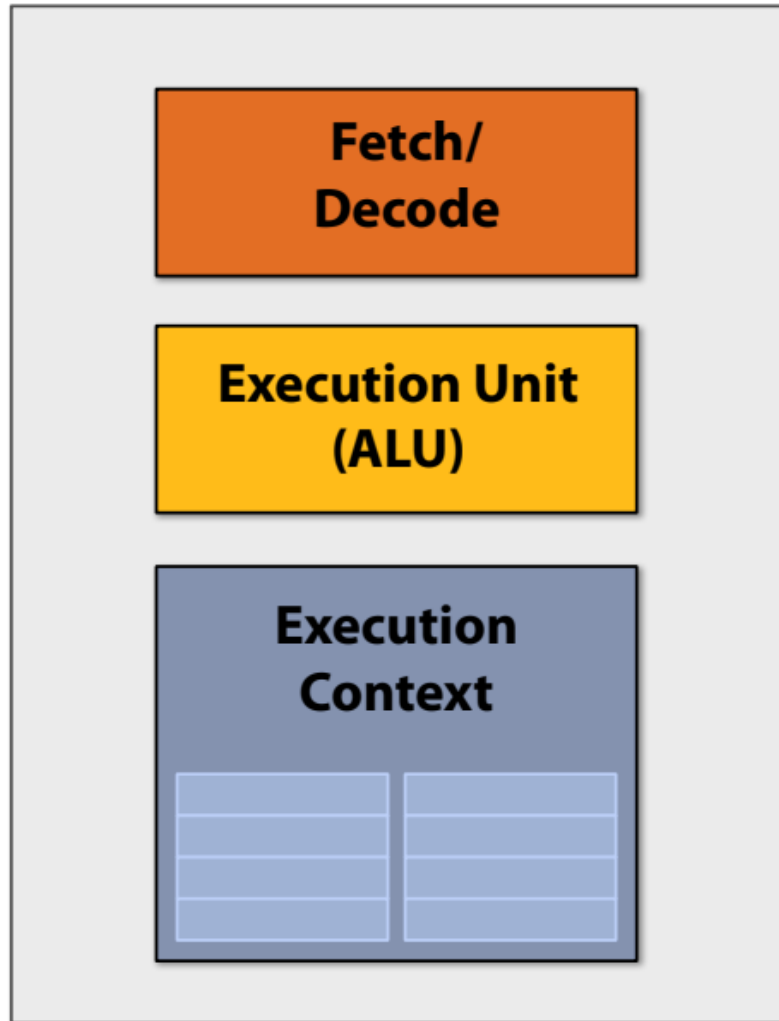
```
...
```

```
...
```

```
st    addr[r2], r0
```


프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
mul   r1, r0, r0
mul   r1, r1, r0
...
...
...
...
...
...
st    addr[r2], r0
```

컴퓨터 작동 원리 살펴보기...

- 컴퓨터 프로그램이란 무엇인가요? (프로세서의 관점에서)
 - 실행할 명령어 목록!
- 명령어란 무엇인가요?
 - 프로세서가 수행해야 할 작업을 기술한 것.
 - 명령어를 실행하면 일반적으로 컴퓨터의 상태가 변경됨.
- 컴퓨터의 “상태(state)”란 무엇을 의미하나요?
 - 프로세서의 레지스터나 메모리에 저장된 프로그램 데이터의 값을 말함.

아주 간단한 코드를 예로 들어 보자

$$a = x*x + y*y + z*z$$

다음 5개의 명령어 프로그램을 고려하자

Assume register R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

- 이 프로그램에는 다섯 개의 명령어가 있으므로 실행하는 데 다섯 번의 클럭이 걸리겠죠?
- 더 좋은 방법이 있을까요?

아주 간단한 코드를 예로 들어 보자

한 번에 최대 두 개의 명령어를 수행할 수 있다면 어떻게 되나요?

$$a = x*x + y*y + z*z$$

Assume register

R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of
program variable 'a'*

1. mul R0, R0, R0

4. add R0, R0, R1

2. mul R1, R1, R1

5. add R3, R0, R2

3. mul R2, R2, R2

time

1

2

3

4

5

Processor 1

Processor 2

아주 간단한 코드를 예로 들어 보자

한 번에 최대 두 개의 명령어를 수행할 수 있다면 어떻게 되나요?

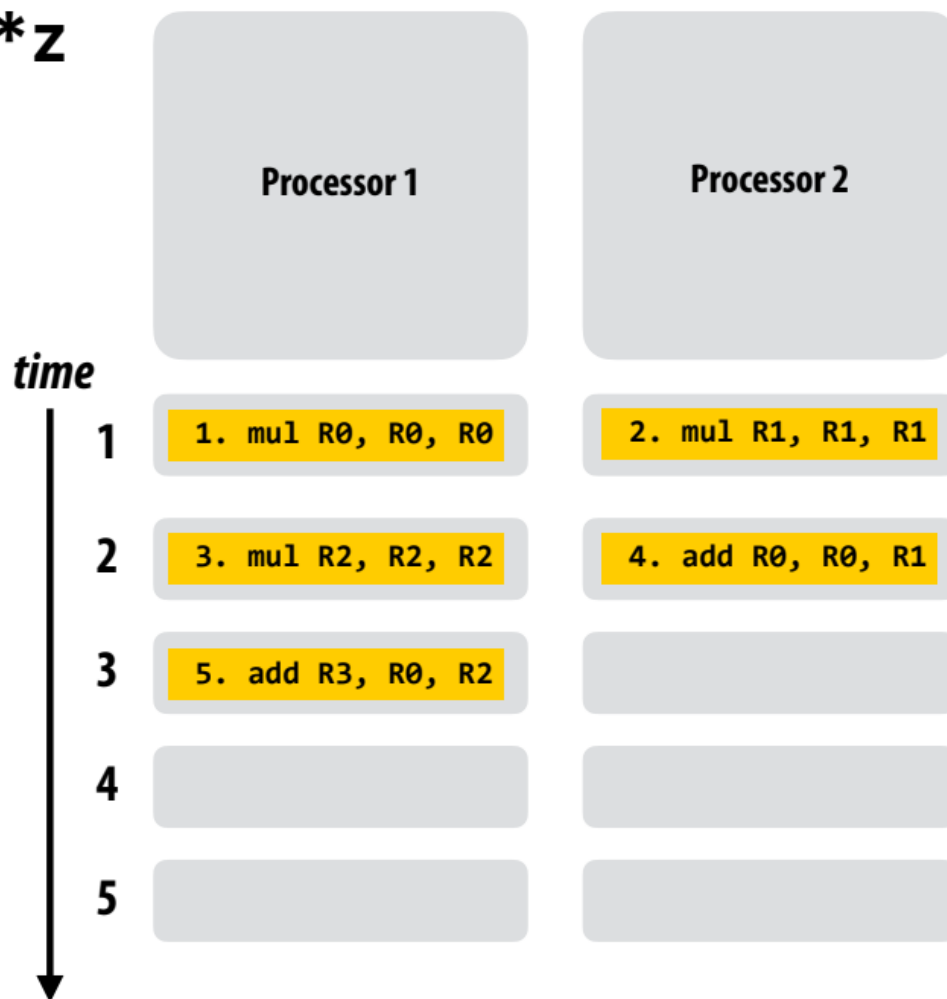
$$a = x * x + y * y + z * z$$

Assume register

R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of
program variable 'a'*



아주 간단한 코드를 예로 들어 보자

한 번에 최대 3개의 명령어를 수행할 수 있다면 어떻게 되나요?

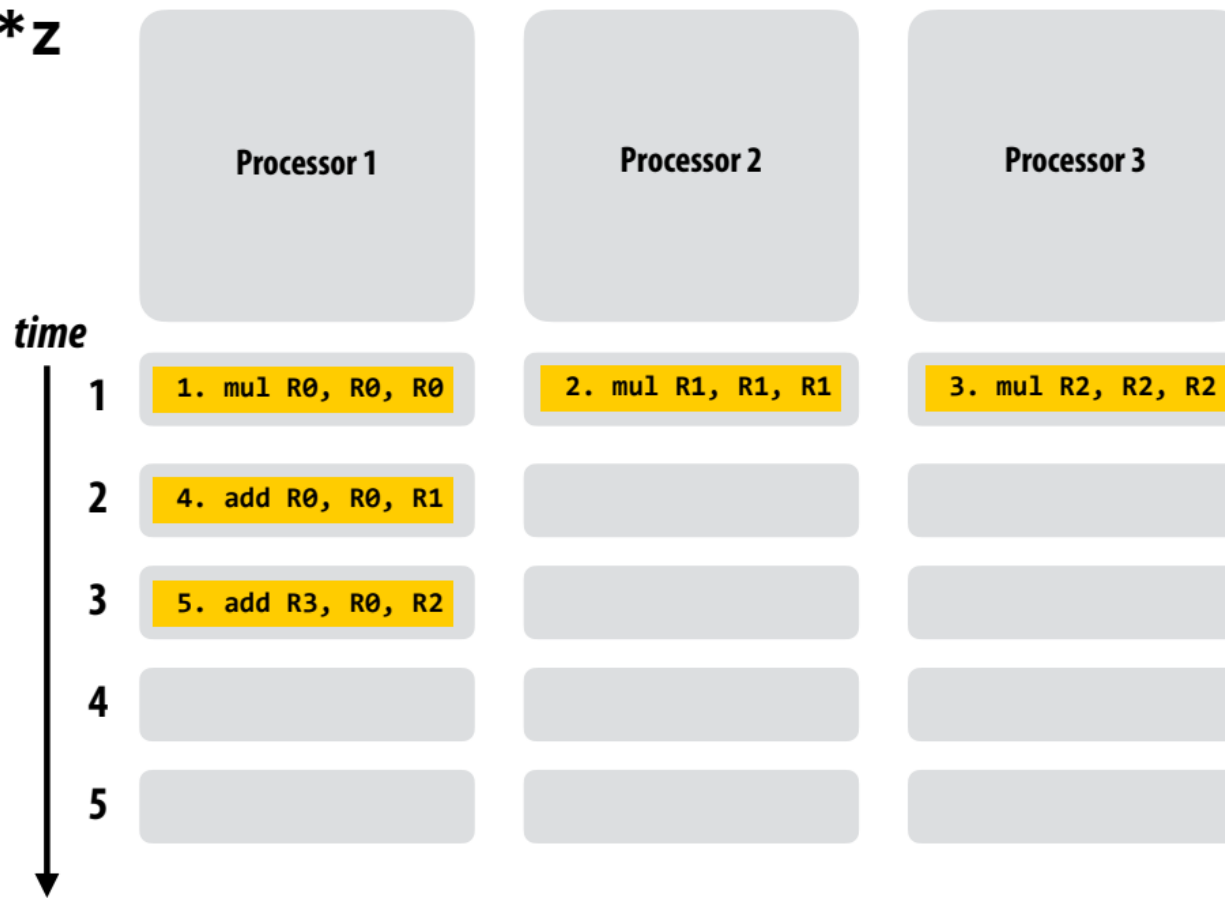
$$a = x*x + y*y + z*z$$

Assume register

R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

*R3 now stores value of
program variable 'a'*

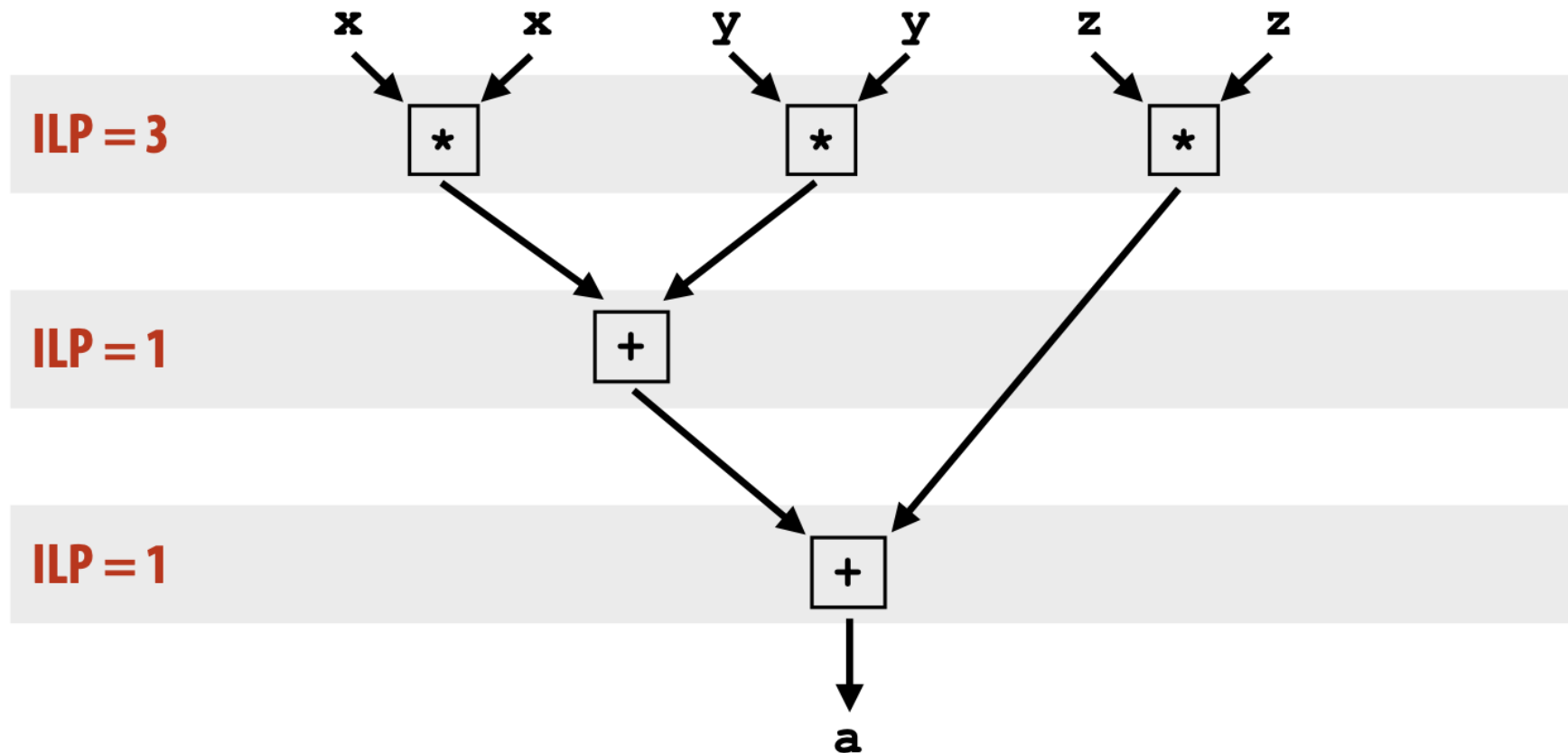


**“프로그램 순서를 존중한다”는 스케줄링에 맞
추어 병렬화 한다는 것은 무엇을 의미하나요?**

Instruction level parallelism (ILP) example

■ ILP = 3

$$a = x * x + y * y + z * z$$



•ILP 를 계산하여 같은 ILP를 갖는 연산들은 한 번에 같이 수행할 수 있지만 그렇지 않으면 기다려야한다.

Superscalar processor execution

$$a = x*x + y*y + z*z$$

Assume register

$R0 = x, R1 = y, R2 = z$

서로 독립적인 instructions 를 자동으로 찾아서 이를 multi-processes 에 잘 배분하여 수행함.

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

Idea #1:

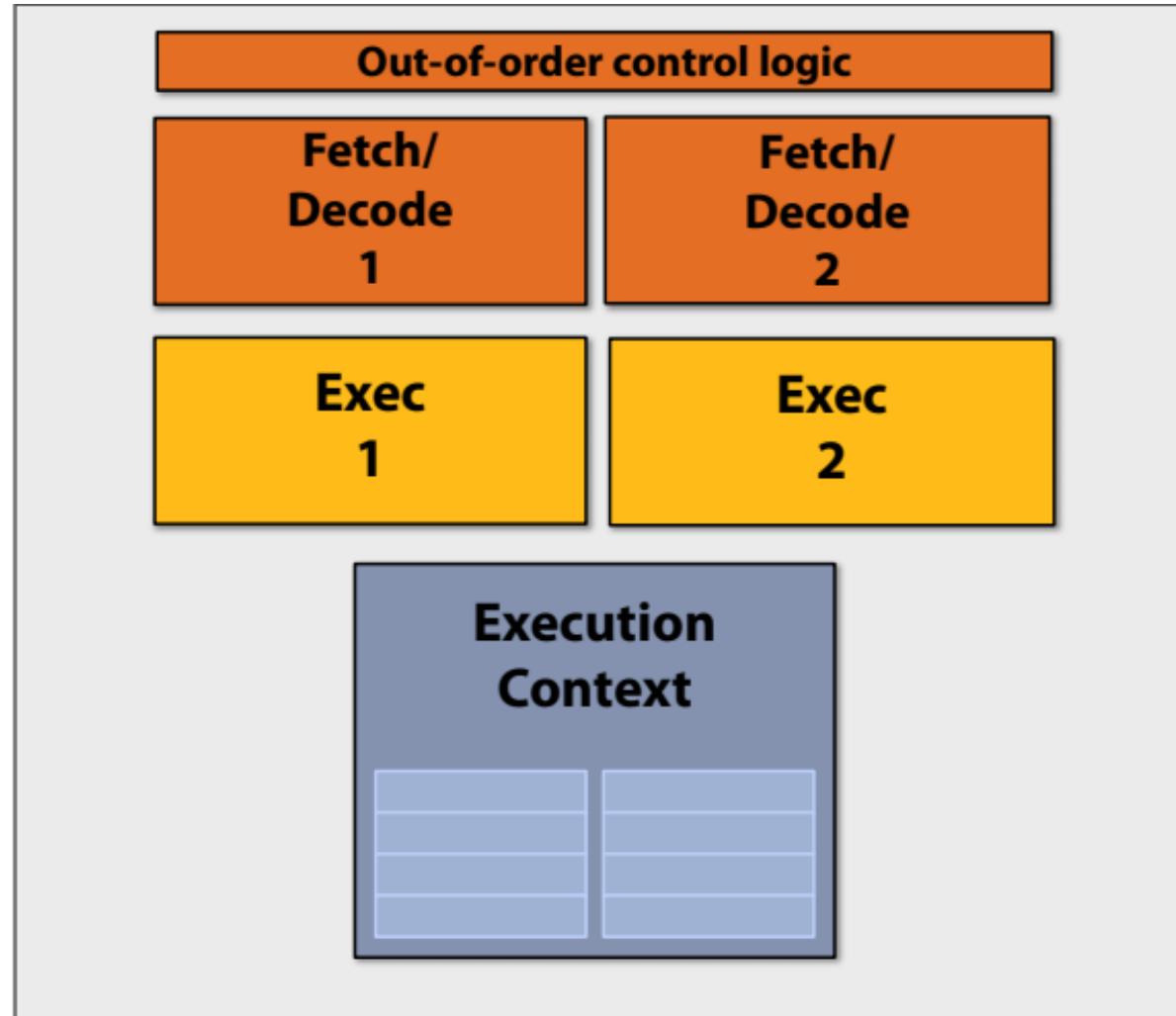
슈퍼스칼라 실행: 프로세서가 명령어 시퀀스에서 독립적인 명령어를 자동으로 찾아서 시퀀스에서 독립적인 명령어를 생성하고 여러 실행 단위에서 병렬로 실행할 수 있습니다.

- 이 예제에서는 명령어 1, 2, 3을 프로그램 정확성에 영향을 주지 않고 병렬로 실행 가능.
(슈퍼스칼라 프로세서상에서는 종속성이 존재하지 않는다고 판단).
- 하지만 명령어 4는 명령어 1과 2 다음에 실행되어야 함.
- 그리고 명령어 5는 명령어 4 이후에 실행되어야 함.

* 또는 컴파일 시 컴파일러가 독립적인 명령어를 컴파일하고 컴파일된 바이너리에 종속성을 명시적으로 인코딩함.

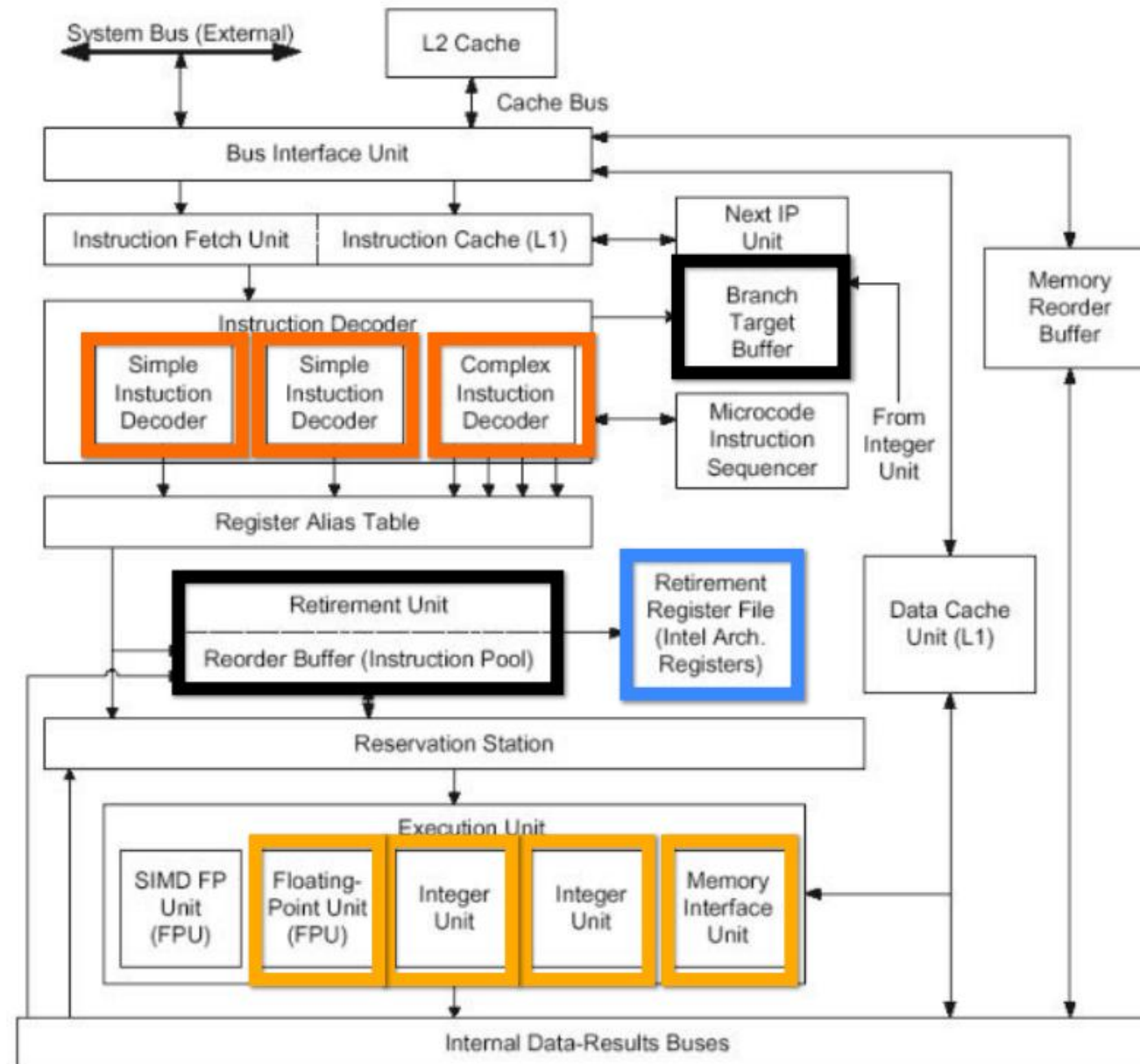
Superscalar processor

이 프로세서는 클럭당 최대 2개의 명령어를 디코딩하고 실행가능



Superscalar processor

Old Intel Pentium 4 CPU



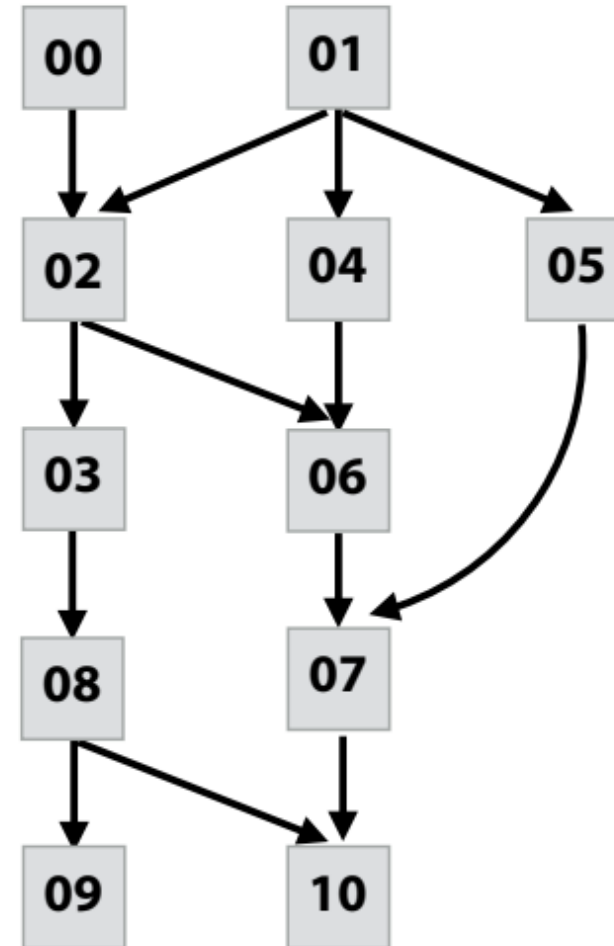
A more complex example

Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
10	else	
10	print tmp7	

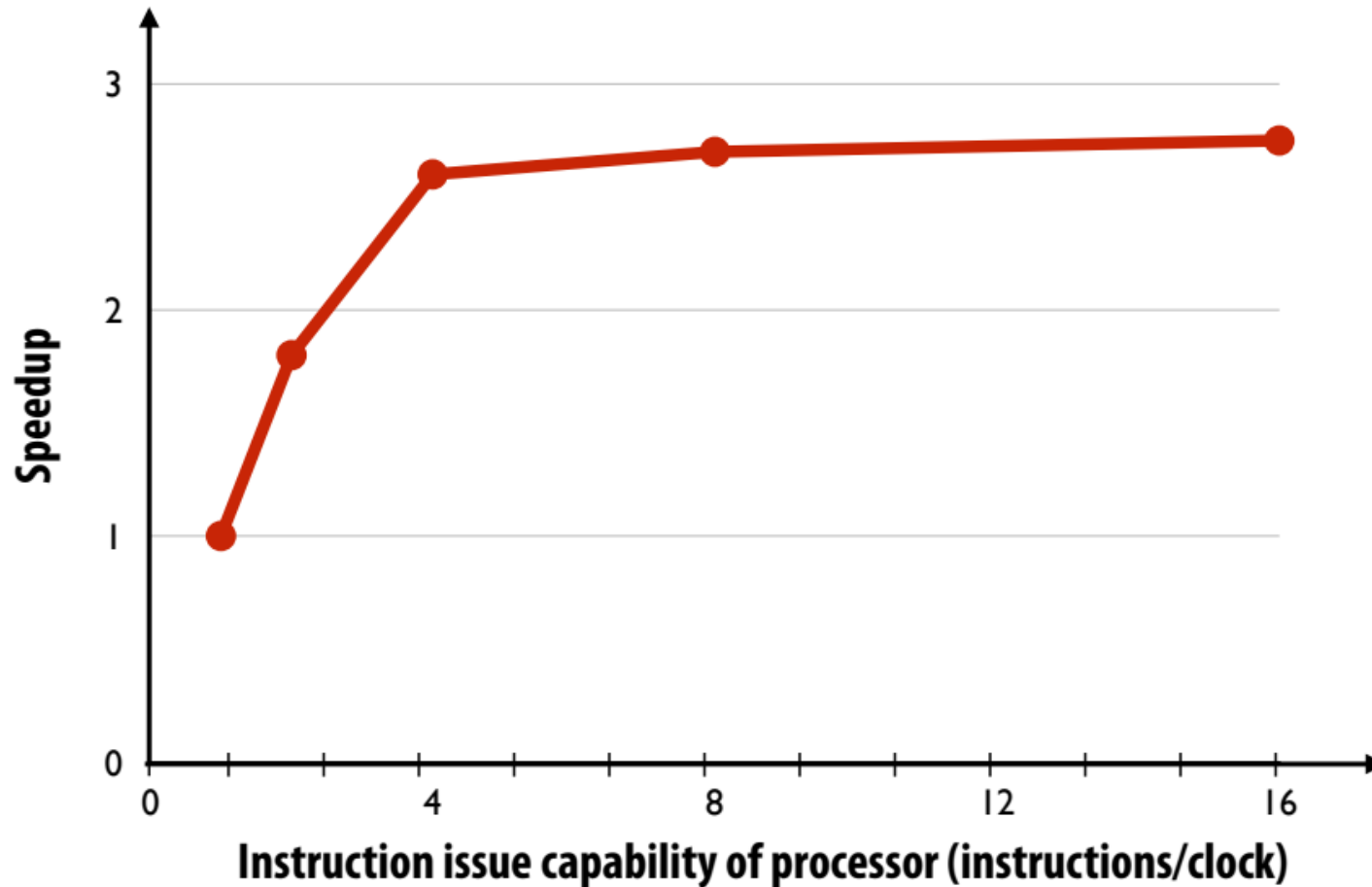
Computed value ↗

Instruction dependency graph



슈퍼스칼라 실행의 리턴(수익) 감소.

- 사용 가능한 대부분의 ILP는 클럭당 4개의 명령을 내릴 수 있는 프로세서에 의해 활용됨 (더 많이 발급할 수 있는 프로세서를 구축해도 성능 이점은 거의 없음.)



Our World
in Data

Our World
in Data

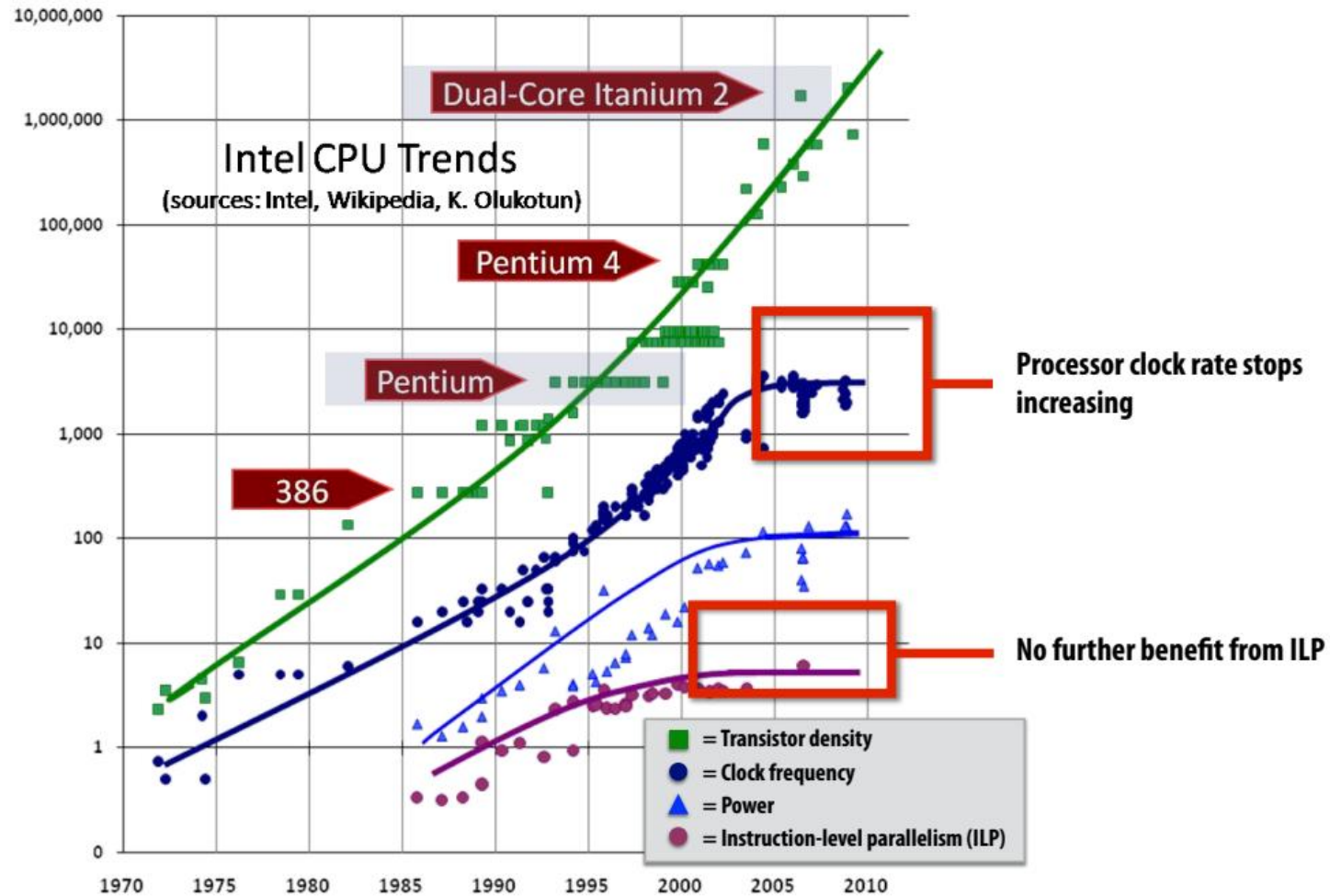
50,000,000,000



Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

ILP tapped out + end of frequency scaling

- 사용 가능한 대부분의 ILP는 클럭당 4개의 명령을 내릴 수 있는 프로세서에 의해 활용됨
(더 많이 발급할 수 있는 프로세서를 구축해도 성능 이점은 거의 없음.)



The “power wall”

Power consumed by a transistor:

Dynamic power $\propto$ capacitive load $\times$ voltage² $\times$ frequency

Static power: transistors burn power even when inactive due to leakage

High power = high heat

Power is a critical design constraint in modern processors



	<u>TDP</u>
Apple M1 laptop:	13W
Intel Core i9 10900K (in desktop CPU):	95W
NVIDIA RTX 4090 GPU	450W
Mobile phone processor	1/2 - 2W
World's fastest supercomputer	megawatts
Standard microwave oven	900W

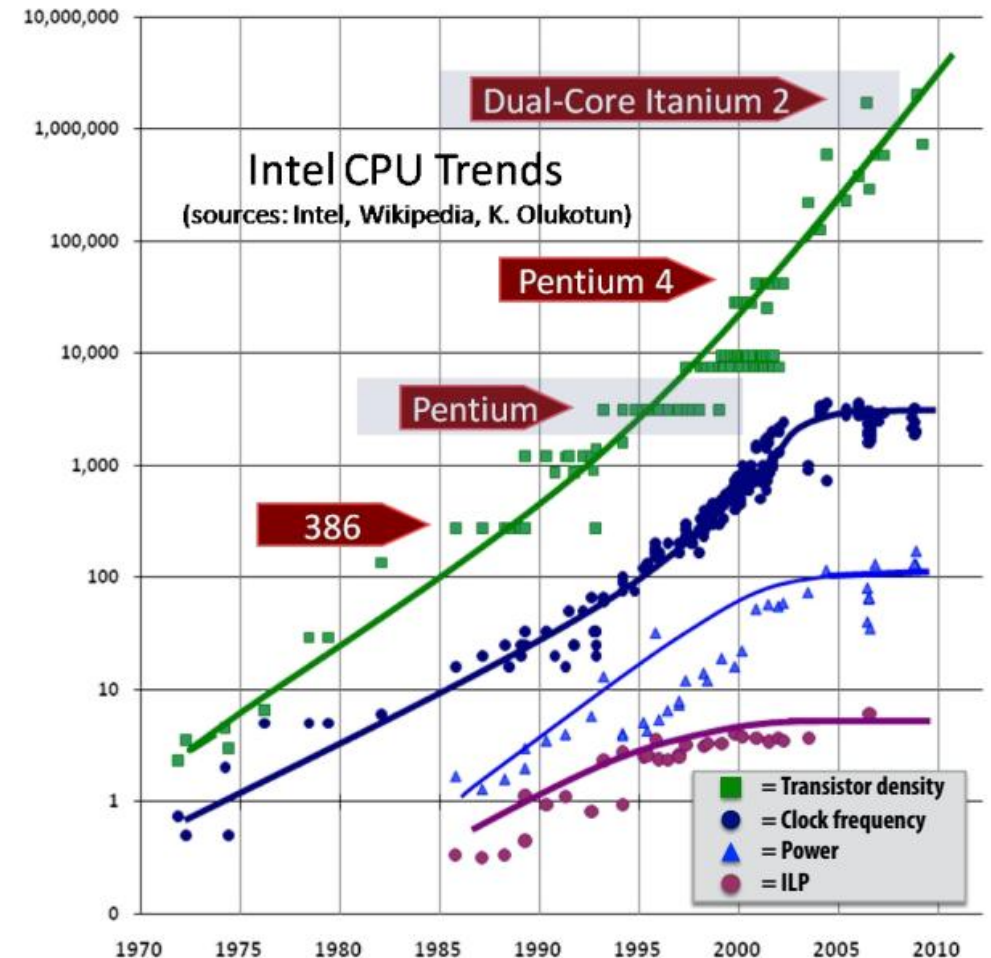


Single-core performance scaling

단일 명령어 스트림 성능 비율
스케일링이 감소했습니다(거의 0에 가까워짐).

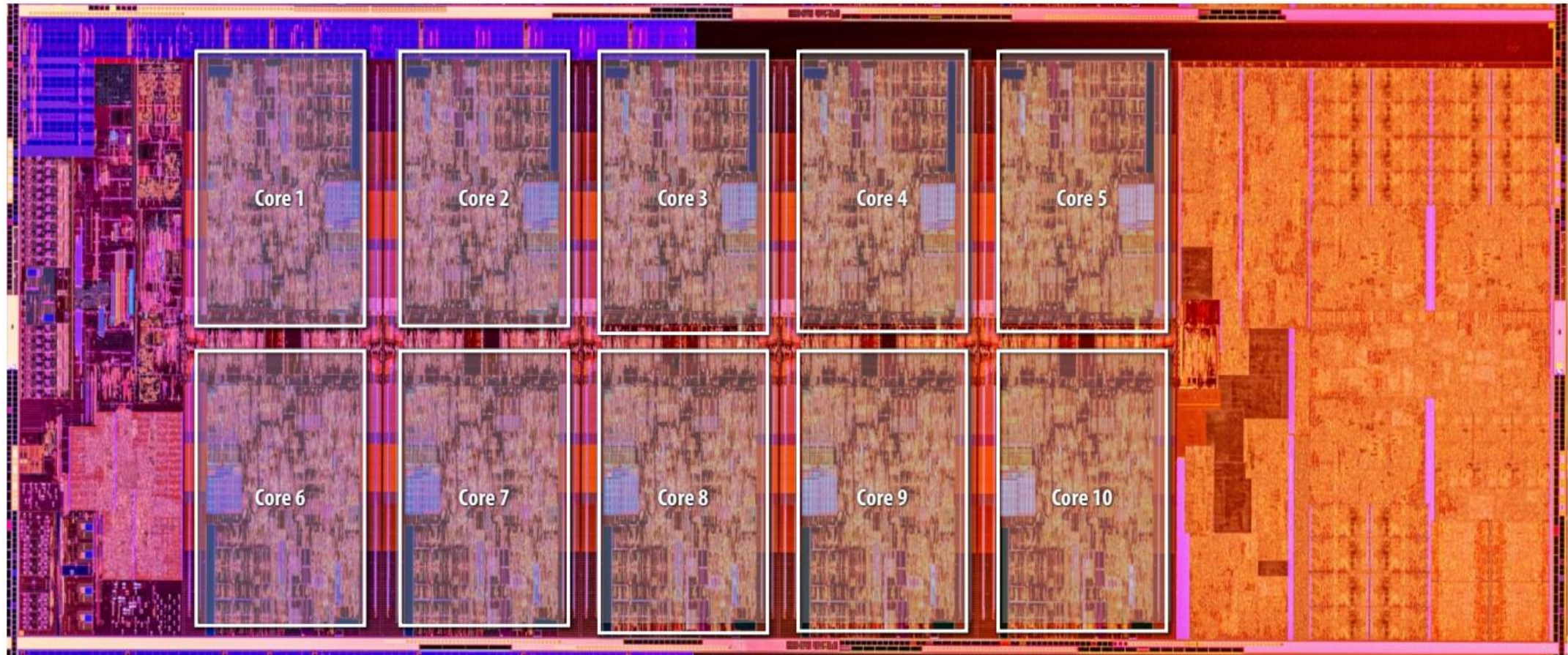
1. 전력에 의해 주파수 스케일링이 제한됨
2. ILP 스케일링 한계

- 이제 설계자들은 병렬로 실행되는 실행 유닛을 더 추가하여 더 빠른 프로세서를 구축하고 있습니다.
(혹은 그래픽 또는 오디오/비디오 재생과 같은 특정 작업에 특화된 유닛).
- 성능 향상을 보려면 소프트웨어를 병렬로 작성해야 함.
 - 소프트웨어 개발자에게 더 이상 공짜 점심은 없습니다!



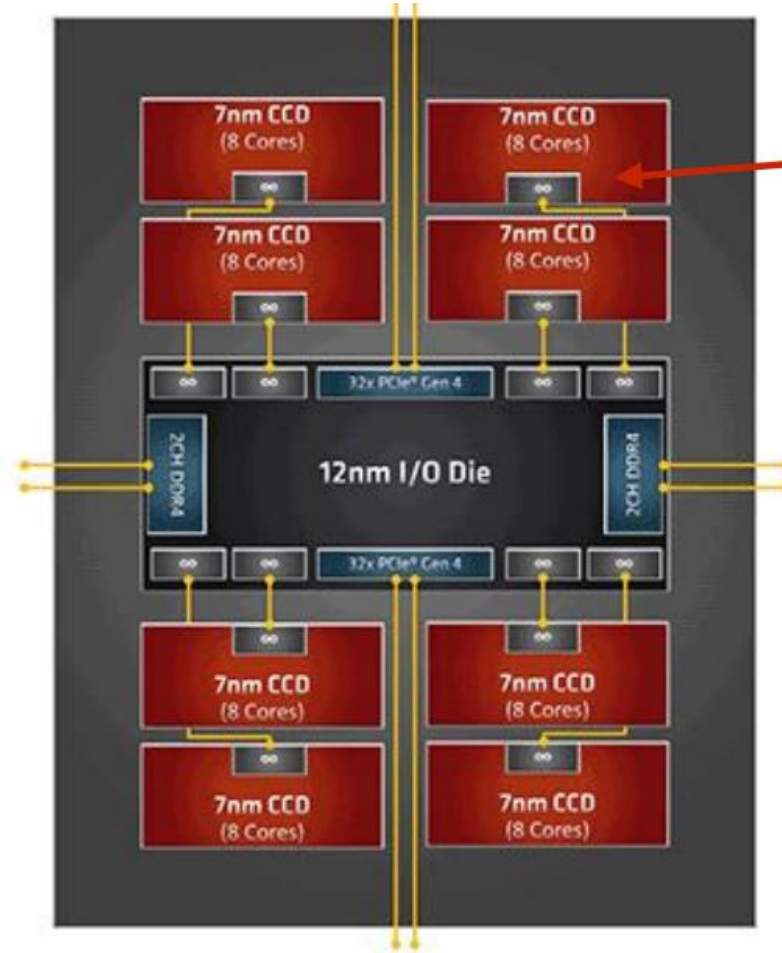
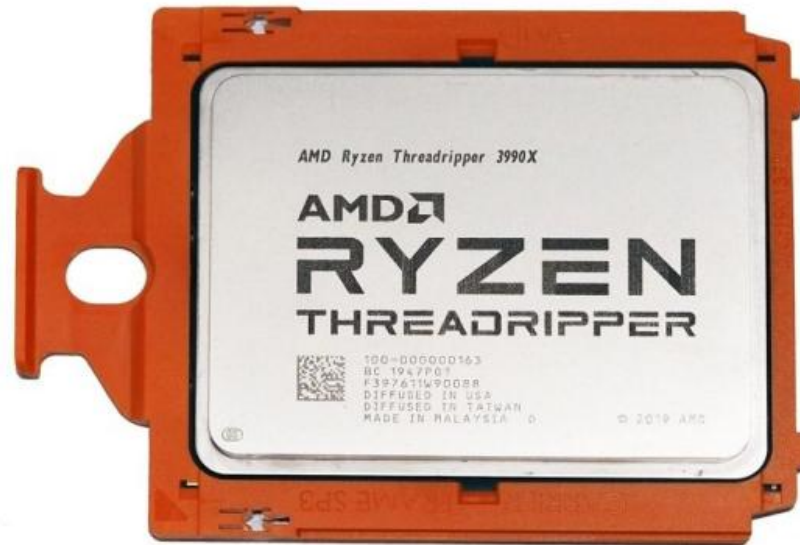
Example: multi-core CPU

Intel "Comet Lake" 10th Generation Core i9 10-core CPU (2020)



AMD Ryzen Threadripper 3990X

64 cores, 4.3 GHz



Four 8-core chiplets

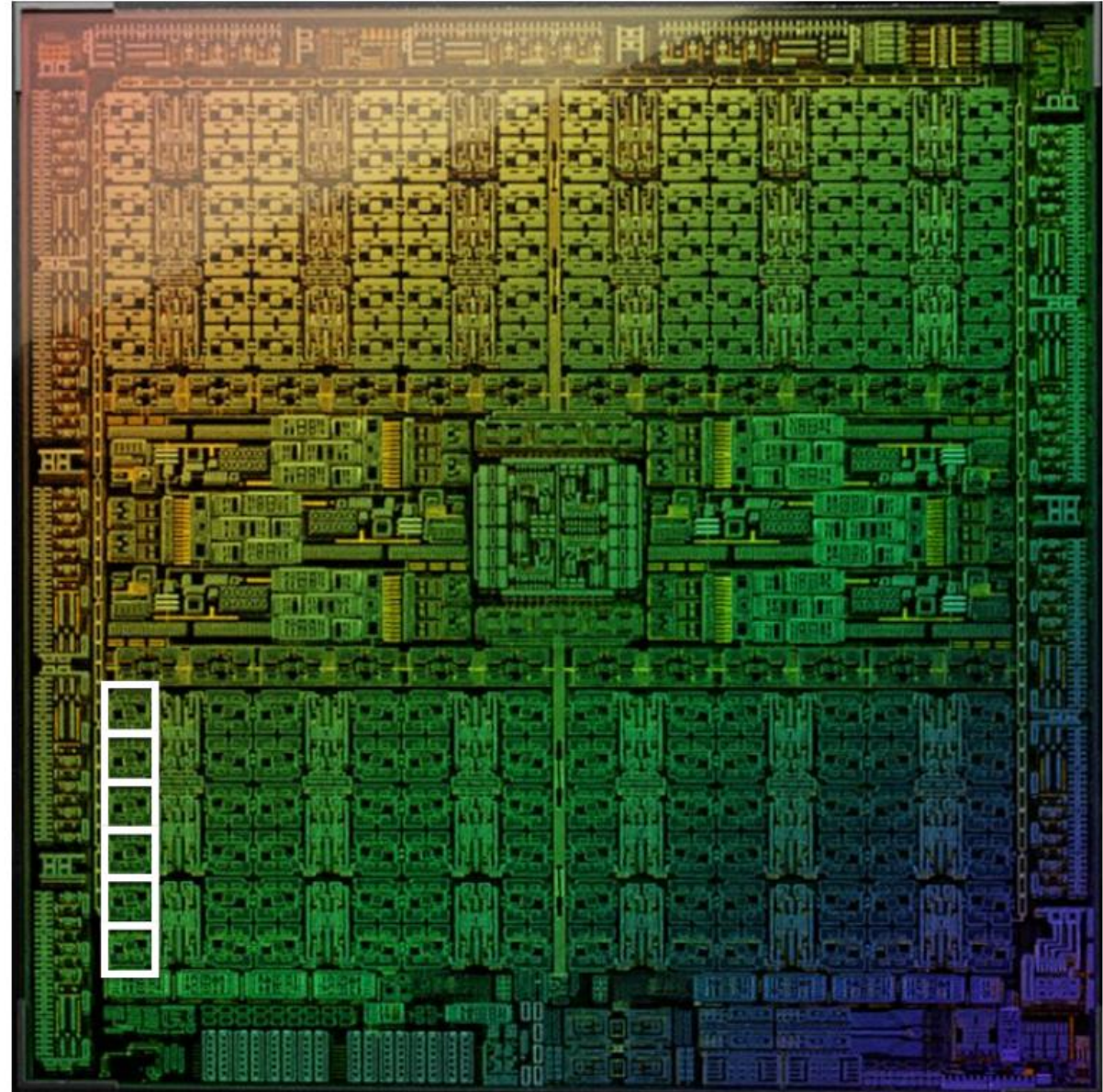
NVIDIA AD102 GPU

NVIDIA AD102 GPU

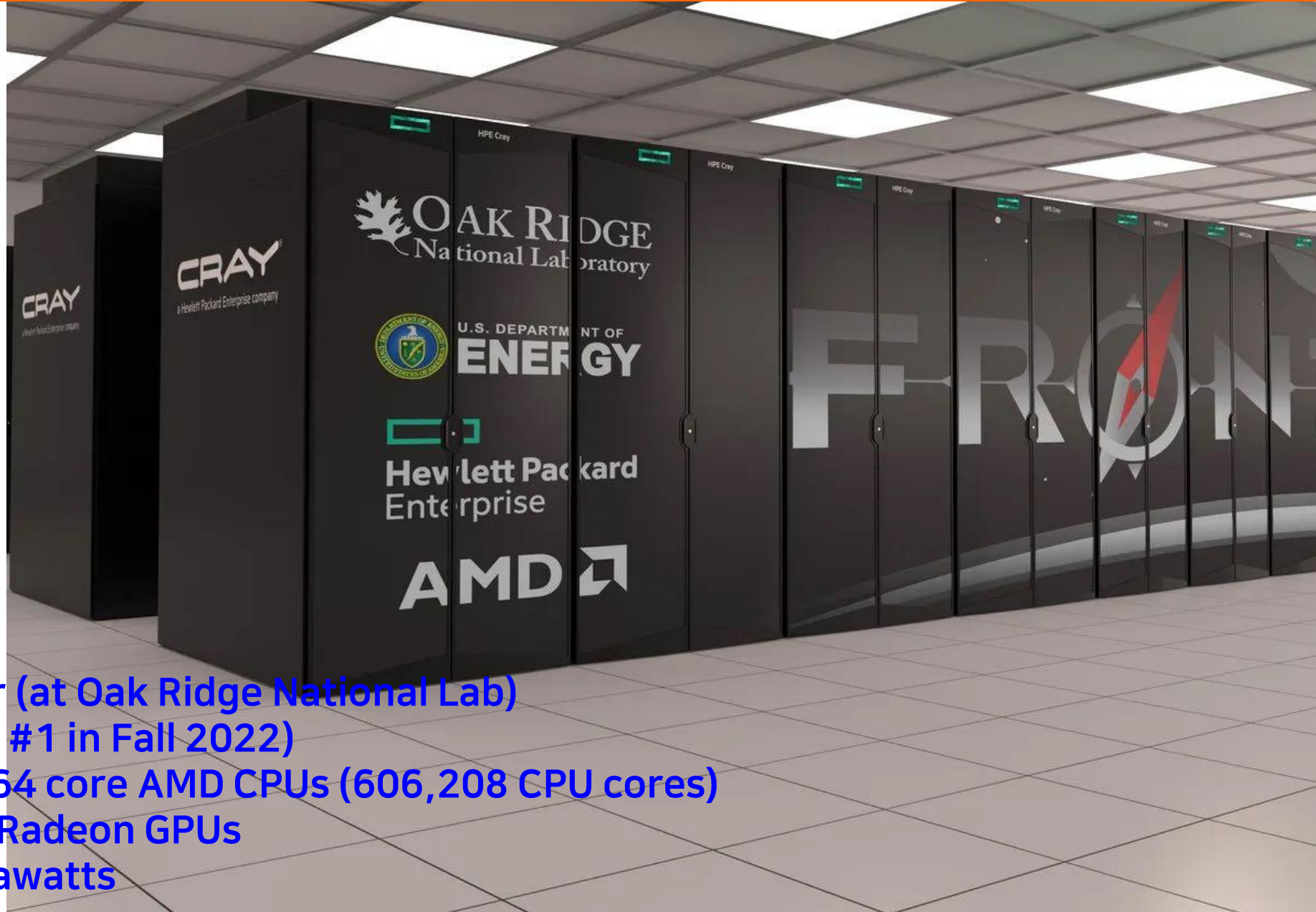
GeForce RTX 4090 (2022)

76 billion transistors

18,432 fp32 multipliers organized in
144 processing blocks (called SMs)



GPU-accelerated supercomputing



Frontier (at Oak Ridge National Lab)

(world's #1 in Fall 2022)

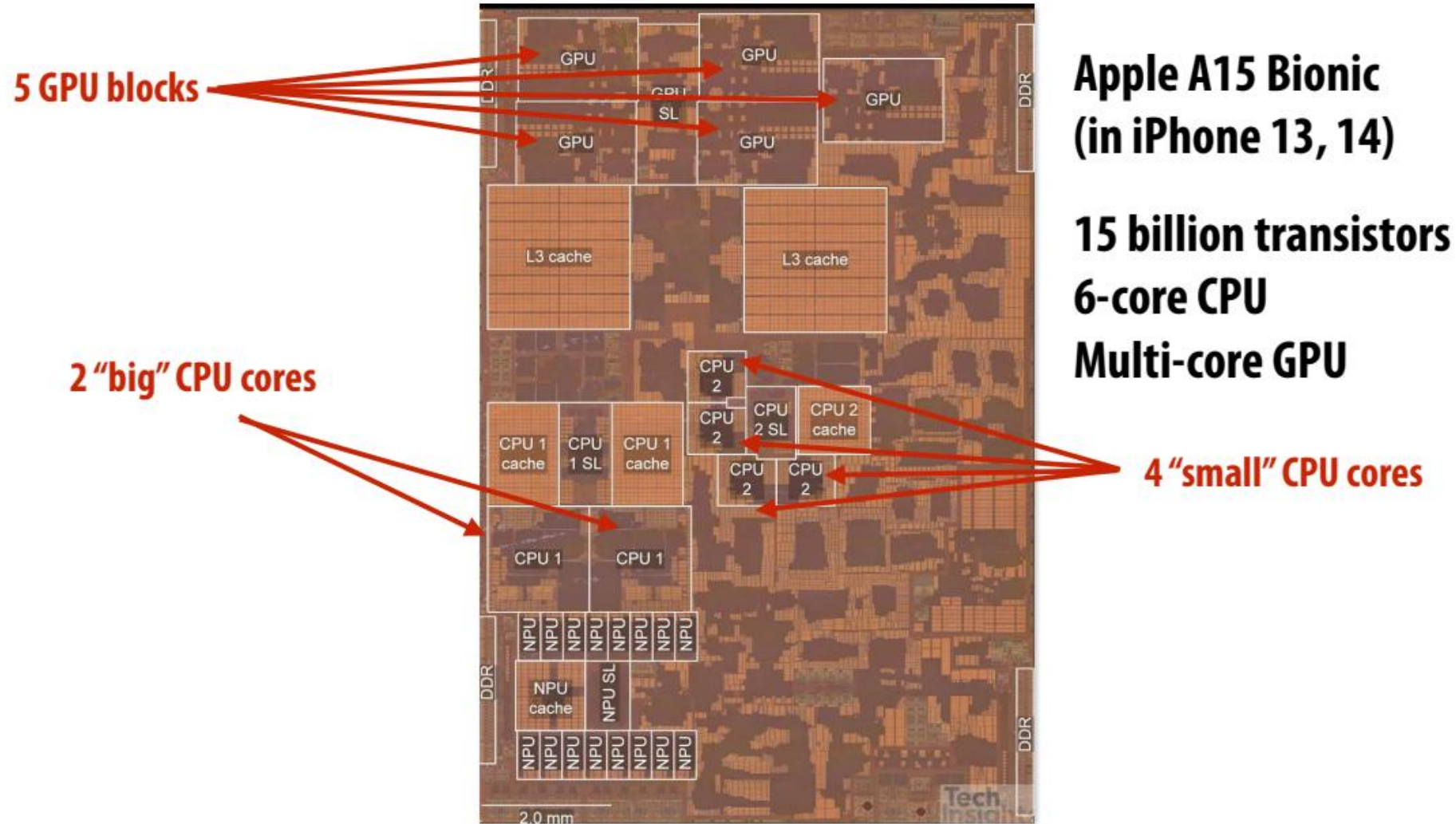
9472 x 64 core AMD CPUs (606,208 CPU cores)

37,888 Radeon GPUs

21 Megawatts

Mobile parallel processing

전력 제약은 모바일 시스템 설계에도 큰 영향을 미칩



Mobile parallel processing

Raspberry Pi 3

Quad-core ARM A53 CPU



그러나 현대 컴퓨팅에서 소프트웨어는 단순한 병렬 이상의 기능을 제공해야 합니다...

또한 효율적이어야 합니다

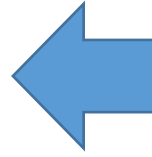
~~모바일~~ 컴퓨팅에서 가장 큰 관심사는 무엇인가요?

모든

A. Power

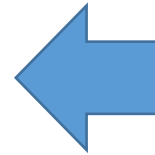
절전을 해야 하는 두 가지 이유

- 일정 시간 동안 더 높은 성능으로 실행



전력 = 열
칩이 너무 뜨거워지면 반드시 클럭을 낮추어 식혀야 함

- 더 오랜 시간 동안 충분한 성능으로 실행



전원 = 배터리
긴 배터리 수명은 모바일 디바이스에서 바람직한 기능

Mobile phone example

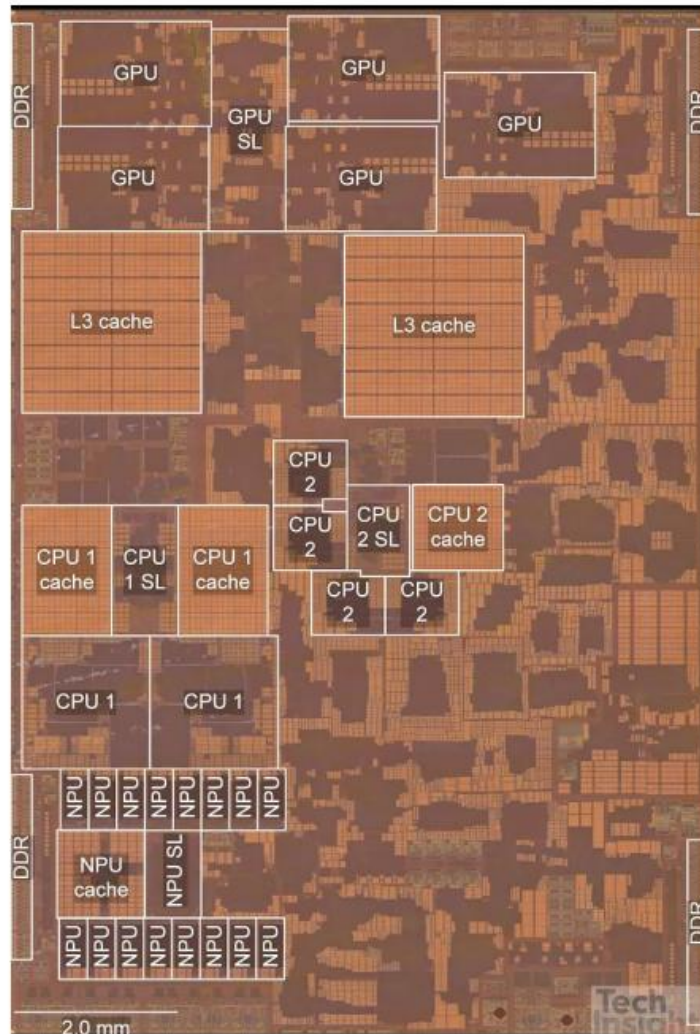
Apple iPhone 13



**3227 mAmp hours
(12.4 Watt hours)**

Mobile phone example

- 모바일 시스템에서는 특수 처리 (Specialized processing) 가 보편화되고 있음



Apple A15 Bionic (in iPhone 13, 14)

15 billion transistors

6-core GPU

2 "big" CPU cores

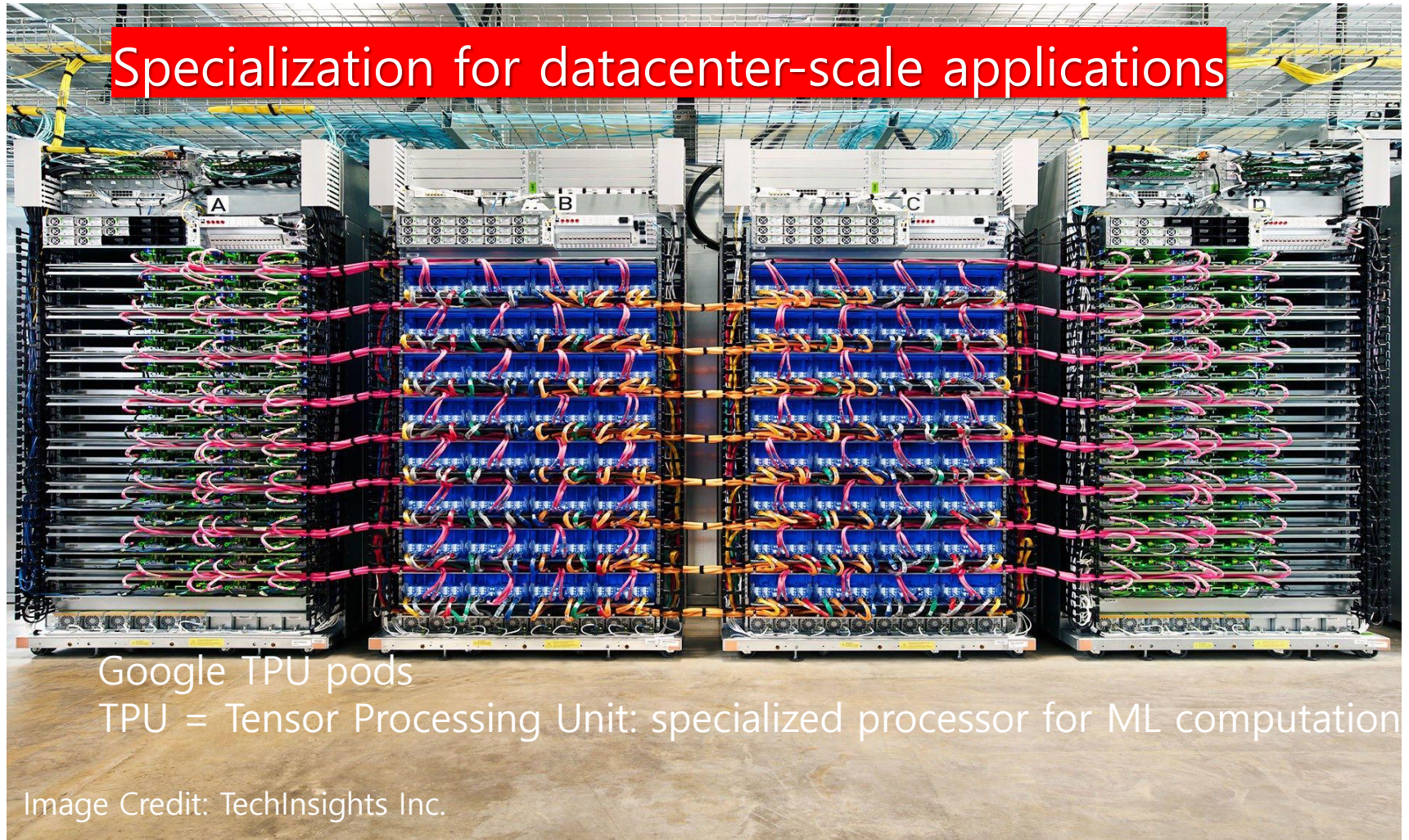
4 "small" CPU cores

Apple-designed multi-core GPU

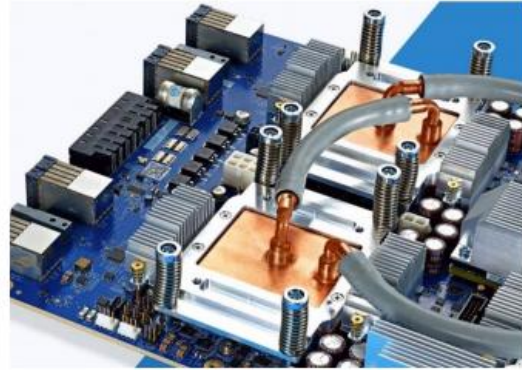
**Neural Engine (NPU) for DNN acceleration +
Image/video encode/decode processor +
Motion (sensor) processor**

병렬 + 특수 HW.

- 고효율 달성은 이 강의의 핵심 주제입니다.
- 최신 시스템은 많은 처리 장치를 사용할 뿐만 아니라 특수 처리 장치를 활용하여 높은 수준의 전력 효율을 달성하는 방법에 대해 학습



Specialized hardware to accelerate DNN inference/training



Google TPU3



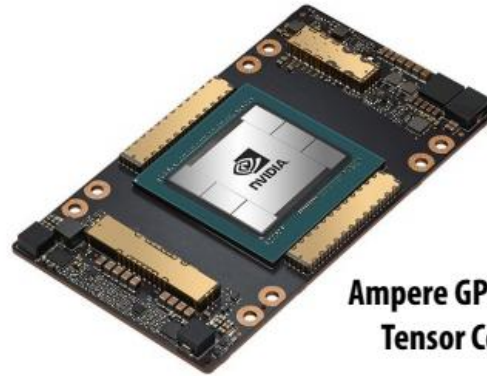
GraphCore IPU



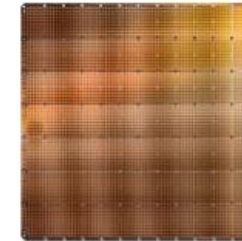
Apple Neural Engine



AWS Trainium



**Ampere GPU with
Tensor Cores**



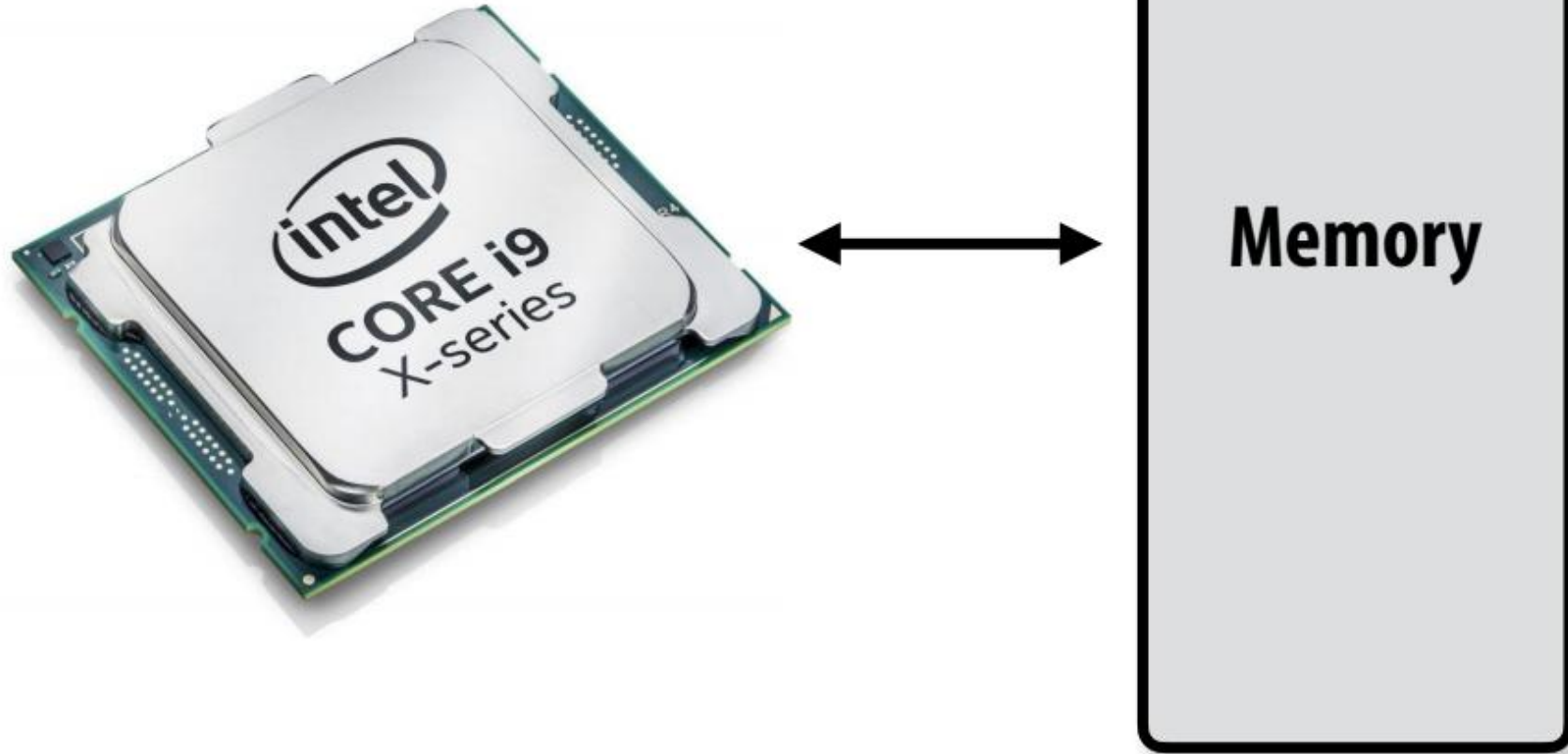
Cerebras Wafer Scale Engine



**SambaNova
Cardinal SN10**

효율적인 처리를 달성하는 것은 거의 항상 데이터에 효율적으로 액세스하는 데 달려 있습니다.

What is memory?



프로그램의 메모리 주소 공간

- 컴퓨터의 메모리는 바이트 배열로 구성
- 각 바이트는 메모리 내 '주소'로 식별
(이 배열에서의 위치)로 식별.
(메모리는 바이트 주소 지정이 가능하다고 가정.)

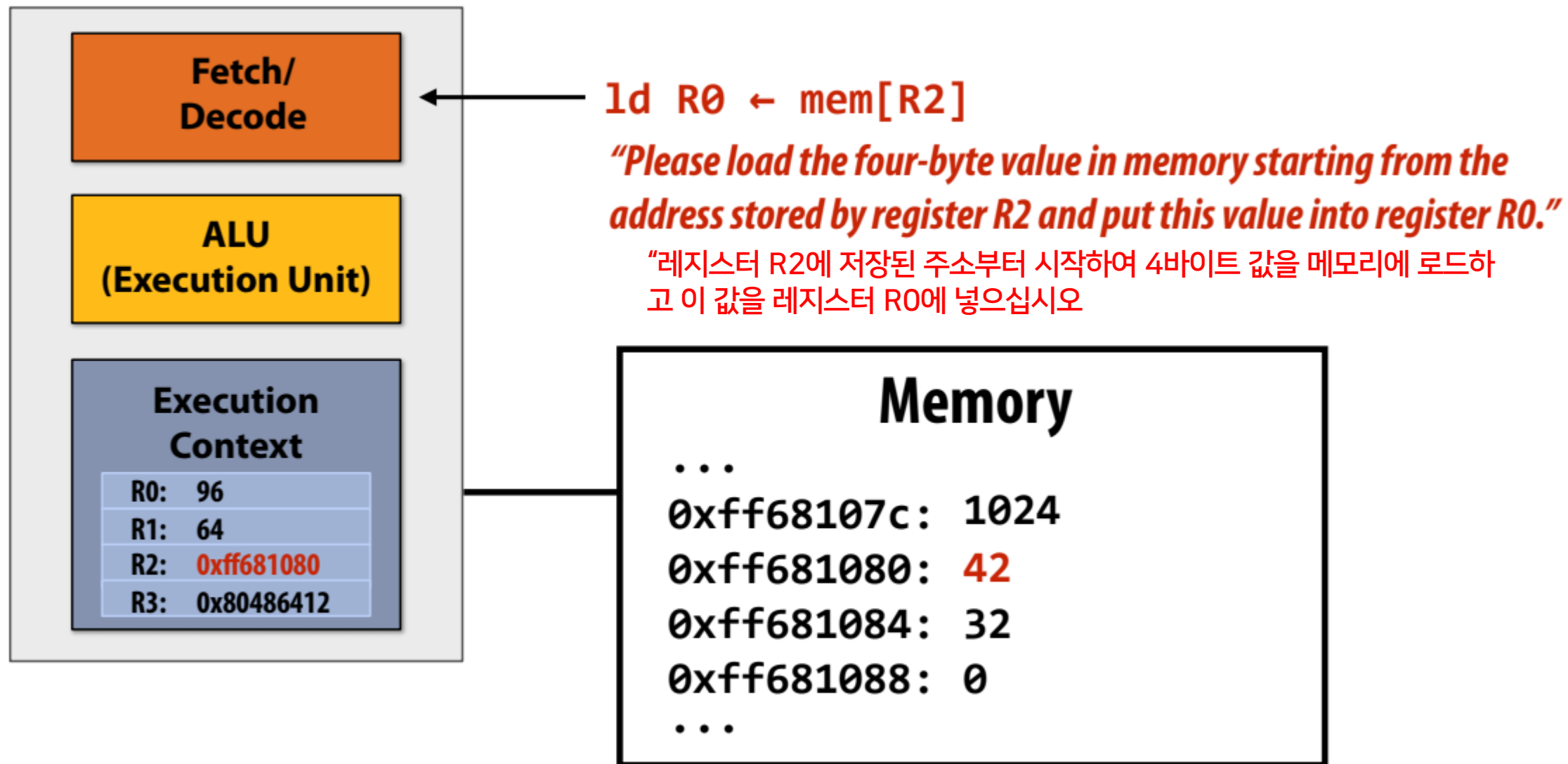
“0x8 주소에 저장된 바이트의 값은 32입니다.”

“주소 0x10(16)에 저장된 바이트의 값은 128입니다.”

오른쪽 그림에서 프로그램의 메모리 주소 공간은 32바이트 크기
(따라서 유효한 주소 범위는 0x0에서 0x1F까지).

Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0
0x10	128
⋮	⋮
0x1F	0

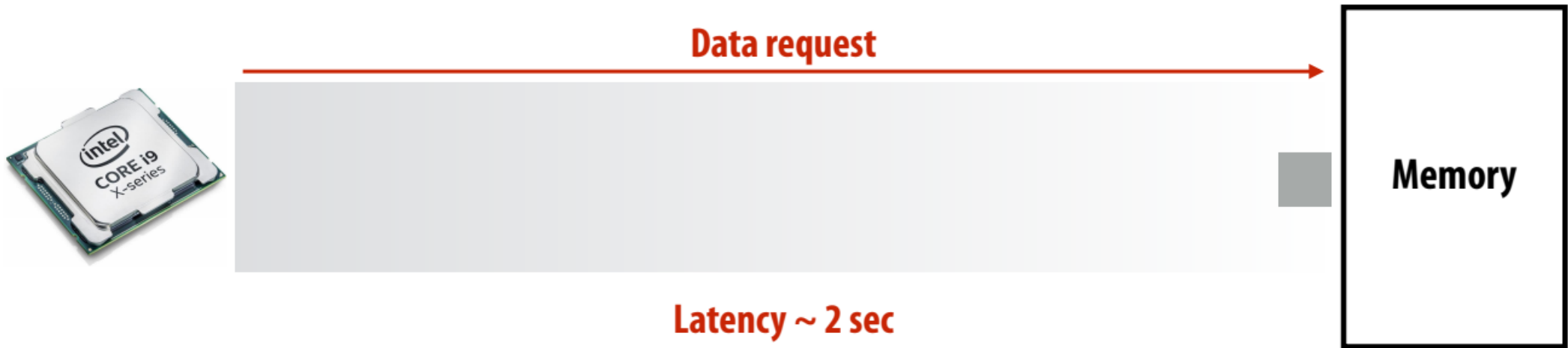
Load: 메모리 내용(the contents of memory)에 액세스하기 위한 명령어



Terminology

- Memory access latency

- 메모리 시스템이 프로세서에 데이터를 제공하는 데 걸리는 시간.
- Example: 100 clock cycles, 100 nsec



- 프로세서는 다음 명령어가 아직 완료되지 않은 이전 명령어에 의존하기 때문에 명령어 스트림에서 다음 명령어를 실행할 수 없을 때 “멈춤(Stalls)”(진행이 불가능)됩니다.
- 메모리 액세스는 지연(Stalls)의 주요 원인

```
ld r0 mem[r2]  
ld r1 mem[r3]  
add r0, r0, r1
```



종속성(Dependency): mem[r2] 및 mem[r3]의 데이터가 메모리에서 로드될 때까지 'add' 명령을 실행할 수 없음

- Memory access times ~ 100's of cycles
 - 메모리 '액세스 시간'은 지연 시간을 측정하는 척도

- 프로세서는 다음 명령어가 아직 완료되지 않은 이전 명령어에 의존하기 때문에 명령어 스트림에서 다음 명령어를 실행할 수 없을 때 "멈춤(Stalls)"(진행이 불가능)됩니다.

- 메모리 액세스는 지연(Stalls)의 주요 원인

```
ld r0 mem[r2]
ld r1 mem[r3]
add r0, r0, r1
```

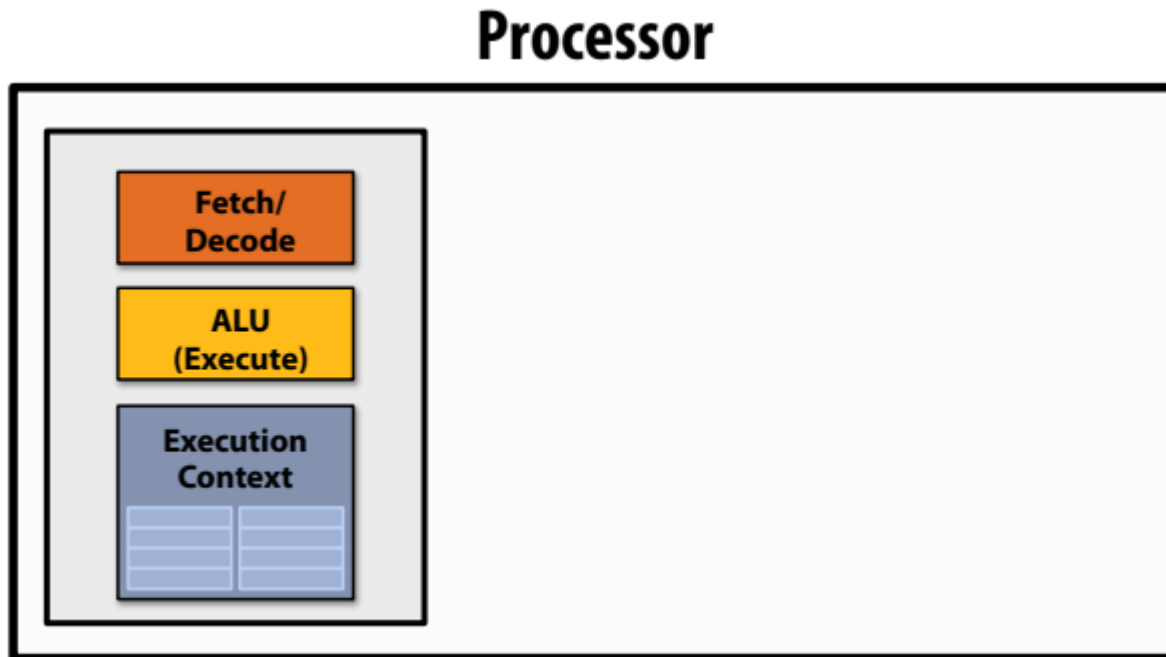


종속성(Dependency): mem[r2] 및 mem[r3]의 데이터가 메모리에서 로드될 때까지 'add' 명령을 실행할 수 없음

- Memory access times ~ 100's of cycles
 - 메모리 '액세스 시간'은 지연 시간을 측정하는 척도

What are caches?

- 리콜 메모리는 값의 배열일 뿐
- 그리고 프로세서에는 메모리에서 레지스터로 데이터를 이동(로드)하고 레지스터에서 메모리로 데이터를 저장(스토어)하는 명령어가 있음



Memory

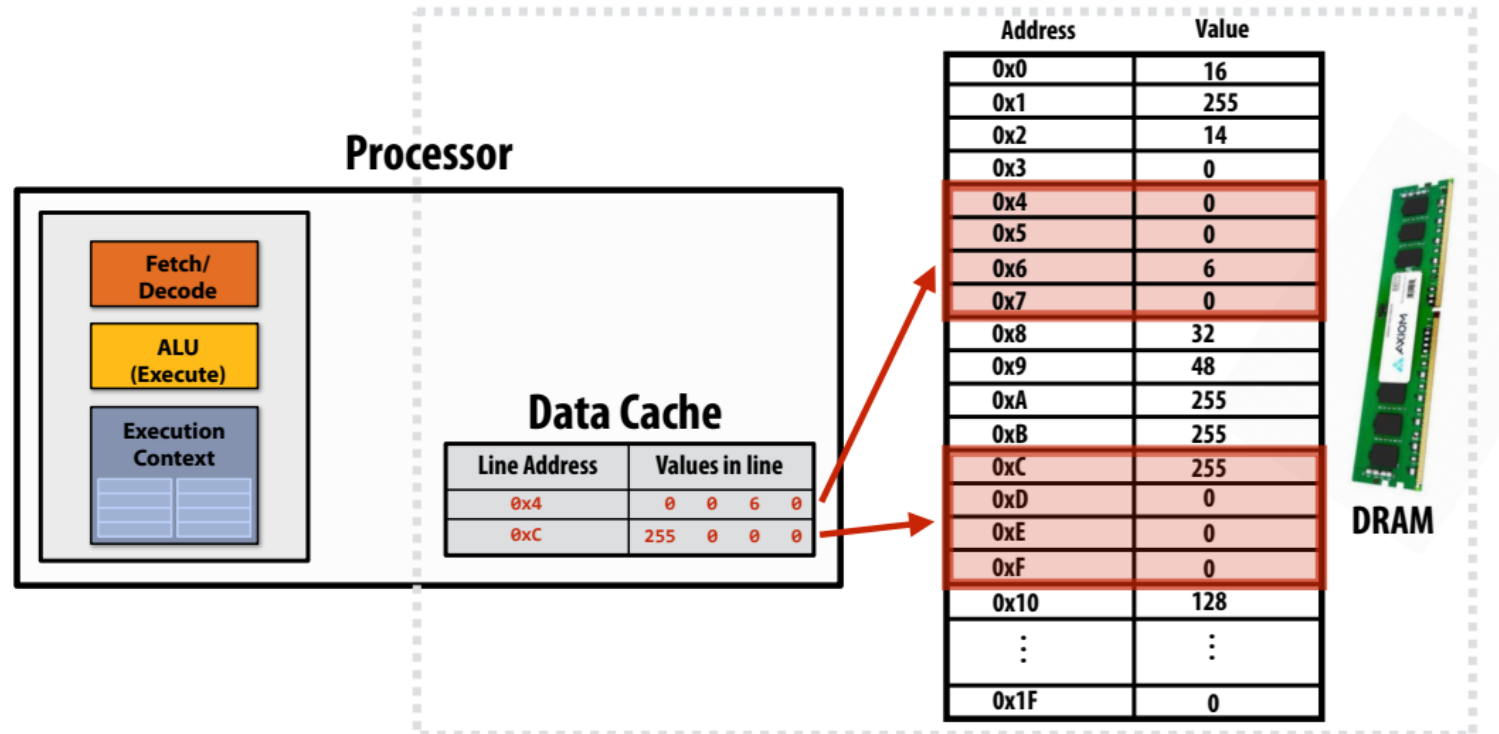
Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0
0x10	128
⋮	⋮
0x1F	0

What are caches?

- 캐시는 프로그램의 출력에는 영향을 주지 않고 성능에만 영향을 주는 하드웨어 구현 세부 사항.
- 캐시는 메모리에 값의 subset 사본을 유지하는 **온칩 스토리지**.
- Address(주소)가 “캐시에” 저장되면 프로세서는 데이터가 DRAM에만 있는 경우보다 더 빠르게 이 주소로 로드/저장할 수 있음.
- 캐시는 '캐시 라인'이라는 세분화(granularity)된 단위로 작동.

Implementation of memory abstraction

- Has a capacity of 2 lines
- Each line holds 4 bytes of data



프로세서는 어떤 데이터를 캐시에 보관할지 어떻게 결정하나요?

- 이 강좌의 범위를 벗어나지만 다음 용어를 구글에서 검색해 보시기 바랍니다...
 - 직접 매핑 캐시
 - 집합 연관 캐시
 - 캐시 라인
- 지금은 N바이트 크기의 캐시가 마지막으로 액세스한 N개의 주소에 대한 값을 저장한다고 가정합니다. - LRU 교체 정책("가장 최근에 사용된") - 새 데이터를 위한 공간을 확보하기 위해 가장 오래 전에 액세스한 데이터를 캐시에 있는 데이터를 버립니다.

Cache example 1

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:
Total cache capacity of 8 bytes
Cache with 4-byte cache lines
(So 2 lines fit in cache)
Least recently used (LRU)
replacement policy

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ● ● ● ●
0x1	hit	0x0 ● ● ● ●
0x2	hit	0x0 ● ● ● ●
0x3	hit	0x0 ● ● ● ●
0x2	hit	0x0 ● ● ● ●
0x1	hit	0x0 ● ● ● ●
0x4	"cold miss", load 0x4	0x0 ● ● ● ● 0x4 ● ● ● ●
0x1	hit	0x0 ● ● ● ● 0x4 ● ● ● ●

time ↓

이 시퀀스에는 두 가지 형태의 "데이터 로컬리티"가 있음:

- ① 공간적 위치: 캐시 라인에 데이터를 로드하면 같은 라인의 다른 주소에 대한 후속 액세스에 필요한 데이터를 '미리 로드'하여 캐시 히트로 이어짐.
- ② 시간적 위치: 동일한 주소에 반복적으로 액세스하면 히트가 발생.

Cache example 2

Array of 16 bytes in memory

Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0

Line 0x0

Line 0x4

Line 0x8

Line 0xC

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

time

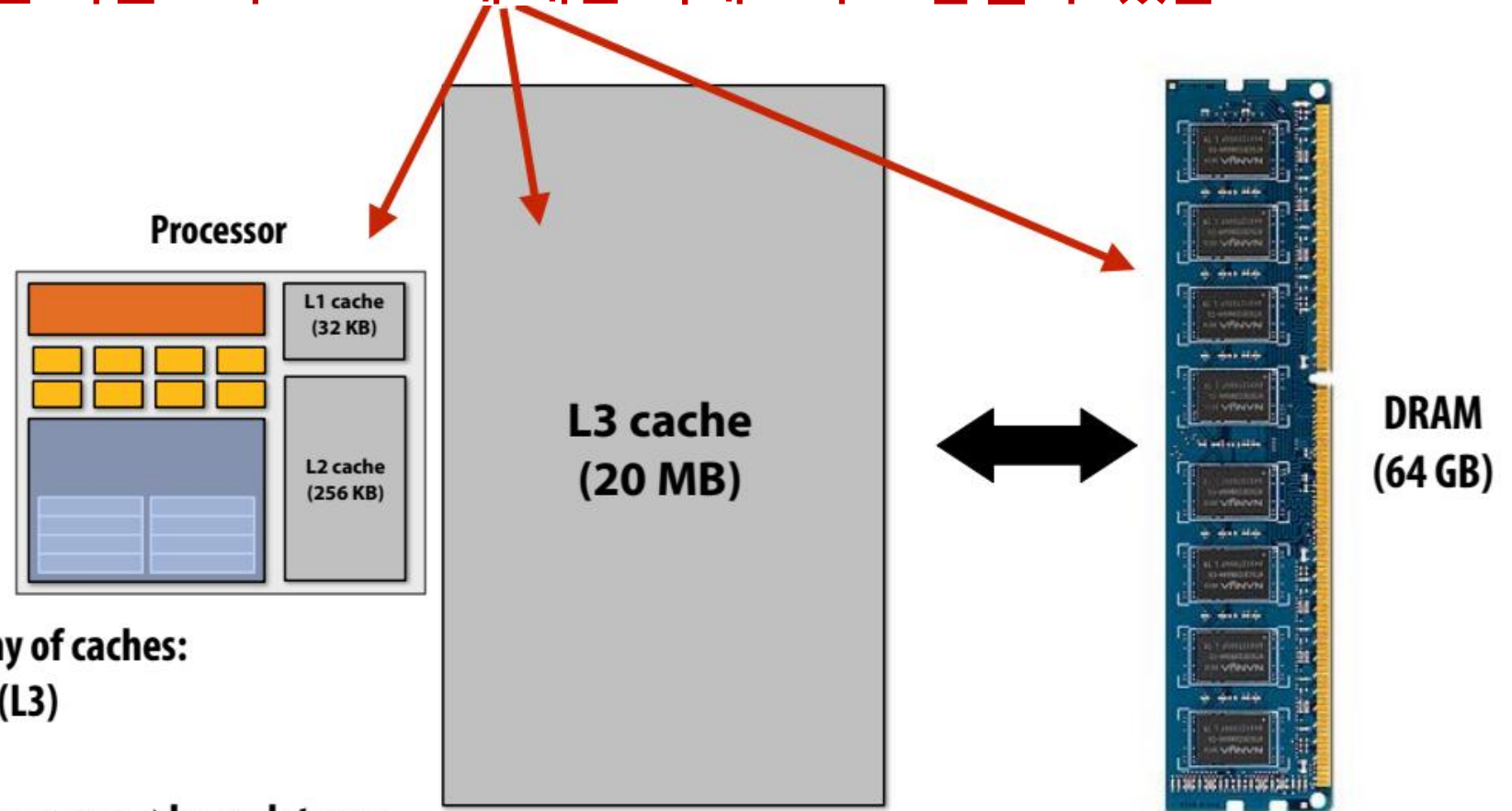
Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ●●●●
0x1	hit	0x0 ●●●●
0x2	hit	0x0 ●●●●
0x3	hit	0x0 ●●●●
0x4	"cold miss", load 0x4	0x0 ●●●● 0x4 ●●●●
0x5	hit	0x0 ●●●● 0x4 ●●●●
0x6	hit	0x0 ●●●● 0x4 ●●●●
0x7	hit	0x0 ●●●● 0x4 ●●●●
0x8	"cold miss", load 0x8 (evict 0x0)	0x8 ●●●● 0x4 ●●●●
0x9	hit	0x8 ●●●● 0x4 ●●●●
0xA	hit	0x8 ●●●● 0x4 ●●●●
0xB	hit	0x8 ●●●● 0x4 ●●●●
0xC	"cold miss", load 0xC (evict 0x4)	0x8 ●●●● 0xC ●●●●
0xD	hit	0x8 ●●●● 0xC ●●●●
0xE	hit	0x8 ●●●● 0xC ●●●●
0xF	hit	0x8 ●●●● 0xC ●●●●
0x0	"capacity miss", load 0x0 (evict 0x8)	0x0 ●●●● 0xC ●●●●

캐시로 지연 시간 감소(메모리 액세스 지연 시간 감소)

- 프로세서는 캐시에 상주하는 데이터에 액세스할 때 효율적으로 실행됩니다.
- 캐시는 프로세서가 최근에 액세스한 데이터에 액세스할 때 메모리 액세스 대기 시간을 줄입니다. 메모리 액세스 지연을 줄입니다! *

최신 컴퓨터에서 선형 메모리 주소 공간을 추상화하는 것은 복잡함

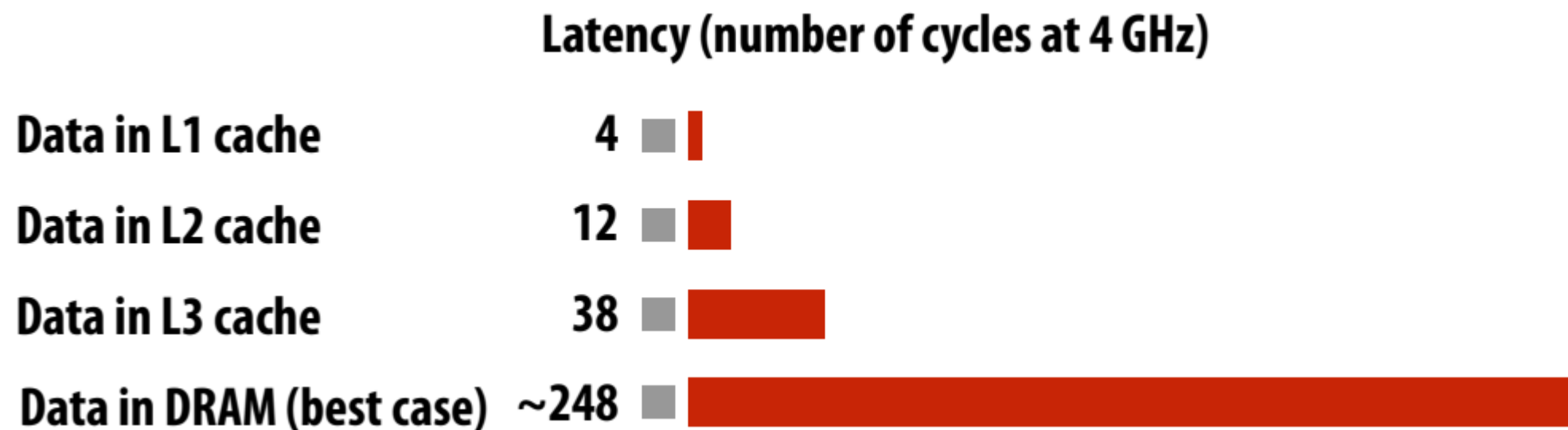
“주소 X에 저장된 값을 레지스터 R0에 로드하라”는 명령에는 여러 데이터 캐시에 의한 복잡한 연산 시퀀스와 DRAM에 대한 액세스가 포함될 수 있음.



Common organization: hierarchy of caches:
Level 1 (L1), level 2 (L2), level 3 (L3)

Smaller capacity caches near processor → lower latency
Larger capacity caches farther away → larger latency

Data access times(Kaby Lake CPU)



데이터 이동에는 높은 에너지 비용이 소요됨

- 최신 시스템 설계의 경험 법칙: 항상 컴퓨터에서 데이터 이동량을 줄이려고 노력
- “야구장” 숫자
 - Integer op: $\sim 1 \mu\text{J}^*$
 - Floating point op: $\sim 20 \mu\text{J}^*$
 - Reading 64 bits from small local SRAM (1mm away on chip): $\sim 26 \mu\text{J}$
 - Reading 64 bits from low power mobile DRAM (LPDDR): $\sim 1200 \mu\text{J}$
- Implications
 - Reading 10 GB/sec from memory: $\sim 1.6 \text{ watts}$
 - Entire power budget for mobile GPU $\sim 1 \text{ watt}$
(remember phone is also running CPU, display, radios, etc.)
 - iPhone 6 battery: $\sim 7 \text{ watt-hours}$ (note: my Macbook Pro laptop: 99 watt-hour battery)
 - Exploiting locality matters!!!

[Sources: Bill Dally (NMDA), Tom Olson (ARM)]

*명령어 디코딩, 레지스터에서 데이터 로드 등의 오버헤드를 계산하지 않고 논리적 연산만 수행하는 데 드는 비용입니다.

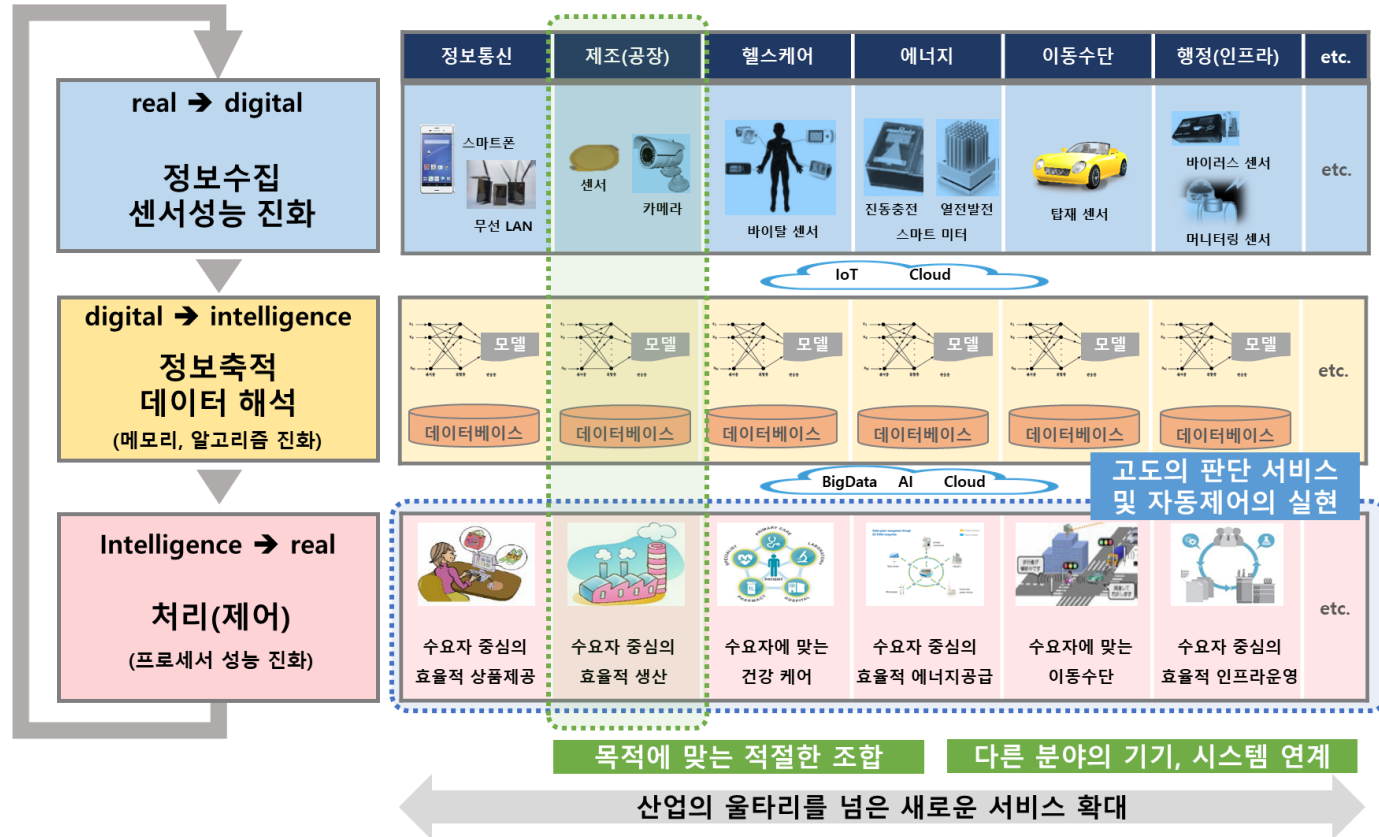
1. 왜 병렬(분산)처리 인가

2) IT기술로 새로운 비즈니스사이클 출현

데이터 활용 관점에서 모든 분야의 경쟁 영역이 변화

데이터수집 → 해석 → 처리 사이클

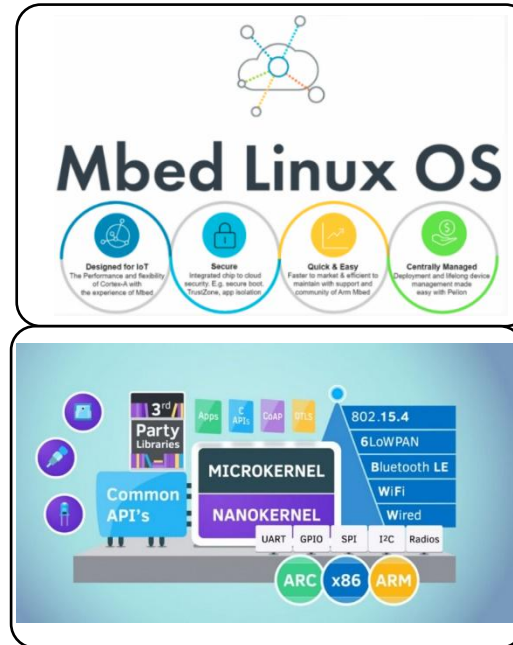
인더스트리 4.0으로 대표되는 새로운 비즈니스사이클



1. 왜 병렬(분산)처리인가

3) 4차 산업 혁명의 핵심 기술: Linux 기반으로 운영

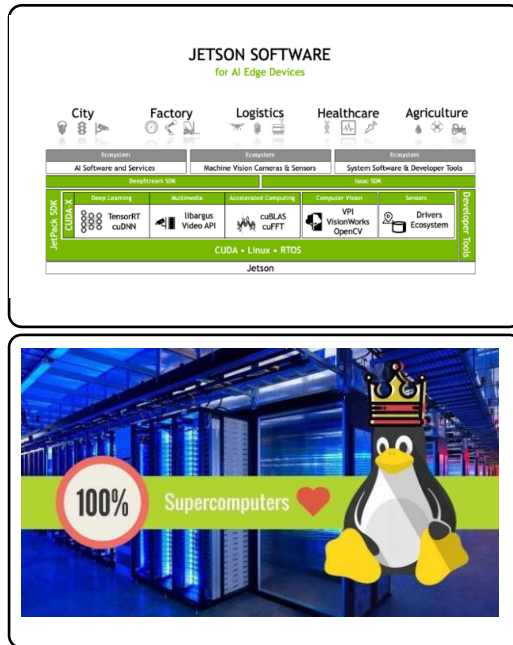
IOT Computing



Cloud Computing

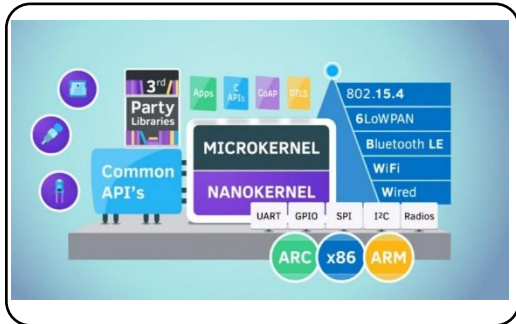
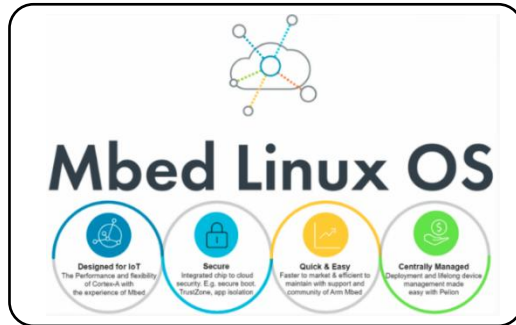


AI Computing



3) 4차 산업 혁명의 핵심 기술: Linux 기반으로 운영

IOT Computing



Cortex-M를 구동시킬 수 있는 OS가 Mbed OS 입니다.

ARM

- ARM은 (Advanced RISC Machine) 약자이며 1985년 영국 캠브릿지 대학의 연구진들이 시작한 벤처 기업으로 시작한 회사.
- ARM은 반도체 회사지만 반도체를 직접 만들지 않는 팹리스 회사이며, MCU의 아키텍처 개발만 하는 전문 회사.
- 타 반도체 회사에서 ARM IP(아키텍처)를 갖고 자신만의 반도체를 만드는 회사가 대표적으로 TI, STM, 삼성, 프리스케일, Nvidia, 퀄컴 등이 있음.
- 그리고, ARM은 2016년에 일본 소프트뱅크에서 36조원 전액 현금으로 인수한 회사.
- 통계적으로 전세계 스마트폰의 90% 이상이 ARM 반도체로 만들어져 있음.
- 스마트폰 뿐만 아니라, 태블릿, 스마트워치, 저장장치 컨트롤러, 차량용 메인 컨트롤러, 무선통신기기 등 다양한 사업군의 기기에서 ARM 반도체로 사용
- 다양한 산업군에 사용되고 있는 ARM Architecture의 종류로는 Cortex A, Cortex R, Cortex M 포트폴리오로 갖고 있음.

1. 왜 병렬(분산)처리 인가

3) 4차 산업 혁명의 핵심 기술: Linux 기반으로 운영

Cloud Computing



가상화

가상화는 단일한 물리 하드웨어 시스템에서 여러 시뮬레이션 환경이나 전용 리소스를 생성할 수 있는 기술

- 가상화는 하이퍼바이저라 불리는 소프트웨어가 하드웨어에 직접 연결되며 1개의 시스템을 **가상 머신(VM)**이라는 별도의 고유하고 안전한 환경으로 분할.
- 이러한 VM은 하이퍼바이저의 기능을 사용하여 머신의 리소스를 하드웨어에서 분리한 후 적절하게 배포

클라우드

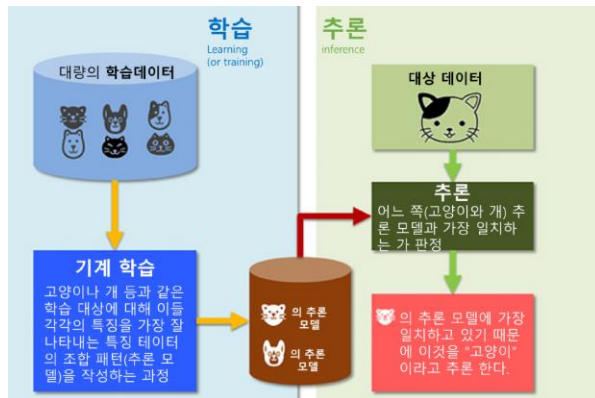
클라우드는 네트워크 전체에서 확장 가능한 리소스를 추상화하고 풀링하는 IT 환경



1. 왜 병렬(분산)처리 인가

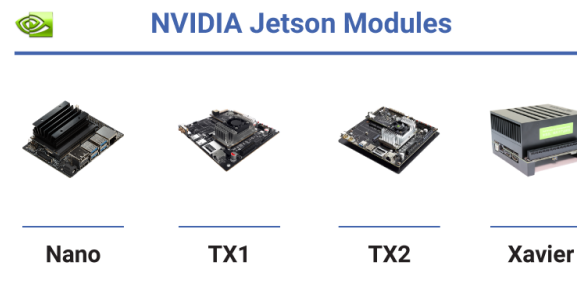
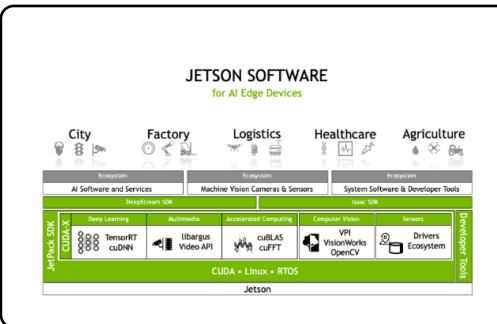
3) 4차 산업 혁명의 핵심 기술: Linux 기반으로 운영

AI Computing



학습

추론



Linux OS

1. 왜 병렬(분산)처리 인가

3) 4차 산업 혁명의 핵심 기술: Linux 기반으로 운영



1. 클라우드 비용

2. 대기업의 리눅스 채택

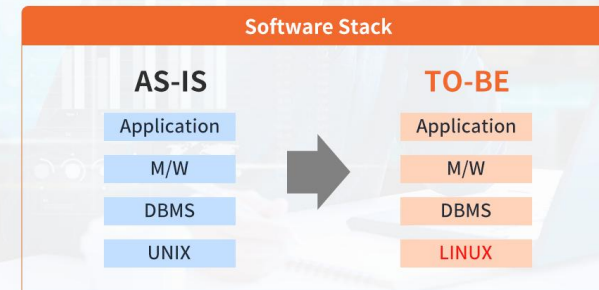
3. 사물인터넷
(Internet of Things)

4. 보안 문제
(Security concerns)

5. 비용 민감도
(Cost Sensitivity)

미래형 운영체제?

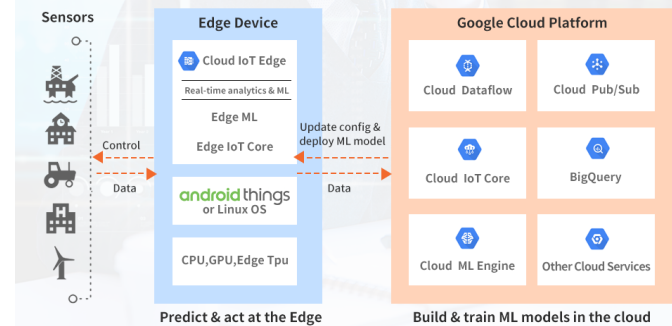
U2L이란?



<출처 : 티맥스 U2L 전환 >

U2L이란 유닉스 플랫폼 환경(하드웨어, OS, DBMS, 미들웨어, 애플리케이션 등)을 리눅스 환경으로 마이그레이션(Migration)하는 방법론

리눅스 기반 IoT엣지 컴퓨팅



<출처 : google cloud iot edge >

1. 왜 병렬(분산)처리 인가

2) UNIX일화



2) UNIX / Linux 역사

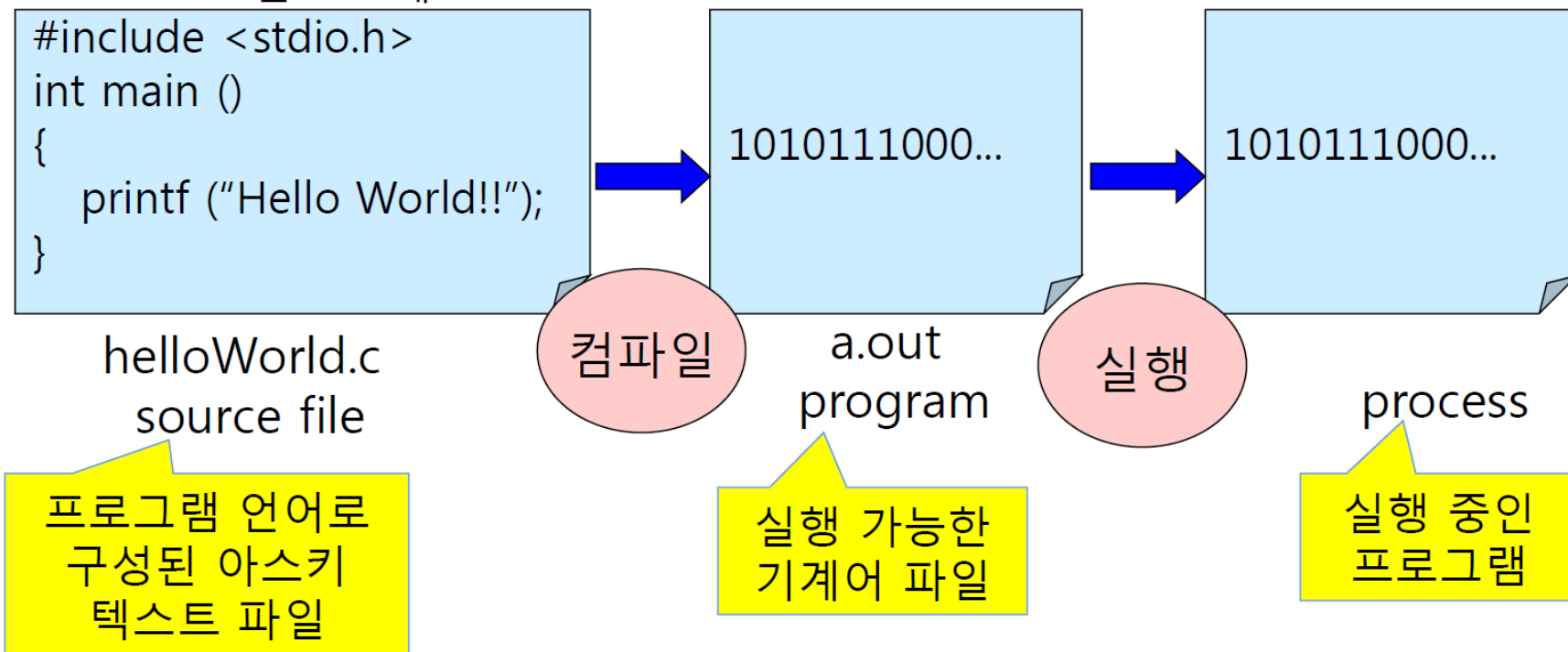
- UNIX 개발
 - 1969년 AT&T Bell Labs, Ken Thompson, Dennis Ritchie, Douglas McIlroy, Brian Kernighan(Unics)
 - Multics (Multiplexed Information and Computing Service) 프로젝트에서 파생
 - Open System : License with source
- 개발 의도
 - Portable
 - Multi-Tasking
 - Multi-User
 - Time-Sharing
 - Network 와 Security 개념은 없었다.



프로세스(process)란

▶ 프로세스(process)

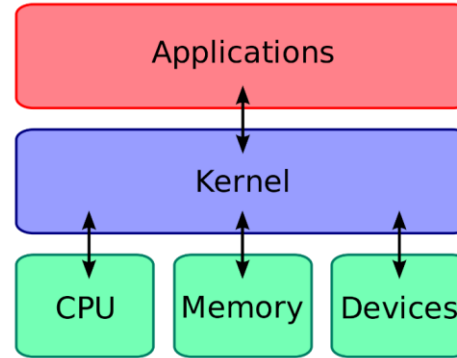
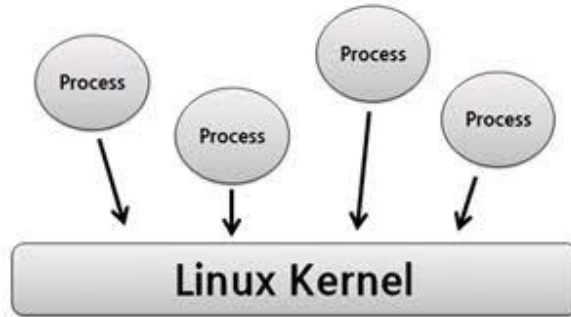
- ▶ 현재 시스템에서 실행 중인 프로그램
- ▶ 프로세스는 고유 번호를 가진다.
 - ▶ Process ID : PID
 - ▶ 1번 프로세스 : init



프로세스(process) 관리

프로세스(process)

- 실행중인 프로그램을 프로세스(process)라고 부른다.
 - 각 프로세스는 유일한 프로세스 번호 PID를 갖는다.
 - 각 프로세스는 부모 프로세스에 의해 생성된다.
- ✓ 하나의 프로세스에서 다수의 프로세스(프로그램)이 생성되는 경우도 많이 있음(네트워크 프로그램)
- ✓ Linux 데스크탑 환경에서는 100개 이상의 프로세스가 동작하고 있음.



[Memo]

- Linux 커널은 Linux 운영 체제(OS)의 주요 구성 요소이며 컴퓨터 하드웨어와 프로세스를 잇는 핵심 인터페이스임

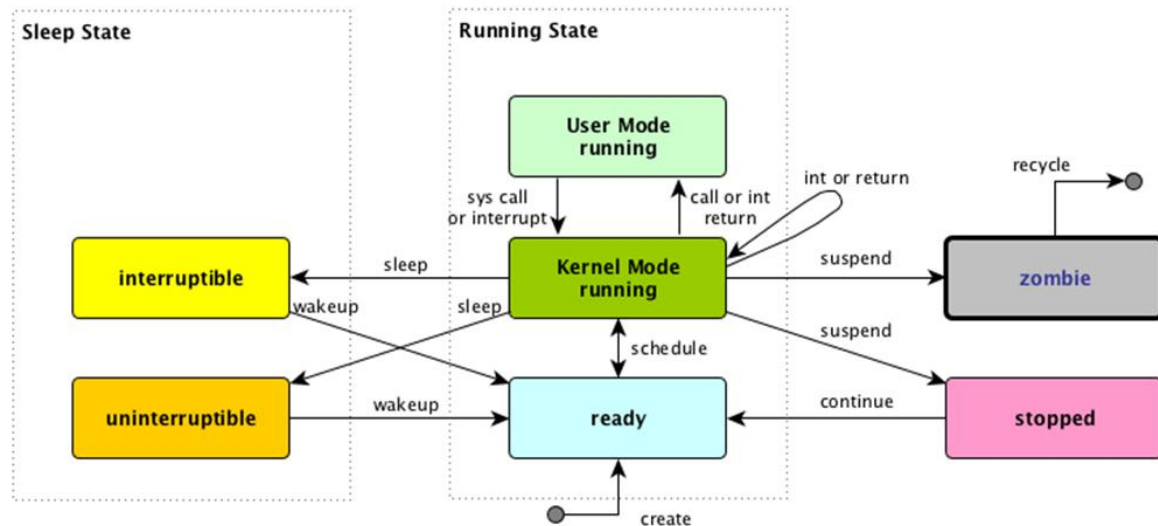
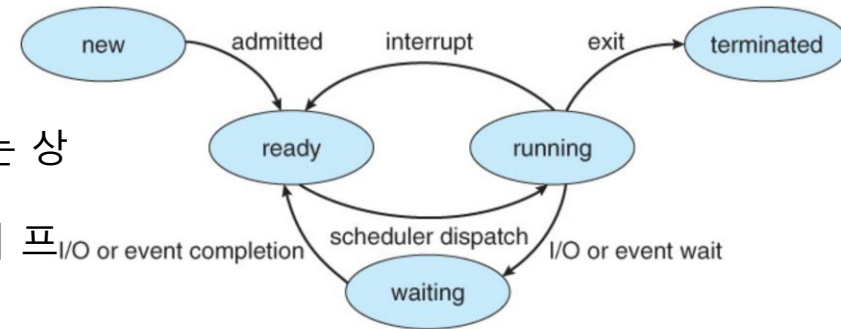
▶ 유닉스 프로세스의 종류

종류	설명
데몬 (daemon)	UNIX 커널에 의해 시작되는 프로세스로 서비스 제공을 위한 프로세스들이다.
부모 (parent)	자식 프로세스를 만드는 프로세스
자식 (child)	부모에 의해 생성된 프로세스로 실행이 끝나면 부모 프로세스로 돌아간다.
고아 (orphan)	자식프로세스가 종료하기 전에 부모가 종료된 프로세스. 고아프로세스는 1번 프로세스를 새로운 부모로 가진다.
좀비 (zombie)	부모프로세스가 종료처리를 하지 않은 프로세스 프로세스 테이블만 차지하고 있다.

프로세스(process) 관리

프로세스(process)란

new: 프로세스가 만들어지는 과정의 상태
terminated: 프로세스가 다 수행되어서 종료할 때 생기는 상태
이 외의 상태 3개(running, waiting, ready)가 돌아 가면서 프로세스가 수행
running: CPU에서 수행되고 있는 상태
Ready: CPU에서 언제든지 수행할 수 있도록 대기하고 있는 상태
waiting: I/O나 다른 이벤트가 발생하기를 기다리고 있는 상태



Linux Signals

Signal	Value	Description
1	SIGHUP	Hang up the process.
2	SIGINT	Interrupt the process.
3	SIGQUIT	Stop the process.
9	SIGKILL	Unconditionally terminate the process.
15	SIGTERM	Terminate the process if possible.
17	SIGSTOP	Unconditionally stop, but don't terminate the process.
18	SIGTSTP	Stop or pause the process, but don't terminate.
19	SIGCONT	Continue a stopped process.

프로세스(process) 관리

프로세스(process)란

사전적 의미

- “컴퓨터에서 연속적으로 실행되고 있는 컴퓨터 프로그램”메모리에 올라와 실행되고 있는 프로그램의 인스턴스(독립적인 개체)
- 운영체제로부터 시스템 자원을 할당받는 작업의 단위
- 즉, 동적인 개념으로는 실행된 프로그램을 의미한다.

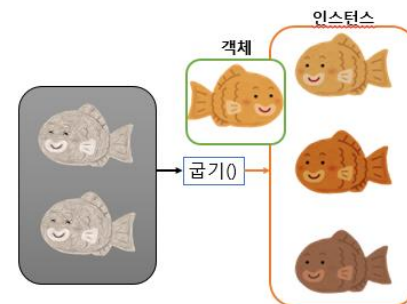
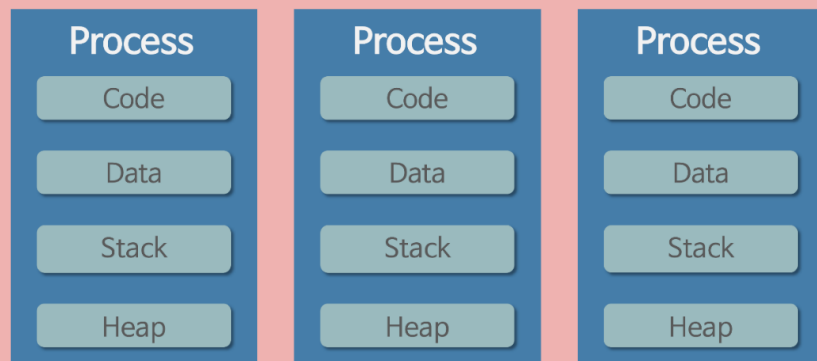
참고 할당받는 시스템 자원의 예

- CPU 시간
- 운영되기 위해 필요한 주소 공간
- Code, Data, Stack, Heap의 구조로 되어 있는 독립된 메모리 영역

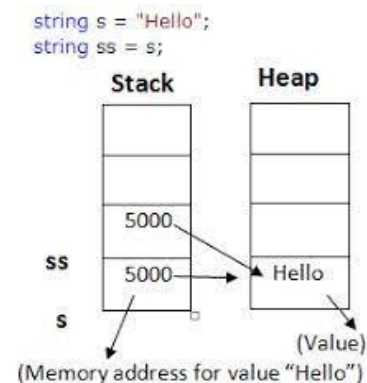
특징

- 프로세스는 각각 독립된 메모리 영역(Code, Data, Stack, Heap의 구조)을 할당받는다.
- 기본적으로 프로세스당 최소 1개의 스레드(메인 스레드)를 가지고 있다.
- 각 프로세스는 별도의 주소 공간에서 실행되며, 한 프로세스는 다른 프로세스의 변수나 자료구조에 접근할 수 없다.
- 한 프로세스가 다른 프로세스의 자원에 접근하려면 프로세스 간의 통신(IPC, inter-process communication)을 사용해야 한다.Ex. 파이프, 파일, 소켓 등을 이용한 통신 방법 이용

Operation System



인스턴스는 일반적으로 실행 중인 임의의 프로세스



리눅스의 프로세스(process) 관리

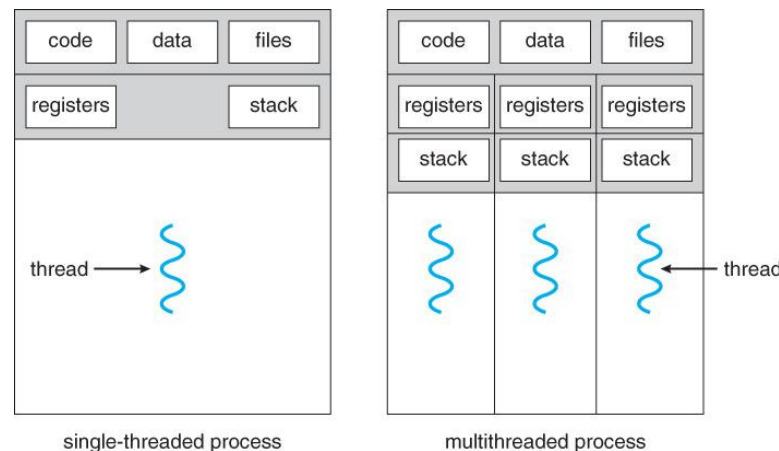
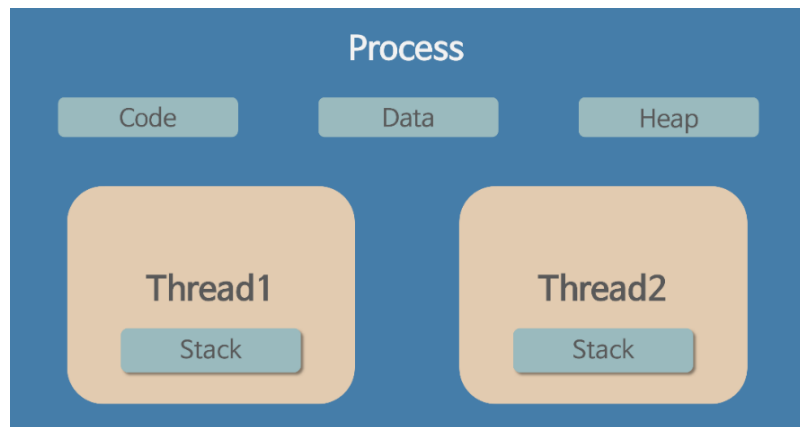
스레드(Thread) 란

▪ 사전적 의미

- “프로세스 내에서 실행되는 여러 흐름의 단위”
- 프로세스의 특정한 수행 경로
- 프로세스가 할당받은 자원을 이용하는 실행의 단위

▪ 특징

- 스레드는 프로세스 내에서 각각 Stack만 따로 할당받고 Code, Data, Heap 영역은 공유한다.
- 스레드는 한 프로세스 내에서 동작되는 여러 실행의 흐름으로, 프로세스 내의 주소 공간이나 자원들(힙 공간 등)을 같은 프로세스 내에 스레드끼리 공유하면서 실행된다.
- 같은 프로세스 안에 있는 여러 스레드들은 같은 힙 공간을 공유한다. 반면에 프로세스는 다른 프로세스의 메모리에 직접 접근할 수 없다.
- 각각의 스레드는 별도의 레지스터와 스택을 갖고 있지만, 힙 메모리는 서로 읽고 쓸 수 있다.
- 한 스레드가 프로세스 자원을 변경하면, 다른 이웃 스레드(sibling thread)도 그 변경 결과를 즉시 볼 수 있다.



리눅스의 프로세스(process) 관리

프로세스(process)

➤ Process

- 프로그램이나 명령어를 실행하면 메모리에 적재되어 실제로 실행되고 있는 상태를 의미
- 이러한 프로세스들은 프로세스가 시작하면서 할당받는 프로세스 식별번호인 PID(Process ID), 해당 프로세스를 실행한 부모 프로세스를 나타내는 PPID(Parent Process ID), UID와 GID 정보를 통해 해당 프로세스가 어느 사용자에게 속해 있는지, 프로세스가 파일에 대해 갖는 권한 및 프로세스가 실행된 터미널, 입력된 명령어, 시작 시간 등 많은 정보를 가지고 있음

➤ ps 명령어

- 현재 실행 중인 프로세스를 확인하는 명령어
- 프로세스의 상태를 확인할 수 있음

프로세스(process) 관리

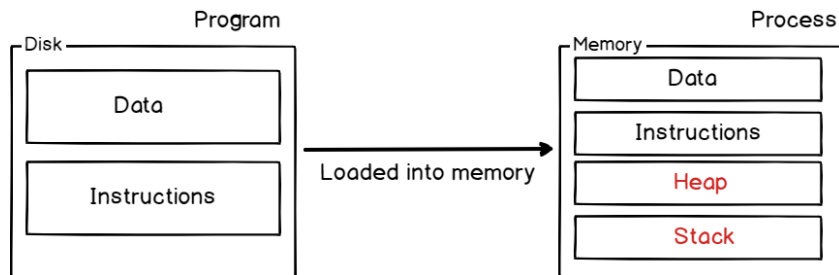
1) 프로세스 관리 명령어 - ps

➤ 예

```
$ ps
PID TTY TIME CMD
8695 pts/3 00:00:00 bash
8720 pts/3 00:00:00 ps
```

```
$ ps u
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
chang 8695 0.0 0.0 5252 1728 pts/3 Ss 11:12 0:00 bash
chang 8793 0.0 0.0 4252 940 pts/3 R+ 11:15 0:00 ps u
```

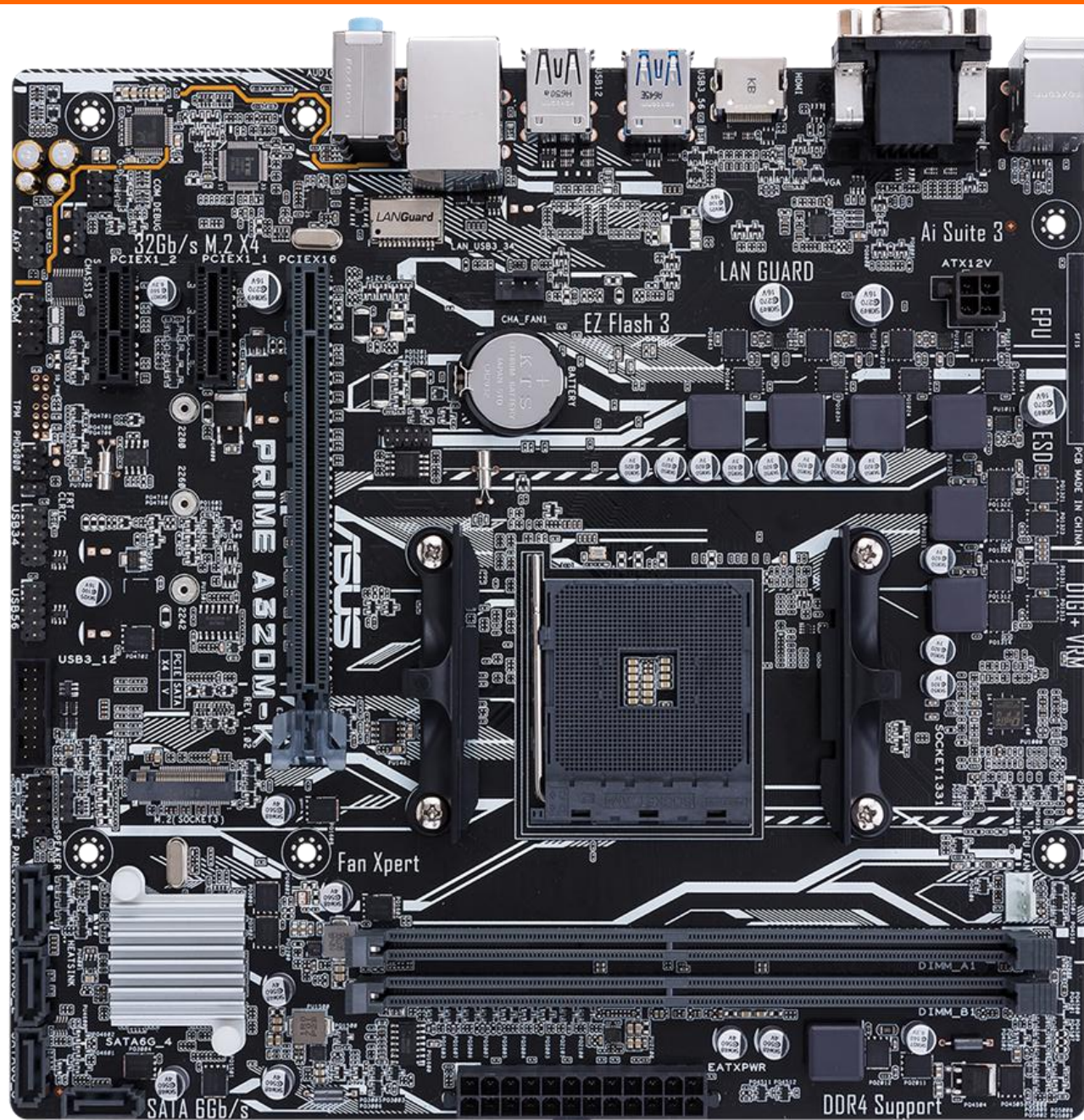
Difference Program & Process



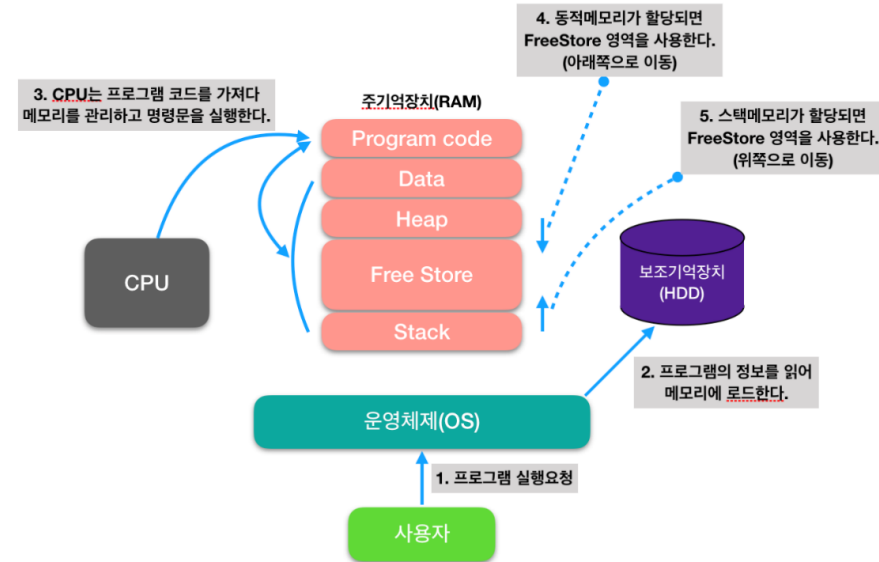
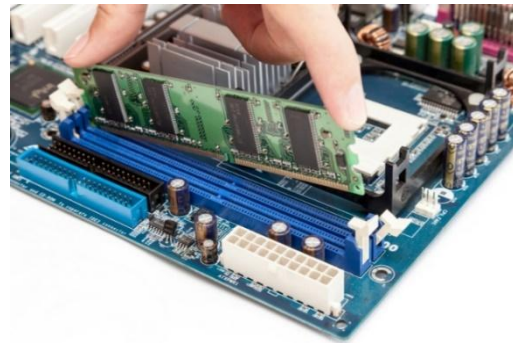
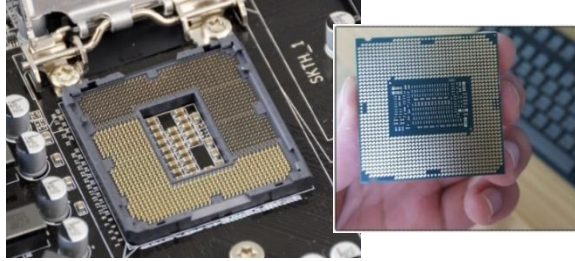
ps 출력 정보

항목	설명
PID	프로세스의 아이디, 식별번호
PPID	부모 프로세스 ID
UID	SYSTEM V계열에서 나타나는 항목으로 프로세스 소유자의 이름
TTY	프로세스를 제어하는 수단, 프로세스와 연결된 터미널로 콘솔접속시 "tty숫자" 형태로 표시되며, 원격이나 에뮬레이터 접속시 "pts/숫자" 형태로 표시
TIME	프로세스에 사용된 CPU 시간
CMD	프로세스 실행 명령어
COMMAND	프로세스의 실행 명령행
USER	BSD계열에서 나타나는 항목으로 프로세스 소유자의 이름
%CPU	CPU 사용 비율의 추정치(BSD)
%MEM	메모리의 사용 비율의 추정치 (BSD)
VSZ	K단위 또는 페이지 단위의 가상메모리 사용량
RSS	실제 메모리 사용량 (Resident Set Size)
S, STAT	현재 프로세스의 상태 코드 (S: Sys V, STAT: BSD)
STIME	프로세스가 시작된 시간 혹은 날짜
C, CP	짧은 기간 동안의 CPU 사용률 (C: Sys V, CP: BSD)
F	프로세스의 플래그
PRI	실제 실행 우선순위
NI	nice 우선순위 번호

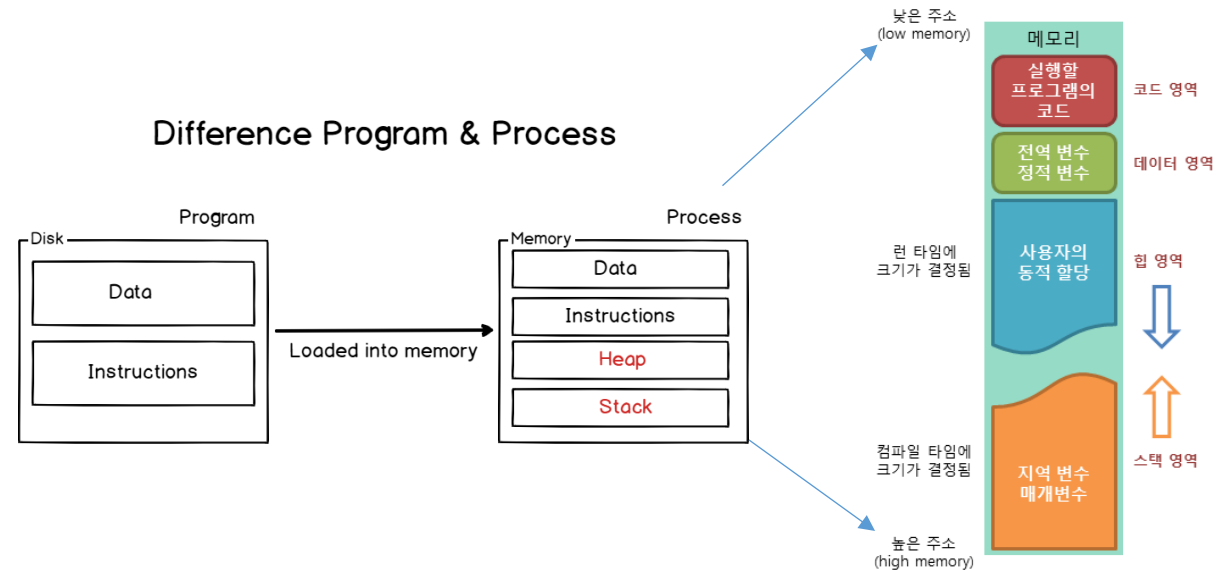
프로세스(process) 관리



프로세스(process) 관리



Difference Program & Process



프로세스(process) 관리

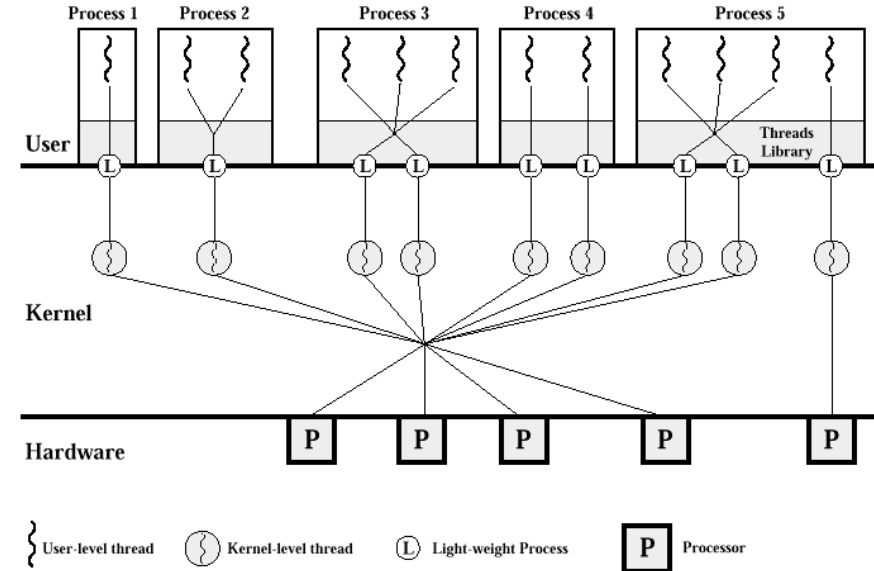
1) 프로세스 관리 명령어 - ps

ps - ② 프로세스를 확인할 때 가장 많이 사용하는 옵션

```

root@localhost:~# ps -ef
UID          PID    PPID  C   STIME TTY          TIME CMD
root         1        0   0   12:25 ?        00:00:01 init [5]
root         2        1   0   12:25 ?        00:00:00 [migration/0]
root         3        1   0   12:25 ?        00:00:00 [ksoftirqd/0]
root         4        1   0   12:25 ?        00:00:00 [events/0]
root         5        1   0   12:25 ?        00:00:00 [khelper]
root         6        1   0   12:25 ?        00:00:00 [kthreadd]
root         9        6   0   12:25 ?        00:00:00 [kblockd/0]
root        10        6   0   12:25 ?        00:00:00 [kacpid]
root       174        6   0   12:25 ?        00:00:00 [cqueue/0]
root       177        6   0   12:25 ?        00:00:00 [khubd]
root       179        6   0   12:25 ?        00:00:00 [kserviod]
root       244        6   0   12:25 ?        00:00:00 [khungtaskd]
root       245        6   0   12:25 ?        00:00:00 [pdflush]
root       246        6   0   12:25 ?        00:00:01 [pdflush]
root       247        6   0   12:25 ?        00:00:00 [kswapd0]
root       248        6   0   12:25 ?        00:00:00 [aio/0]
root       376        1   0   15:39 ?        00:00:00 crond
root       466        6   0   12:26 ?        00:00:00 [kpsmouse]
root       493        6   0   12:26 ?        00:00:00 [mpt_poll_0]
root       494        6   0   12:26 ?        00:00:00 [mpt/0]
root       495        6   0   12:26 ?        00:00:00 [scsi_eh_0]
root       498        6   0   12:26 ?        00:00:00 [ata/0]
    
```

UID	사용자의 UID	STIME	프로세스가 시작된 시간
PID	프로세스 식별 번호	TTY	프로세스가 실행된 터미널 포트
PPID	프로세스의 부모 프로세스 식별 번호	TIME	총 CPU 사용 시간
PIDC	현재는 사용되지 않음	CMD	실행한 명령어 라인



Computation Server

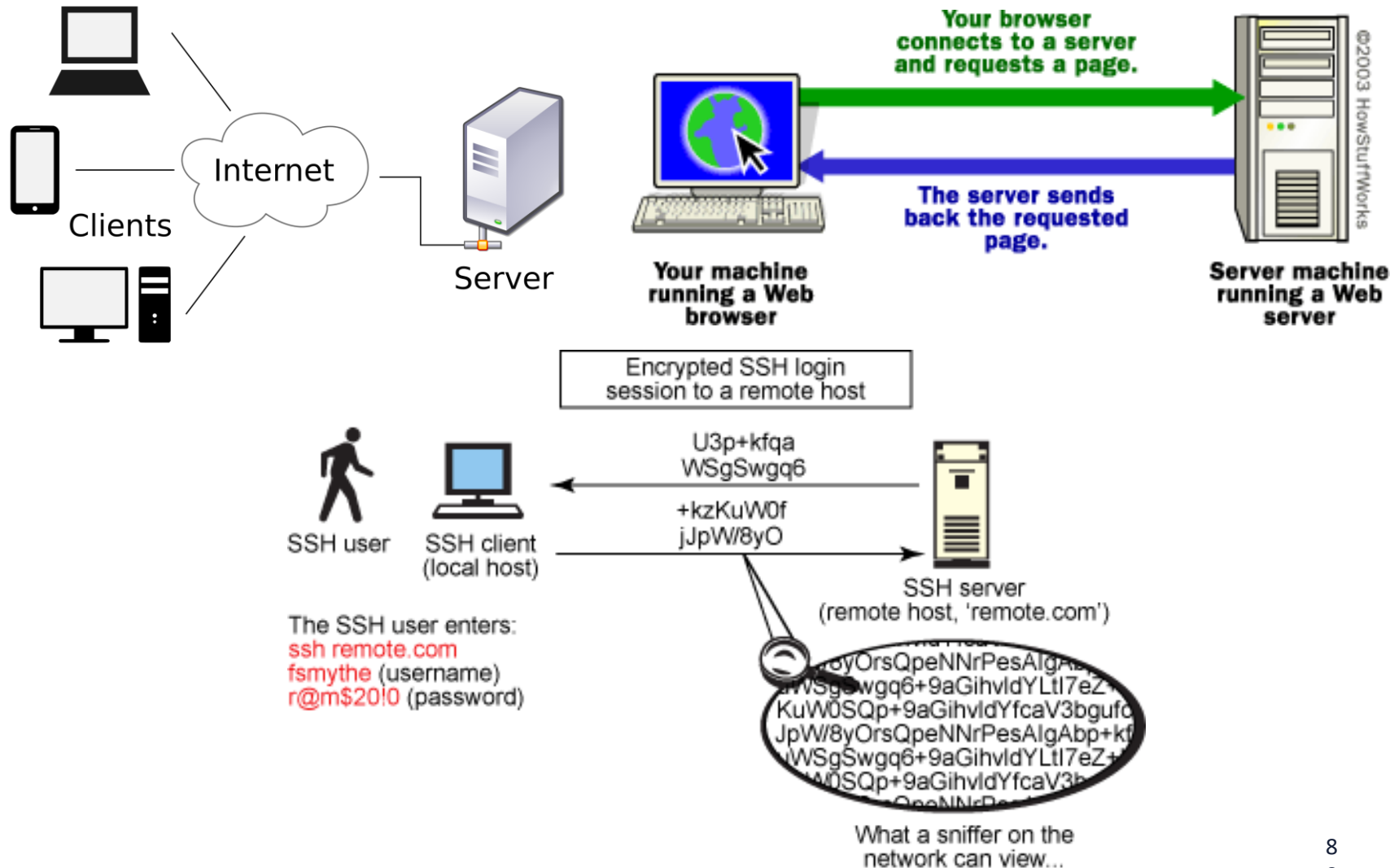


In the cold and dark server room!

**Run Linux/Unix
Operating System**



Client/Server and SSH (Secure Shell)



Linux & C언어

Machine for Development for OpenMP and MPI

- **Linux machines in Swearingen 1D39 and 3D22**
 - **All CSCE students by default have access to these machine using their standard login credentials**
 - **Let me know if you, CSCE or not, cannot access**
 - **Remote access is also available via SSH over port 222. Naming schema is as follows:**
 - **l-1d39-01.cse.sc.edu through l-1d39-26.cse.sc.edu**
 - **l-3d22-01.cse.sc.edu through l-3d22-20.cse.sc.edu**
- **Restricted to 2GB of data in their home folder (~/).**
 - **For more space, create a directory in /scratch on the login machine, however that data is not shared and it will only be available on that specific machine.**

Linux Basic Commands

It is all about dealing with files and folders

Linux folder: /acct/yanyh/...

- ls (list files in the current folder)
 - `$ ls -l`
 - `$ ls -a`
 - `$ ls -la`
 - `$ ls -l --sort=time`
 - `$ ls -l --sort=size -r`
- cd (change directory to)
 - `$ cd /usr/bin`
- pwd (show current folder name)
 - `$ pwd`
- ~ (home folder)
 - `$ cd ~`
- ~user (home folder of a user)
 - `$ cd ~weesan`
- What will “`cd ~/weesan`” do?
- rm (remove a file/folder)
 - `$ rm foo`
 - `$ rm -rf foo`
 - `$ rm -i foo`
 - `$ rm -- -foo`
- cat (print the file contents to terminal)
 - `$ cat /etc/motd`
 - `$ cat /proc/cpuinfo`
- cp (create a copy of a file/folder)
 - `$ cp foo bar`
 - `$ cp -a foo bar`
- mv (move a file/folder to another location. Used also for renaming)
 - `$ mv foo bar`
- mkdir (create a folder)
 - `$ mkdir foo`

Basic Commands (cont)

- df (Disk usage)
 - `$ df -h /`
 - `$ du -sxh ~/`
- man (manual)
 - `$ man ls`
 - `$ man 2 mkdir`
 - `$ man man`
 - `$ man -k mkdir`
- Manpage sections
 - 1 User-level cmds and apps
 - `/bin/mkdir`
 - 2 System calls
 - `int mkdir(const char *, ...);`
 - 3 Library calls
 - `int printf(const char *, ...);`

Search a command or a file

- which
 - `$ which ls`
- whereis
 - `$ whereis ls`
- locate
 - `$ locate stdio.h`
 - `$ locate iostream`
- find
 - `$ find / | grep stdio.h`
 - `$ find /usr/include | grep stdio.h`

Smarty:

1. **[Tab] key:** auto-complete the command sequence



↑ key: to find previous command

3. **[Ctl]+r key:** to search previous command

Editing a File: Vi/Vim

- 2 modes
 - Input mode
 - **ESC to back to cmd mode**
 - Command mode
 - **Cursor movement**
 - h (left), j (down), k (up), l (right)
 - ^f (page down)
 - ^b (page up)
 - ^ (first char.)
 - \$ (last char.)
 - G (bottom page)
 - :1 (goto first line)
 - **Switch to input mode**
 - a (append)
 - i (insert)
 - o (insert line after)
 - O (insert line before)
 - **Delete**
 - dd (delete a line)
 - d10d (delete 10 lines)
 - d\$ (delete till end of line)
 - dG (delete till end of file)
 - x (current char.)
 - **Paste**
 - p (paste after)
 - P (paste before)
 - **Undo**
 - u
 - **Search**
 - /
 - **Save/Quit**
 - :w (write)
 - :q (quit)
 - :wq (write and quit)
 - :q! (give up changes)

C Hello World

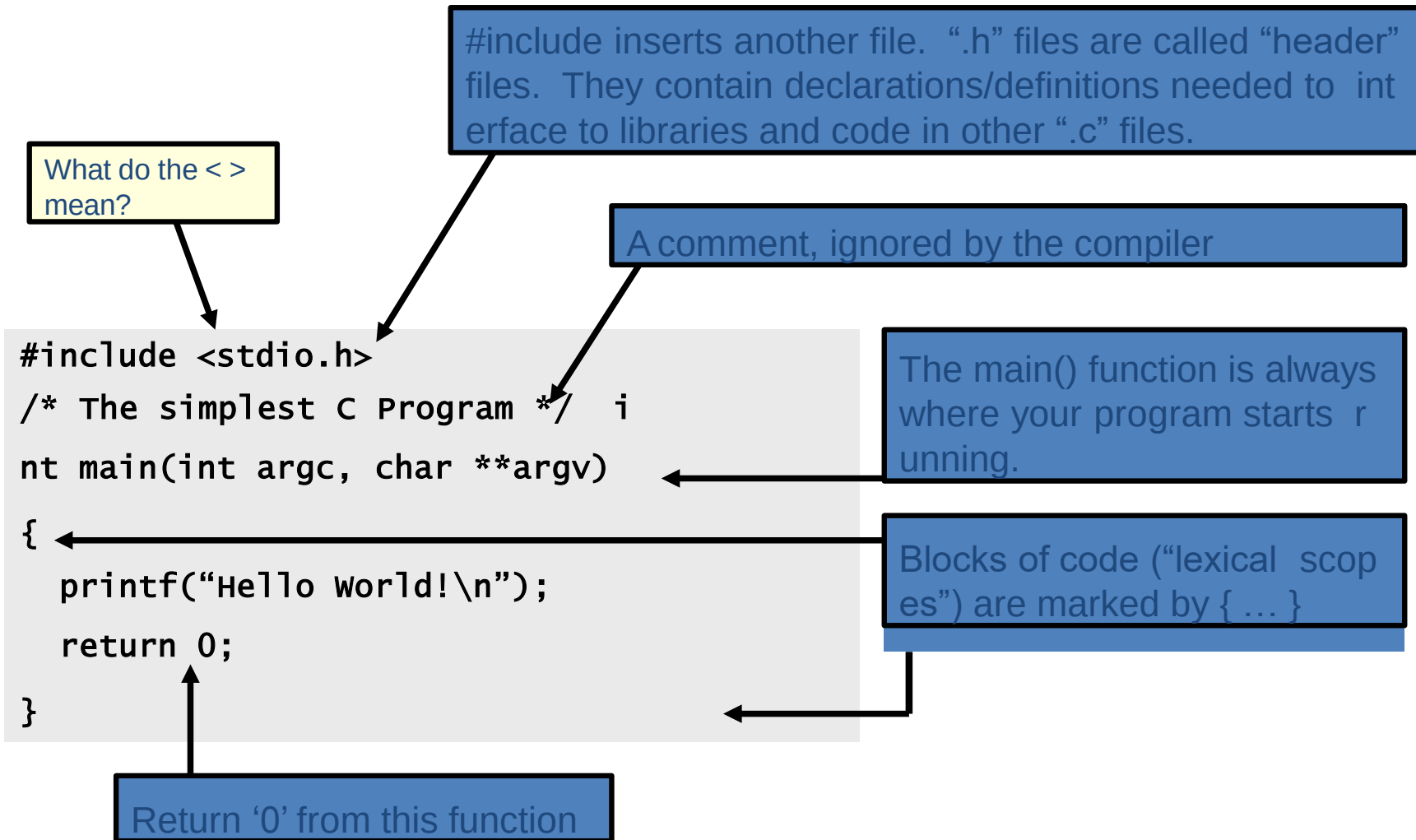
- vi hello.c
- Switch to editing mode: i or a
- Switching to control mode: ESC
- **Save a file: in control mode, :w**
- **To quit, in control mode, :q**
- To quit without saving, :q!
- Copy/paste a line: in control mode, “yy” and then “p”, both from the current cursor
 - 5 line: 5yy and then p
- To delete a whole line, in control mode, : dd

- vi hello.c
- ls hello.c
- **gcc hello.c -o hello**
- ls
- ./hello

```
#include <stdio.h>

/* The simplest C Program */
int main(int argc, char **argv) {
    printf("Hello world\n");
    return 0;
}
```

C Syntax and Hello World



Linux & C언어

Compiling/Building Process in C to Generate Executables

- Compiling/Building process: gcc hello.c -o hello
 - Constructing an executable image for an application
 - FOUR stages
 - Command:
gcc <options> <source_file.c>
- Compiler Tool
 - gcc (GNU Compiler)
 - man gcc (on Linux m/c)
 - icc (Intel C compiler)

4 Stages of Compiling Process

1. Preprocessing (Those with # ...)

- Expansion of Header files (#include ...)
- Substitute macros and inline functions (#define ...)

2. Compilation (the most important one)

- Generates assembly language
- Verification of functions usage using prototypes
- Header files: Prototypes declaration (-I option to provide header folder)

3. Assembling

- Generates re-locatable object file (contains m/c instructions)
- nm app.o: To list functions/symbols provided by a an object file
0000000000000000 T main
 U puts
- objdump to view object files and disassembly

4 Stages of Compiling Process (contd..)

4. Linking

- Generates executable file (nm tool used to view exe file)
- Binds appropriate libraries
 - Static Linking
 - Dynamic Linking (default)
- Loading and Execution (of an executable file)
 - Evaluate size of code and data segment
 - Allocates address space in the user mode and transfers them into memory
 - Load dependent libraries needed by program and links them
 - Invokes Process Manager → Program registration

4 Stages of Compiling Process

View the output of each stage using vi editor: e.g. vim hello.i

Preprocessing

```
gcc -E hello.c -o hello.i  
hello.c → hello.i
```



Compilation (after preprocessing)

```
gcc -S hello.i -o hello.s
```



Assembling (after compilation)

```
gcc -c hello.s -o hello.o
```



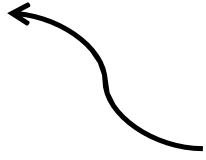
Linking object files

```
gcc hello.o -o hello
```



Output → Executable (a.out)
Run → ./hello (Loader)

Compiling a C Program

- `gcc <options> program_name.c`
- Options:

 - Wall**: Shows all warnings
 - o output_file_name**: By default a.out executable file is created when we compile our program with gcc. Instead, we can specify the output file name using "-o" option.
 - g**: Include debugging information in the binary.
- `man gcc`

Four stages into one

Linking multiple files to make executable file

- Two programs, prog1.c and prog2.c for one single task
 - To make single executable file using following instructions

First, compile these two files with option "-c" `gcc -c prog1.c`
`gcc -c prog2.c`

-c: Tells gcc to compile and assemble the code, but not link. We get two files as output, prog1.o and prog2.o

Then, we can link these object files into single executable file using below instruction.

```
gcc -o prog prog1.o prog2.o
```

Now, the output is prog executable file. We can run our program using
./prog

Linking with other libraries

- Normally, compiler will read/link libraries from /usr/lib directory to our program during compilation process.
 - Library are precompiled object files
- To link our programs with libraries like pthreads and realtime libraries (rt library).
 - gcc <options> program_name.c **-lpthread -lrt**

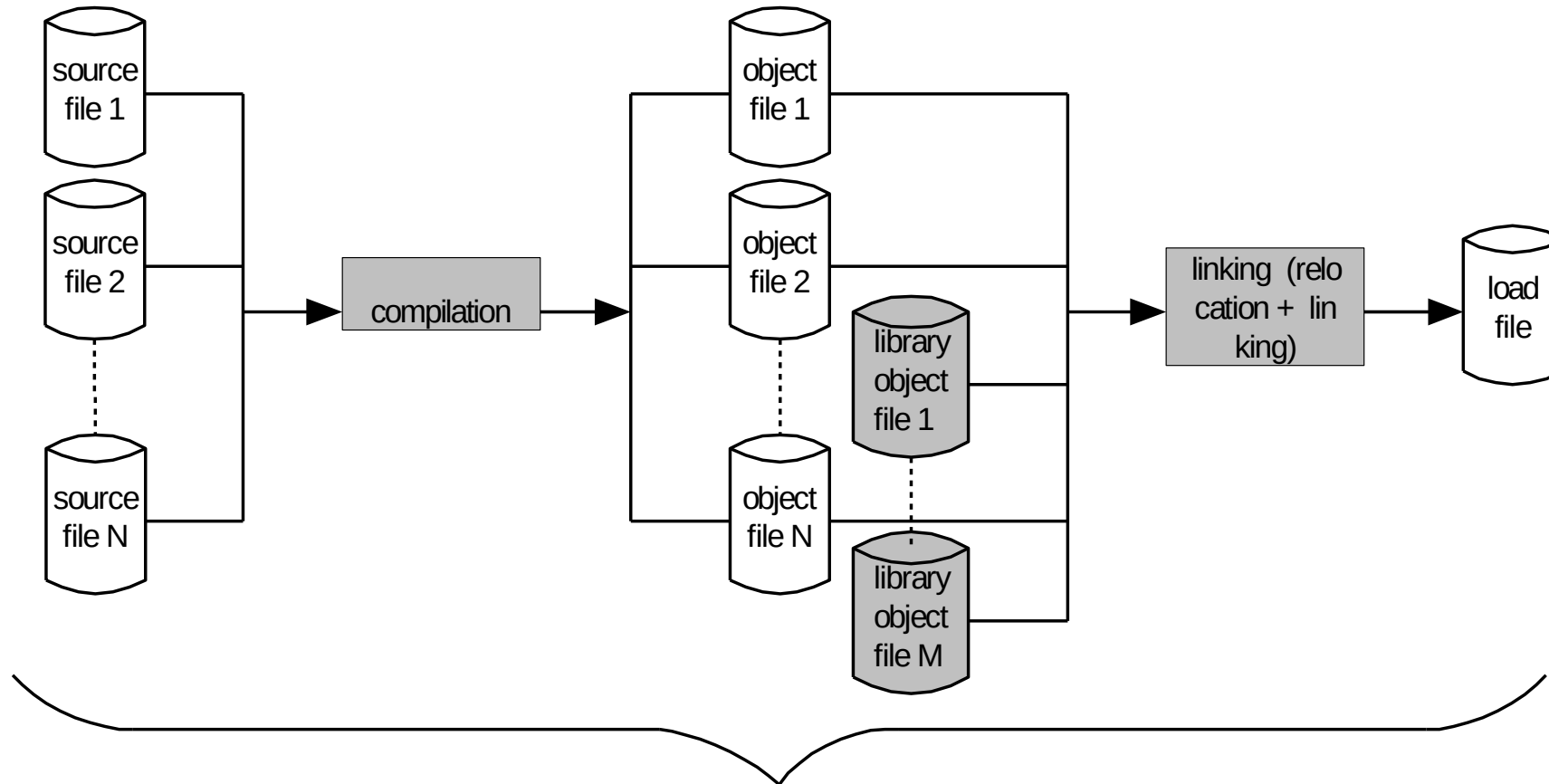
-lpthread: Link with pthread library → **libpthread.so** file

-lrt: Link with rt library → **librt.so** file

Option here is "**-l<library>**"

Another option "**-L<dir>**" used to tell gcc compiler search for library file in given <dir> directory.

Compilation, Linking, Execution of C/C++ Programs



usually performed by a compiler, usually in one uninterrupted sequence

Three Useful Commands

- nm: e.g. “nm a.out”, “nm libc.so”
 - list **symbols** from object files
 - **Symbols: function name, global variables that are exposed or reference by an object file.**
- ldd: e.g. “ldd a.out”, or “ldd hello”
 - List the name and the path of the dynamic library needed by a program
 - LD_LIBRARY_PATH: env for setting runtime lib path
 - **export LD_LIBRARY_PATH=/acct/yanyh/usr/lib:\$LD_LIBRARY_PATH**
- objdump: objdump -d a.out
 - dump information about object files, including disassembly

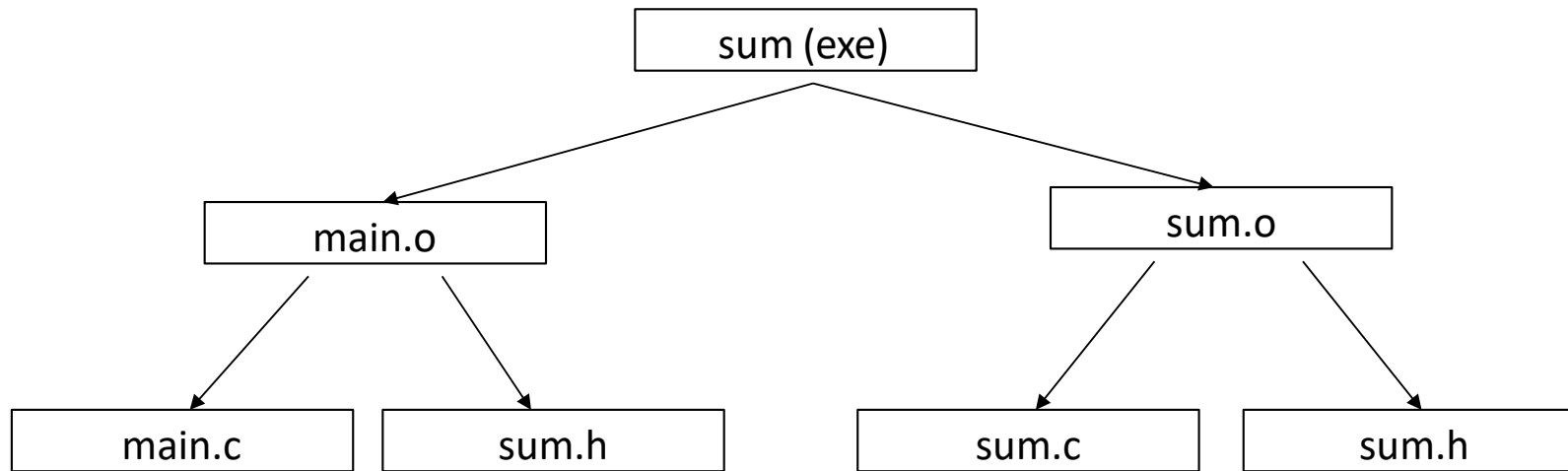
sum.c

- Download the file:
 - `wget https://passlab.github.io/CSCE569/resources/sum.c`
- `gcc sum.c -o sum`
- `./sum 102400`
- `vi sum.c`
- `ldd sum`
- `nm sum`
- Other system commands:
 - `cat /proc/cpuinfo` to show the CPU and #cores
 - `top` command to show system usage and memory

Or step by step

```
gcc -E sum.c -o sum.i g
cc -S sum.i -o sum.s gcc
-c sum.c -o sum.o gcc s
um.o -o sum
```

Makefile



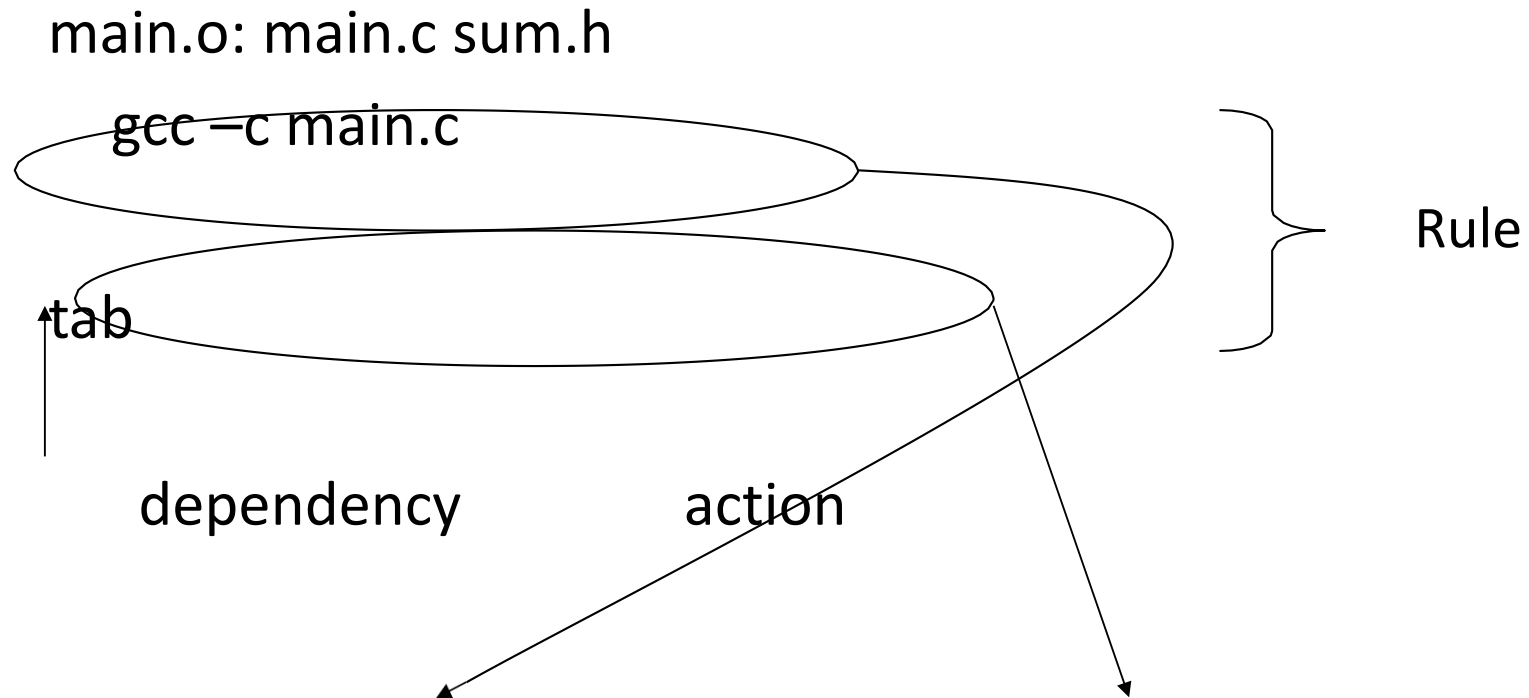
Makefile

```
sum: main.o sum.o  
    gcc -o sum main.o sum.o
```

```
main.o: main.c sum.h  
    gcc -c main.c
```

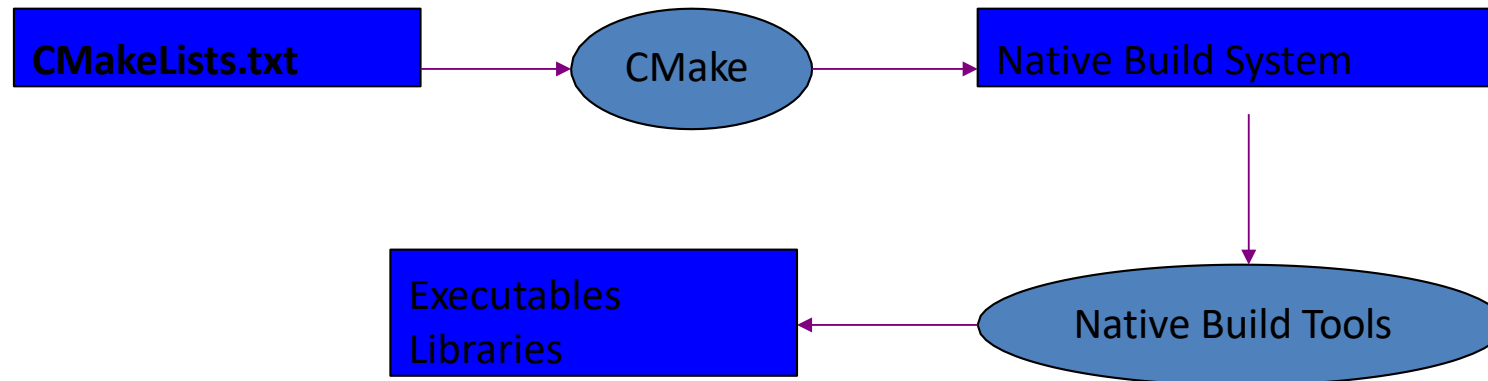
```
sum.o: sum.c sum.h  
    gcc -c sum.c
```


Rule syntax



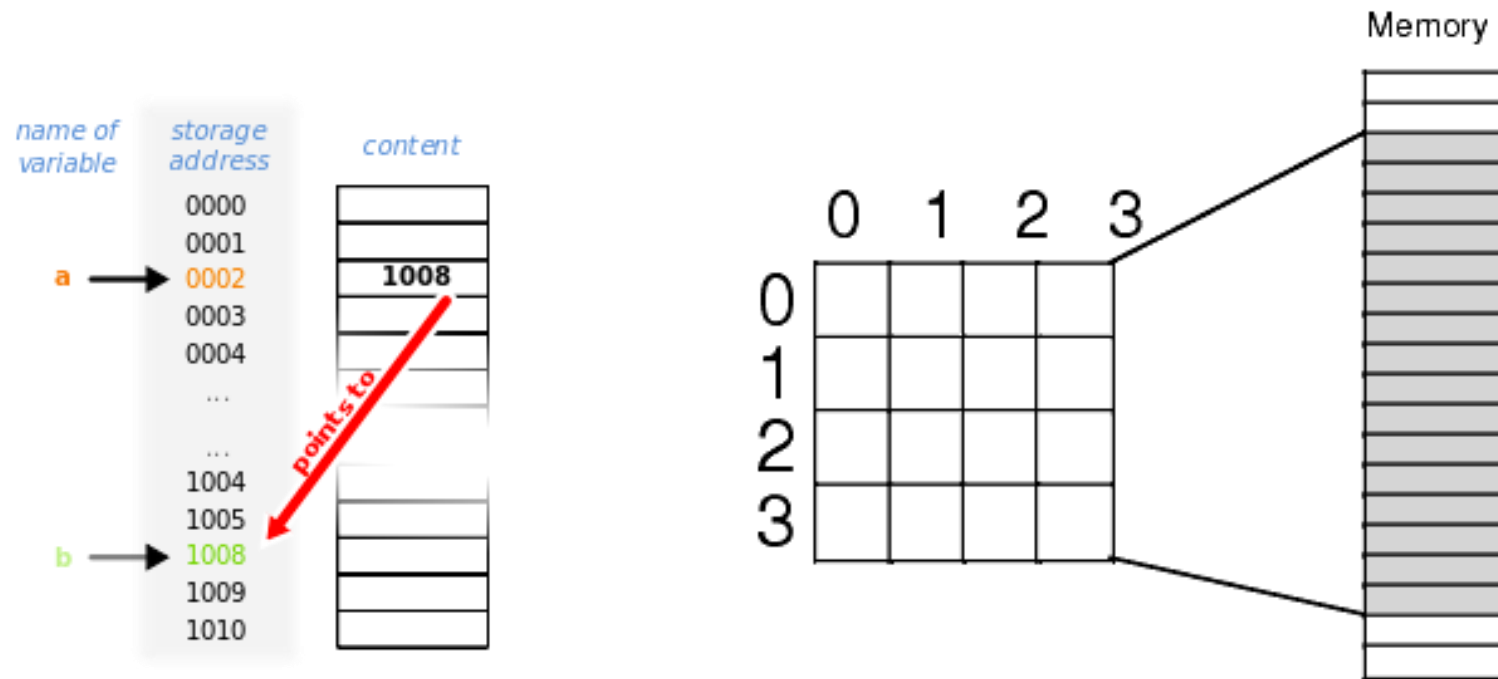
cmake: Makefile/Build-System Generator

- Provides single-sourcing for build systems
- Knowledge of many platforms and tools
- Users configure builds through a GUI



Sequential Memory Regions vs Multi- dimensional Array

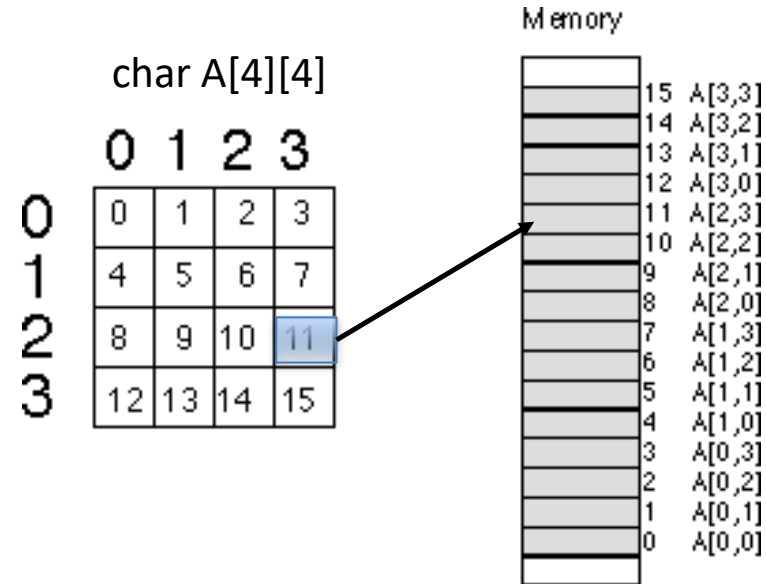
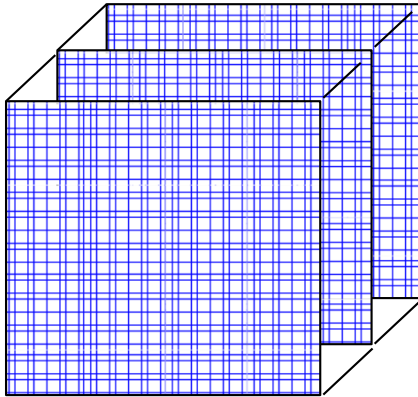
- Memory is sequentially accessed using the address of each byte/word



Vector/Matrix and Array in C

- C has row-major storage for multiple dimensional array
 - $A[2,2]$ is followed by $A[2,3]$

- 3-dimensional array
 - $B[3][100][100]$



Memory address of $A[2][3]$
 $= A[0][0] + \text{offset}$
 $= A + \text{sizeof}(\text{char}) * (2 * \# \text{ columns} + 3)$
 $= 0 + 1 * (2 * 4 + 3) = 11$

Linux & C언어

Store Array in Memory in Row Major

3x4 matrix

8	6	5	4
2	1	9	7
3	6	4	2

C array for matrix:
`int A[3][4]`

Row-Major (Row Wise Arrangement)

C storage type

Address of element `A[1][2]`?



Row 0

Row 1

Row 2

Address of element `A[1][2]`?

$= A + \text{sizeof}(\text{int}) * (1 * 4 + 2)$

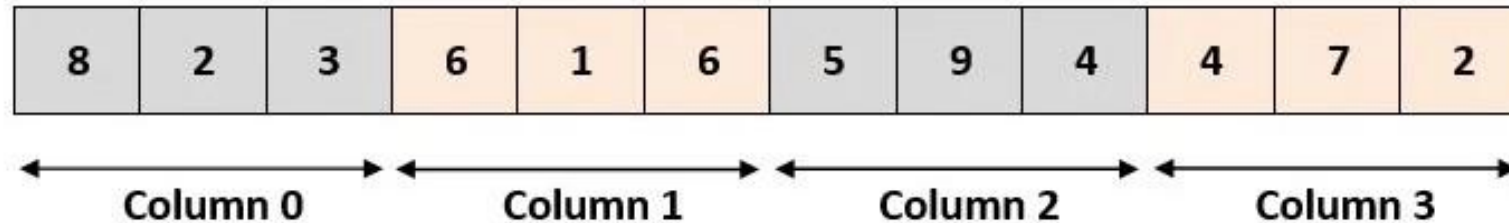
$= A + 4 * 6 = A + 24$

Store Array in Memory in Column Major

3x4 matrix

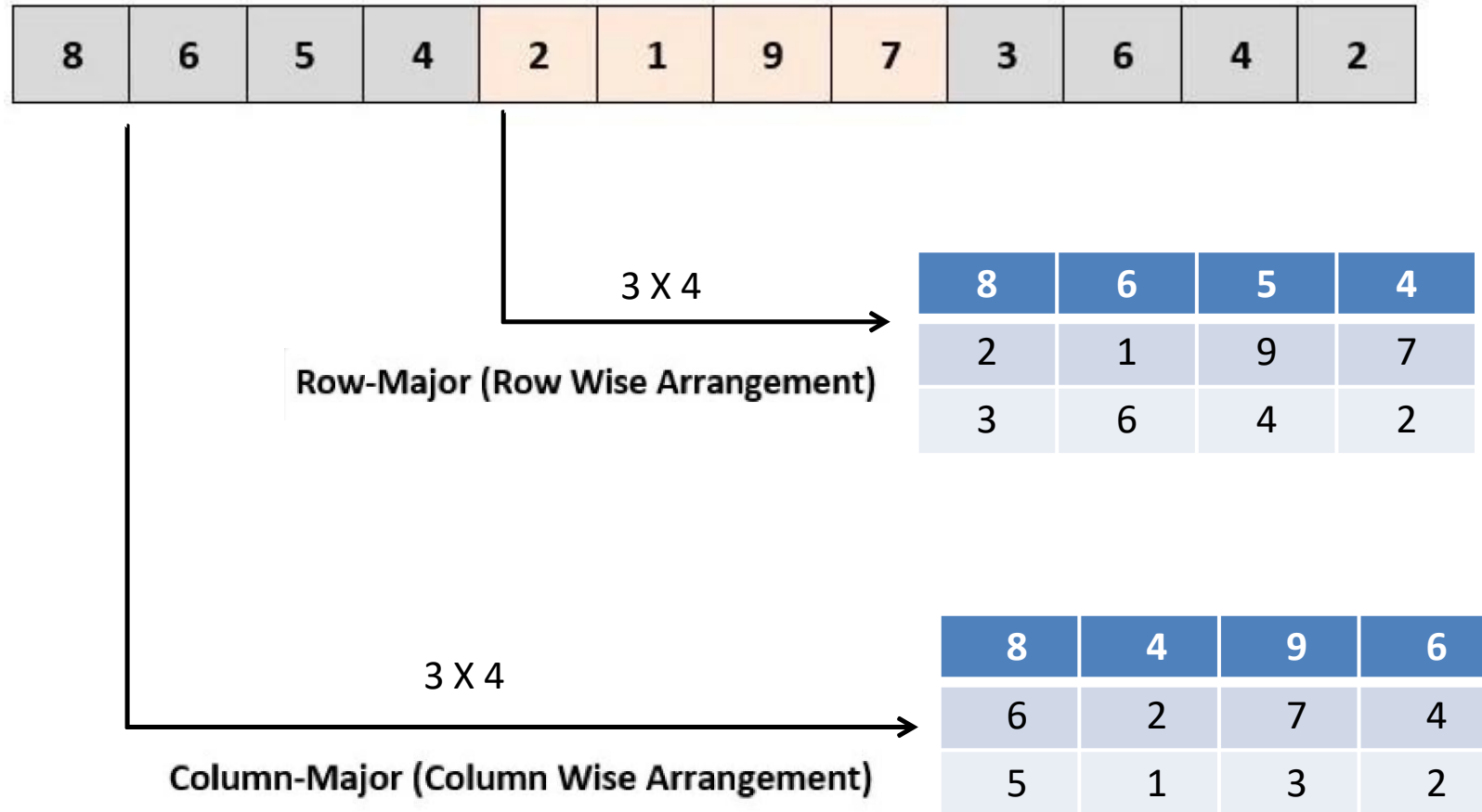
8	6	5	4
2	1	9	7
3	6	4	2

Column-Major (Column Wise Arrangement)



Linux & C언어

For a Memory Region to Store Data for an Array in Either Row or Col Major

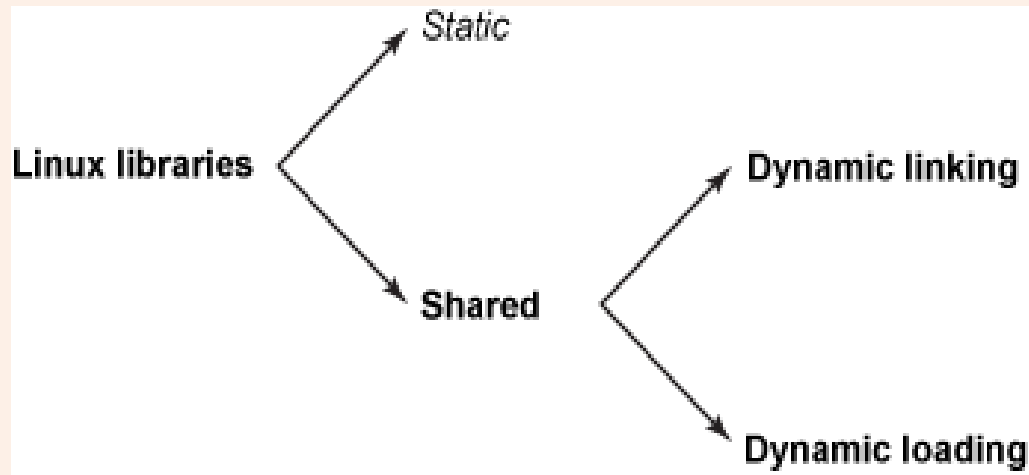


C Programming

- Linux/Unix Introduction
 - <http://www.ee.surrey.ac.uk/Teaching/Unix/>
- VI Editor
 - <https://www.cs.colostate.edu/helpdocs/vi.html>
- C Programming Tutorial
 - <http://www.cprogramming.com/tutorial/c-tutorial.html>
- Compiler, Assembler, Linker and Loader: A Brief Story
 - <http://www.tenouk.com/ModuleW.html>

**linux library 만들기
(static, shared, dynamic)**

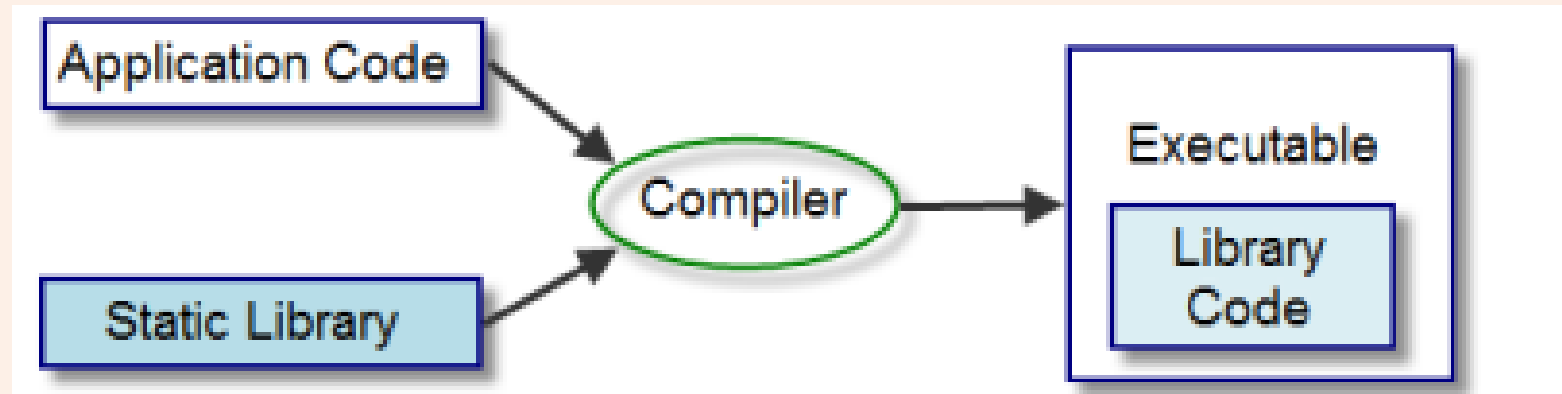
Library와 Link



- 정적 라이브러리와 동적 라이브러리(Static Library, Dynamic Library)
 - 라이브러리에 의존하는 프로그램은 시스템에 필요한 라이브러리가 설치되지 않으면 동작하지 않는다.
 - 실제로 실행될 때는 라이브러리가 제공하는 코드와 링크되어야 프로그램 코드가 동작할 수 있다.
 - 라이브러리는 오브젝트 코드와 결합하는 방법에 따라 정적(Static) 라이브러리와 공유(Shared) 라이브러리로 나뉜다.
 - 공유(shared) 라이브러리는 다시 일반적인 동적 링크(Dynamic link) 라이브러리와 동적 로드(Dynamic load) 라이브러리로 나뉜다.

정적 라이브러리(Static Library)란?

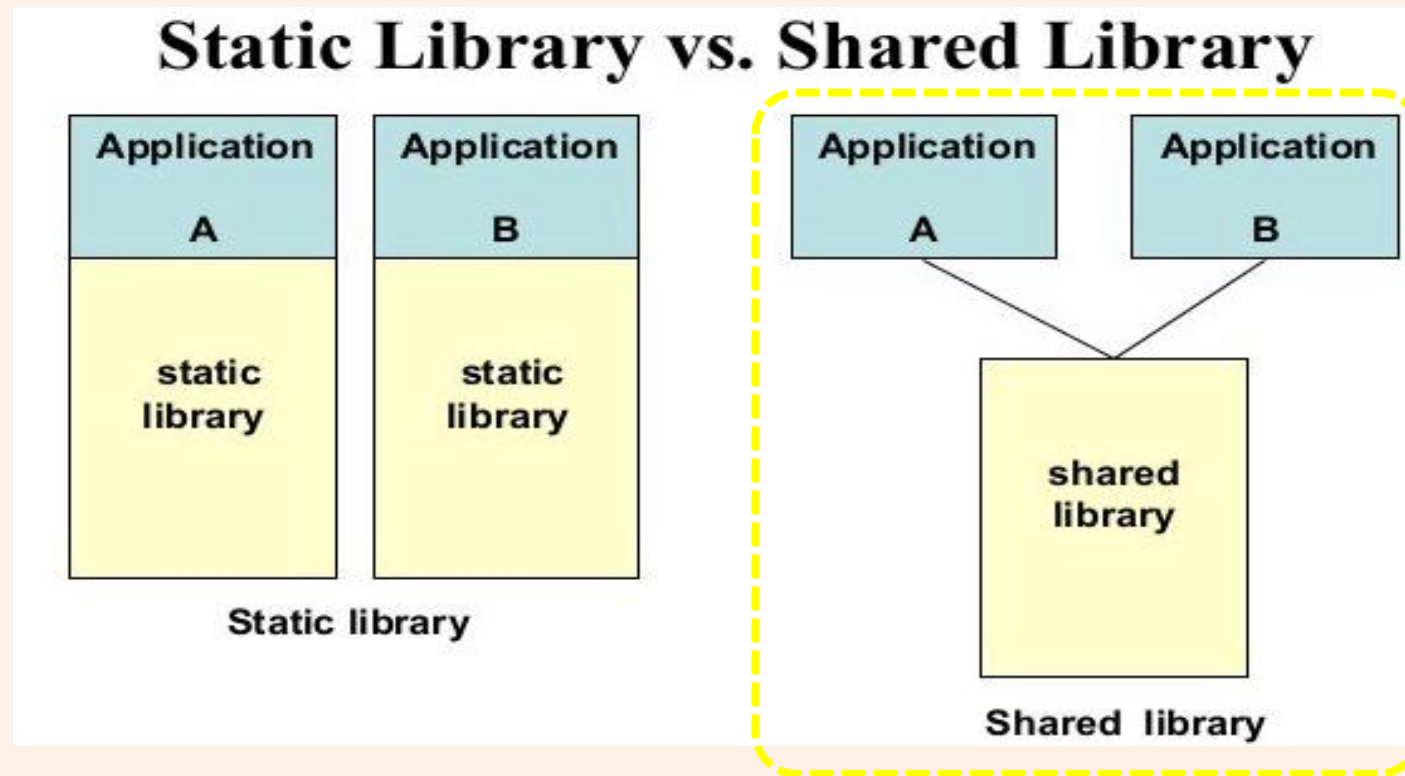
- 프로그램 빌드 시에 라이브러리가 제공하는 코드를 실행 파일에 넣는 방식의 라이브러리를 의미한다.



- 이 방식의 장점은 시스템 환경이 변해도 애플리케이션에 아무런 영향이 없고, 완성된 애플리케이션을 안정적으로 사용할 수 있다는 점이 있다.
- 반면에 사용하는 모든 오브젝트 코드를 실행 파일에 내장하기 때문에 메모리에 로드되는 애플리케이션 코드 크기가 커진다는 단점이 있다.
- 유닉스/리눅스에서는 확장자로 .a가 붙는다.

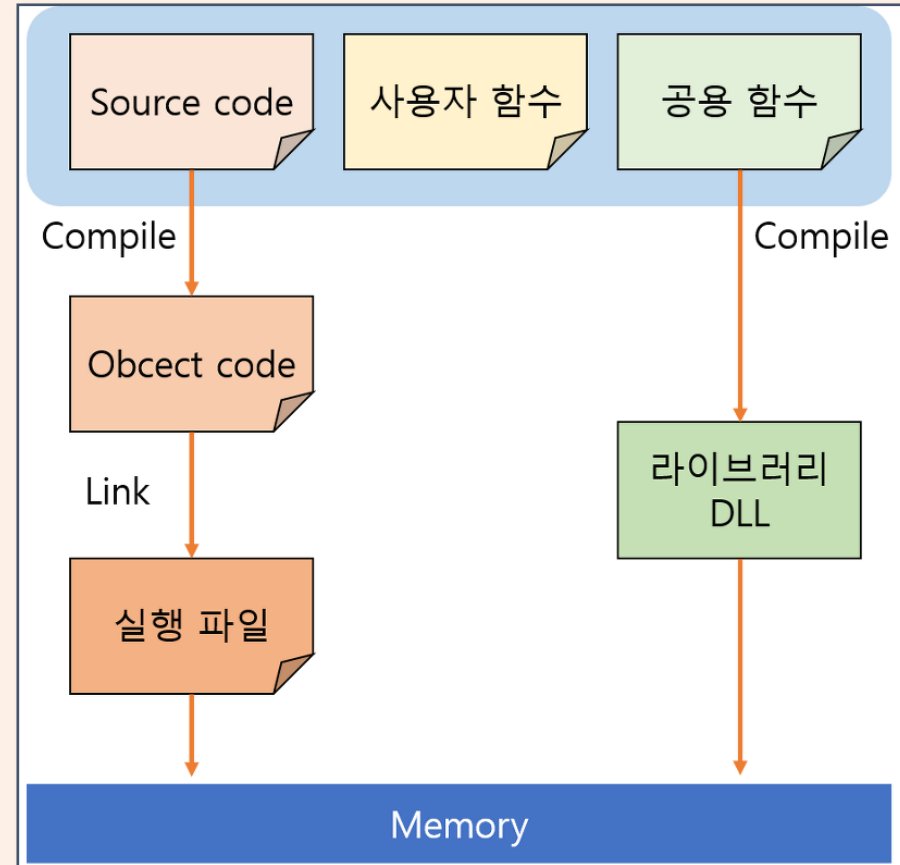
공유 라이브러리(Shared Library)란?

- 어떤 라이브러리가 제공하는 기능을 다른 애플리케이션에서 사용하고 싶을 때 라이브러리 코드를 메모리에 하나만 두고 각 애플리케이션에서 이를 공유하는 방식의 라이브러리를 의미한다.

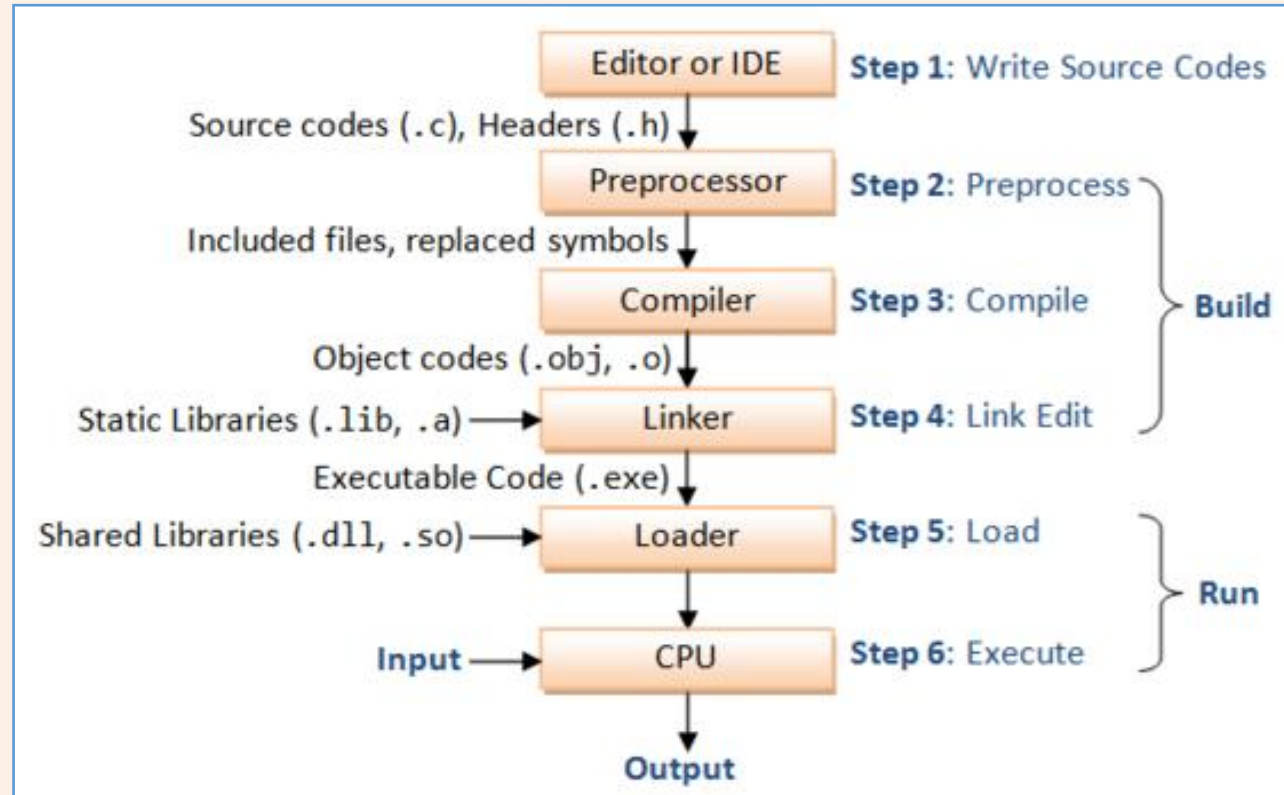
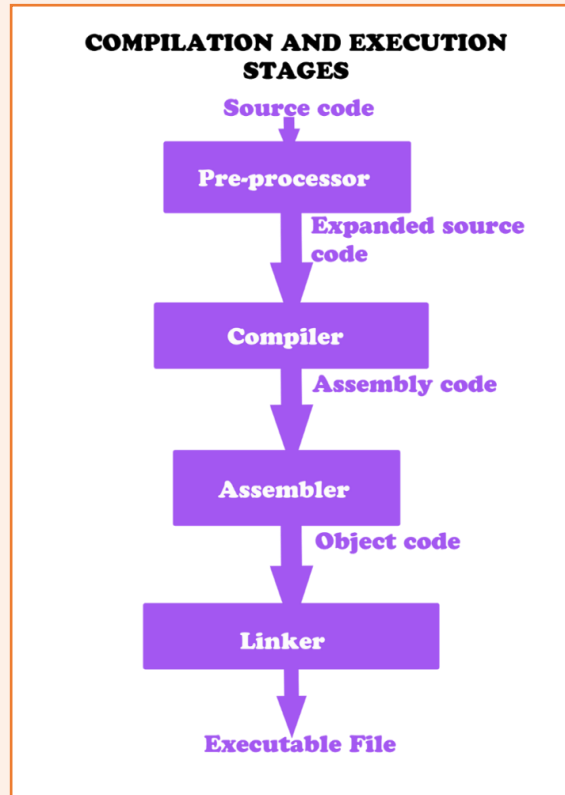


공유 라이브러리(Shared Library)란?

- 공유 라이브러리는 프로그램 실행 시 라이브러리의 코드와 애플리케이션의 코드가 메모리에 로드되는 시점에 링크된다.
- 그렇기 때문에 라이브러리를 이용하는 애플리케이션에는 호출할 라이브러리 함수의 정보만 들어 있다. 애플리케이션이 실행되어 메모리에 로드된 시점에서야 그 함수가 메모리의 어디에 있는지 알 수 있고, 그곳으로 포인터가 쓰여지면서 함수의 호출을 실현한다.
- 이 구조를 이용하면 한 라이브러리의 코드를 여러 애플리케이션에서 공유할 수 있기 때문에 메모리를 효율적으로 이용할 수 있다. 윈도우에서는 확장자로 .DLL이 붙고, 리눅스에서는 확장자로 .so가 붙는다.

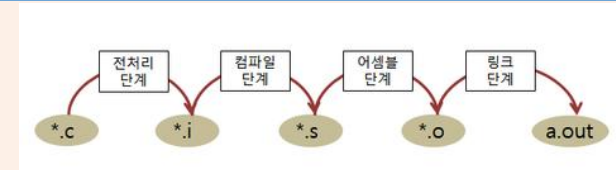


C Programming: The Four Stages of Compilation



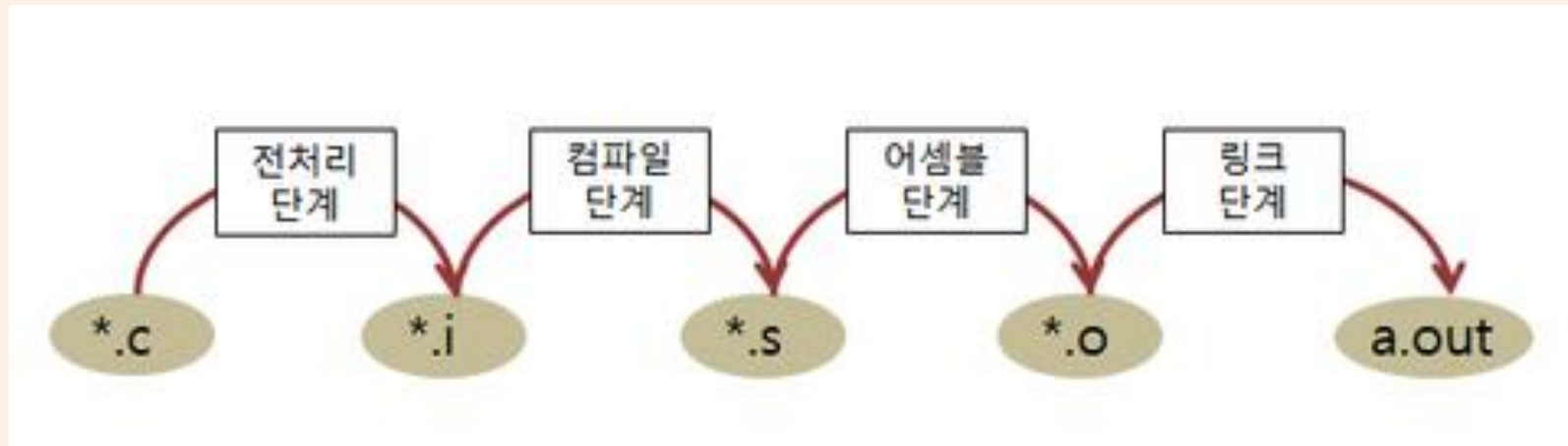
gcc C 컴파일러 드라이버 옵션

- E : 전처리 과정의 결과를 화면에 보이는 옵션
- save -temps 옵션을 사용해 like.i 파일을 읽어보는 것이 더 좋은 방법.
- S : 어셈블리 파일만 생성하고 컴파일 과정을 멈춘다.
- c : 오브젝트 파일까지만 생성하고 컴파일 과정을 멈춘다.
- v : gcc가 컴파일 과정을 어떤 식으로 수행하는지를 화면에 출력한다.
- save-temps : 컴파일 과정에서 생성되는 중간 파일인 전처리 파일(*.i)과 어셈블리 파일(*.s), 오브젝트 파일(*.o)을 지우지 않고 현재 디렉토리에 저장한다.



gcc -E hello.c -o hello.i

C Programming: The Four Stages of Compilation



`gcc -E hello.c -o hello.i` **전처리 단계:** c 파일을 gcc 컴파일러로 컴파일 할 경우 전처리 단계가 진행되는데 결과로 i 파일을 만듦

`gcc -S hello.c` **컴파일 단계:** 전처리된 파일로 컴파일을 진행하여 어셈블리어로 된 파일인 s 파일을 생성

`gcc -c hello.c` **어셈블 단계:** 어셈블리어로 쓰여진 s 파일을 컴퓨터가 이해할 수 있는 기계어로 된 파일인 o 파일로 변환

`gcc hello.c -o hello` **링크 단계:** 라이브러리 함수와 여러 오브젝트 파일들을 연결해서 실행 파일인 a.out을 생성

동적 로드 라이브러리(Dynamic Load Library)란?

- 동적 링크 방식에서는 애플리케이션의 실행 시, 실행 파일과 관련된 라이브러리 코드를 모두 메모리에 읽어들이어 호출. *관계를 조정한 다음 애플리케이션이 실행된다.*
- 동적 로드는 애플리케이션 실행 시에 읽어들이지 않은 라이브러리를 이용할 수도 있기 때문에 동적 링크보다 더욱 자유도가 높은 라이브러리의 활용 방식을 사용한다고 할 수 있다. 심지어 애플리케이션을 빌드 할 때 존재하지 않았던 라이브러리도 사용이 가능하다.
- 동적 로드에서는 프로그램 내부에서 라이브러리를 로드하는데, 어느 라이브러리의 어느 함수를 이용할지는 프로그램의 동작 상황에 따라 변할 수 있다. 대다수 애플리케이션에서 자주 이용되는 플러그인에 의한 기능 확장은 동적 로드를 이용해 구현된다.

라이브러리종류

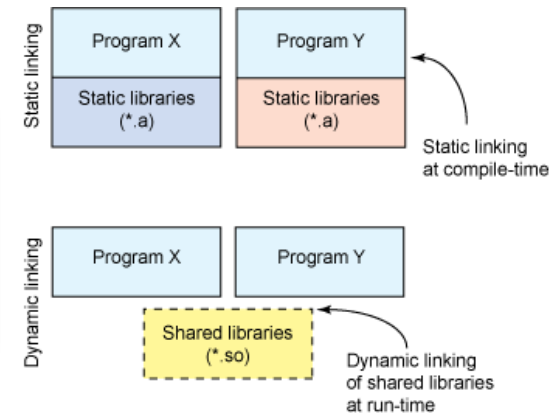
- 라이브러리는 함수, 구조체, 클래스 등을 포함하고 있는 컴파일된 파일이며, 그 종류는 다음과 같이 3가지가 존재한다.

* 정적 라이브러리 (*.a, *.lib)

* 공유 라이브러리 (*.so, *.dll)

* 동적 라이브러리 (*.so, *.dll)

종 류	Symbol reference resolve 시기 즉 프로그램에 라이브러리 적재 시기	라이브러리 사용 ?
정적 라이브러리 (Static Library)	컴파일 타임	컴파일 시 라이브러리를 적재한 그 프로그램만 라이브러리 코드를 사용.
공유 라이브러리 (Dynamic Linking)	런타임 (프로그램 메모리에 적재 시)	메모리에 라이브러리 적재 되 있으면 그 라이브러리 사용하는 프로그램끼리 한 그 메모리영역(라이브러리)를 공유한다.
동적 라이브러리 (Dynamic Loading)	런타임 (프로그램 실행 중 필요할 때) 즉 사용하는 응용프로그램이 결정	메모리에 라이브러리 적재 되 있으면 그 라이브러리 사용하는 프로그램끼리 한 그 메모리영역(라이브러리)를 공유한다.



- 정적 라이브러리는 컴파일시에 해당 정적 라이브러리의 내용이 실행 바이너리안에 포함되어지는 특징을 갖고 있다. 즉, 실행 파일 배포시에 정적라이브러리를 함께 배포하지 않아도 된다.

라이브러리종류

- 반면, 동적 라이브러리는 컴파일시에 해당 동적 라이브러리의 내용이 실행 파일안에 포함되지 않기 때문에, 반드시 실행 파일 배포시에는 동적 라이브러리 역시 함께 배포해야 한다.
- 유닉스 계열의 OS에서는 ar 시스템 유틸리티를 사용하여 정적라이브러리를 생성할 수 있다.
- 참고로 정적 라이브러리 생성시에 반드시 사용되어지는 옵션은 다음과 같다. -c create archive file -r insert object file 예를 들면 다음과 같다.
- `ar rc libmymath.a mymath.o` 그리고, 생성된 정적 라이브러리 파일내에 컴파일 된 Object List를 보기 위해서는 다음과 같이 -t 옵션을 사용할 수 있다.
- `ar t libmymath.a`

ar : 정적 라이브러리 작성 명령어 옵션

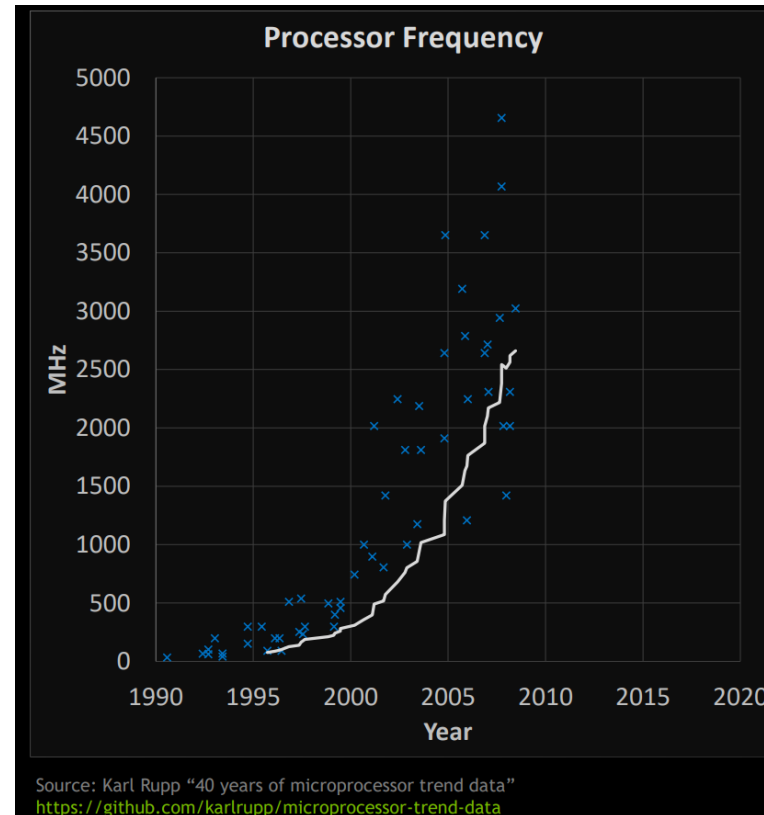
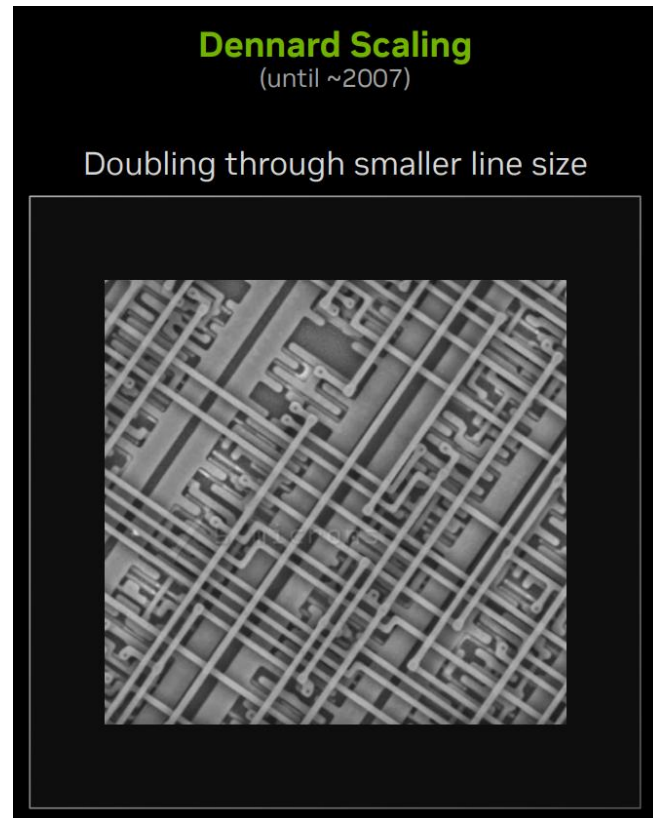
옵 션	내 용
-a	posname으로 피연산자에 의하여 명명된 파일 이후에 archive안에 새로운 파일을 위치 시킨다. 옵션 r, m과 같이 사용되며, 위치 경로 이름에 지정된 파일의 뒤에 파일을 기록한다.
-b 또는 -i	피연자인 posname에 의하여 파일이 명명 되어지기 이전에 archive안의 새로운 파일의 위치를 지정한다.
-c	archive가 생성 되어질 때 기본적으로 표준 에러로 기록되어지는 진단 메시지의 출력을 제한
-C	파일 시스템에 명명된 파일과 같은 이름이 있으면 추출 시 대체되어지는 것을 예방한다. 이 옵션은 prefix와 대체하는 파일들로부터 예방하는데 사용하는 -T 옵션과 같이 사용할 수 있다.
-d	archive파일로부터 하나 이상의 파일을 삭제
-m	파일들을 이동시킨다. 만약, posname 피연산와 -a, -b, -i가 같이 기술되어지면, 파일들을 새로운 위치로 이동시킨다. 이들, archive 파일의 끝으로 이동한다.
-p	archive안에 있는 파일들의 애용을 표준 출력으로 출력한다. 만약, 파일이 기술되어 있지 않으면, archive안에 순서적으로 기록되어 있는 모든 파일들이 출력된다.
-q	archive의 끝에 파일들을 빠르게 추가한다. 이때 추가되어지는 파일은 이미 추가되어 있는지 확인하지 않는다. 추가속도가 빠르기 때문에 여러 개의 파일을 모아서 대규모의 archive파일을 작성할 때 유용하다.
-r	지정한 파일을 archive 파일에 추가하거나 새로운 파일로 교체한다. -a, -i, -b 옵션과 같이 사용되지 않으면 새로운 파일은 archive 파일의 맨 뒤에 추가된다
-s	archive파일의 테이블을 다시 작성한다. 파일에 대한 정보를 삭제하는 strip 명령어와 같은 소프트웨어 개발 명령어 등에 의해 archive가 변경된 경우에 이 옵션이 사용된다.
-t	archive의 포함된 테이블을 출력한다. 파일은 기록되어 있는 목록에 포함된 파일을 기술해야만 한다. 만약, 파일이 기술되지 않으면, archive에 순서적으로 되어있는 모든 파일들을 출력한다.
-T	archive에서 추출하고자 하는 파일 이름이 너무 길면 파일 이름 자르는 것을 파일 시스템이 지원 가능한 것을 인정한다. 기본적으로 추출하는데 너무 긴 이름을 주게 되면 에러(error)가 발생하며, 추출 할 수 없다는 진단 메시지를 출력한다.

-u	archive에서 추출하고자 하는 파일 이름이 너무 길면 파일 이름 자르는 것을 파일 시스템이 지원 가능한 것을 인정한다. 기본적으로 추출하는데 너무 긴 이름을 주게 되면 에러(error)가 발생하며, 추출 할 수 없다는 진단 메시지를 출력한다.
-V	표준 에러상의 버전 번호(number)의 출력
-v	출력에 있어서 verbose를 준다. -d, -r, -x 옵션과 같이 사용되어지면, archive를 생성하는 파일의 설명과 구성하는 파일들, 유지 활동하는 것에 대하여 상세히 기록되어진다. -p옵션과 같이 사용되어지면, 파일 스스로 표준 출력으로 쓰기를 하기 전에 표준 출력의 파일의 이름을 출력(write)한다. -t 옵션과 같이 사용되어지면 archive안에 있는 파일에 관한 정보를 긴 목록 형식으로 출력한다. -x 오 k같이 사용되어지면, 선행적으로 각각 추출하는 파일이름을 출력(print) 한다. archive로 기록 되어지면, 메시지는 표준 에러로 기록되어진다.
-x	archive로부터 주어진 파일이름에 대하여 추출한다. Archive안에 있는 내용은 변화가 없다. 만약, 파일이 주어지지 않으면, archive에 있는 모든 파일이 추출되어진다. 만약, archive로부터 추출되어지는 파일의 파일 이름은, 추출되어지는 곳의 디렉토리의 내부까지 지원되어진다. archive로부터 추출되어지는 파일의 시간은 각각의 추출되어지는 파일들의 시간으로 수정된다.(지정된 파일만 출력)

왜 분산처리(병행처리)처리인가.??

Dennard scaling

반도체 전자 장치에서 Dennard 스케일링(MOSFET 스케일링이라고도 함)은 트랜지스터가 작아질수록 전력 밀도가 일정하게 유지되어 전력 사용이 면적에 비례하여 유지된다는 스케일링 법칙

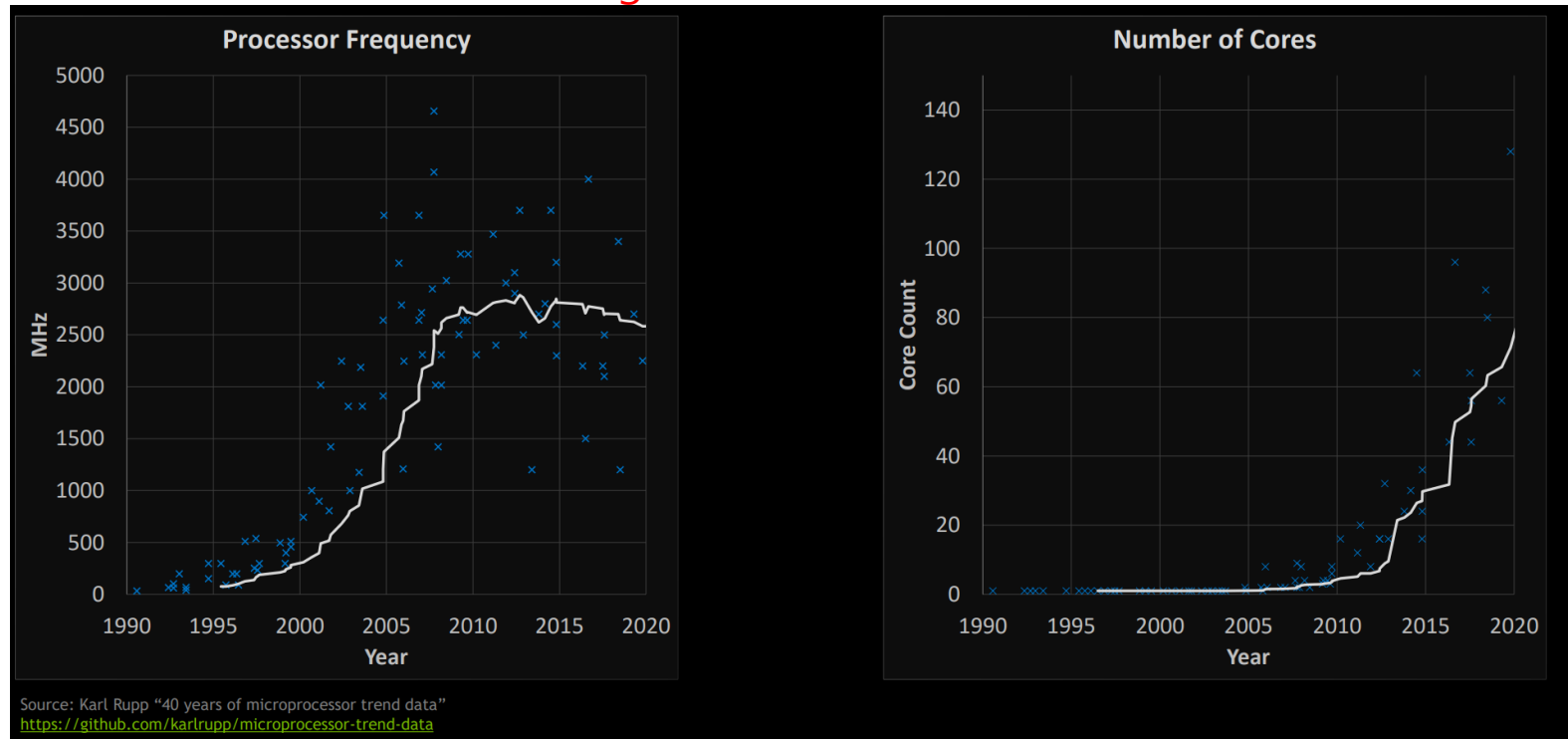


왜 분산처리(병행처리)처리인가.??

Dennard scaling

반도체 전자 장치에서 Dennard 스케일링(MOSFET 스케일링이라고도 함)은 트랜지스터가 작아 질수록 전력 밀도가 일정하게 유지되어 전력 사용이 면적에 비례하여 유지된다는 스케일링 법칙

The End of Dennard Scaling



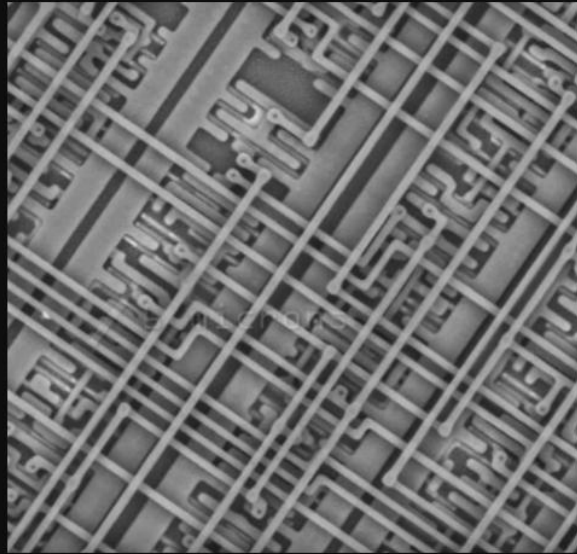
왜 분산처리(병행처리)처리인가.??

Second Phase of Moore's Law

Dennard Scaling

(until ~2007)

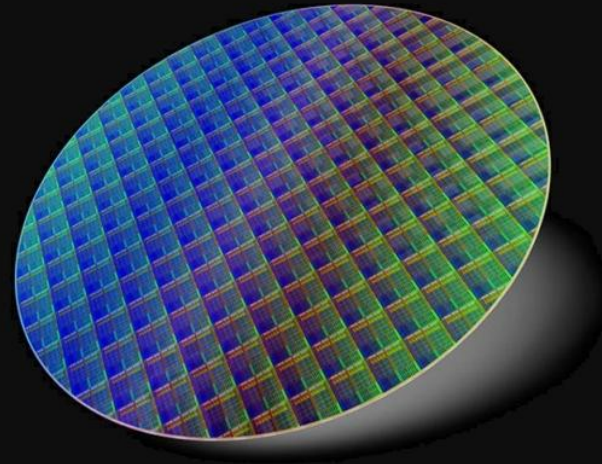
Doubling through smaller line size



Single-Die Scaling

(~2007 → Present)

Doubling through increasing core count



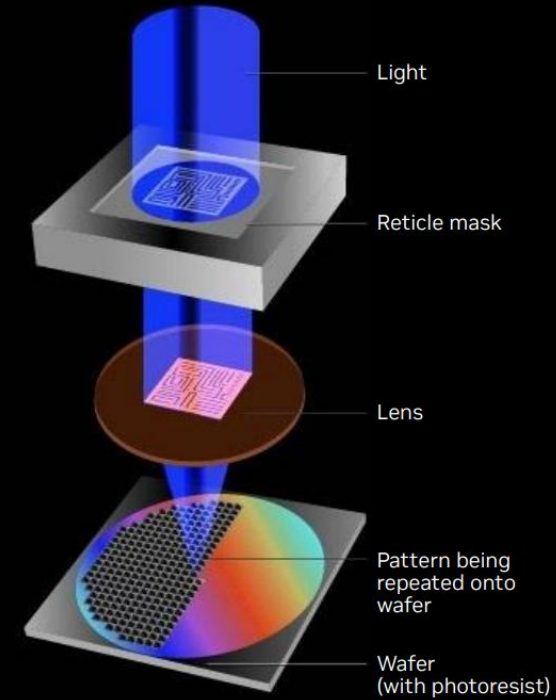
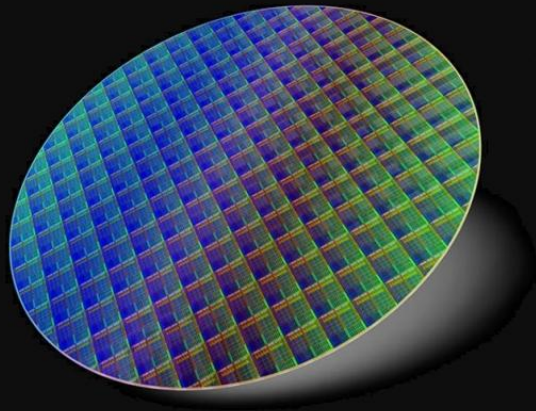
왜 분산처리(병행처리)처리인가.??

The End of Single-Die Scaling

Single-Die Scaling

(~2007 → Present)

Doubling through increasing core count



왜 분산처리(병행처리)처리인가.??

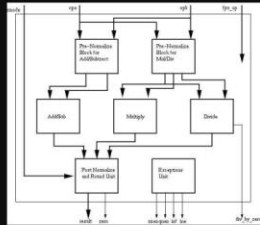
레티클 한계(Reticle Limit) 극복: Heterogeneous Hardware

heterogeneous

adjective US: /ˌhet̬.ə.rouˈdʒiː.ni.əs/ UK: /ˌhet.ər.əˈdʒiː.ni.əs/

consisting of parts or things that are very different from each other

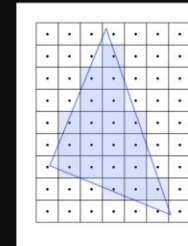
Floating point unit



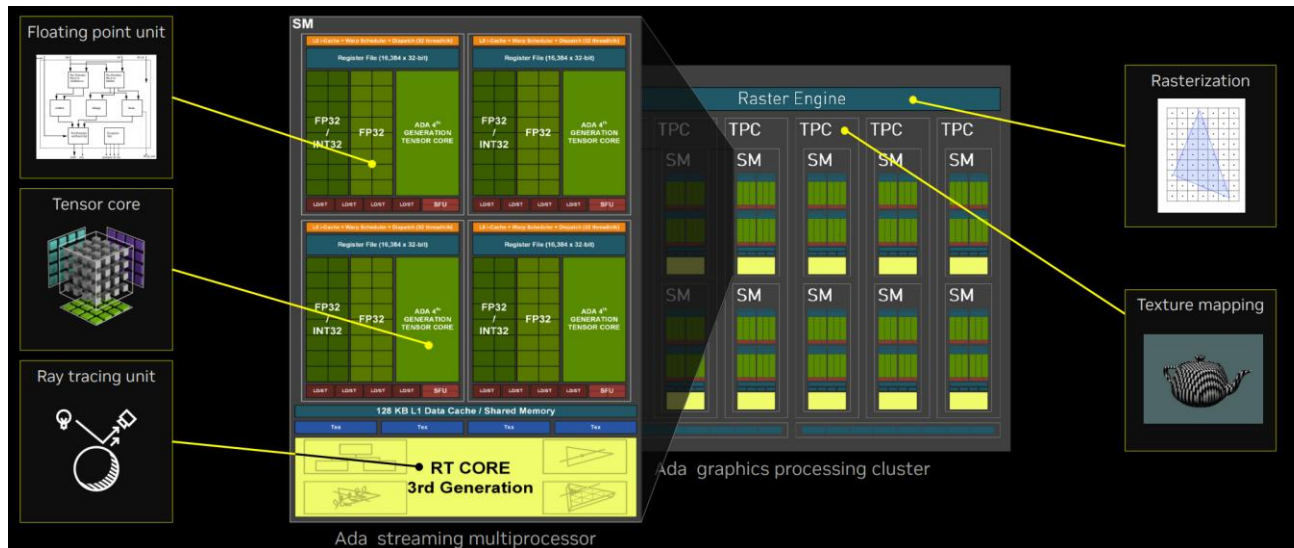
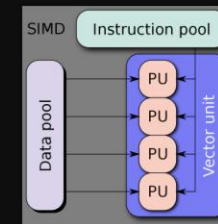
Texture mapping



Rasterization



Vector unit



왜 분산처리(병행처리)처리인가.??

Heterogeneous Software

Software is also made up of heterogeneous components written in many different languages

Example: Heterogeneous Programming in Python

```
import numpy as np

# Array of 1M random numbers
data = np.random.randint(1, 100, (1024 * 1024))

total = np.sum(data)      # Reduction across array
max = np.amax(data)      # Max in array
average = np.mean(data)  # Average across array
```

← User application is written in Python for productivity

NumPy Library

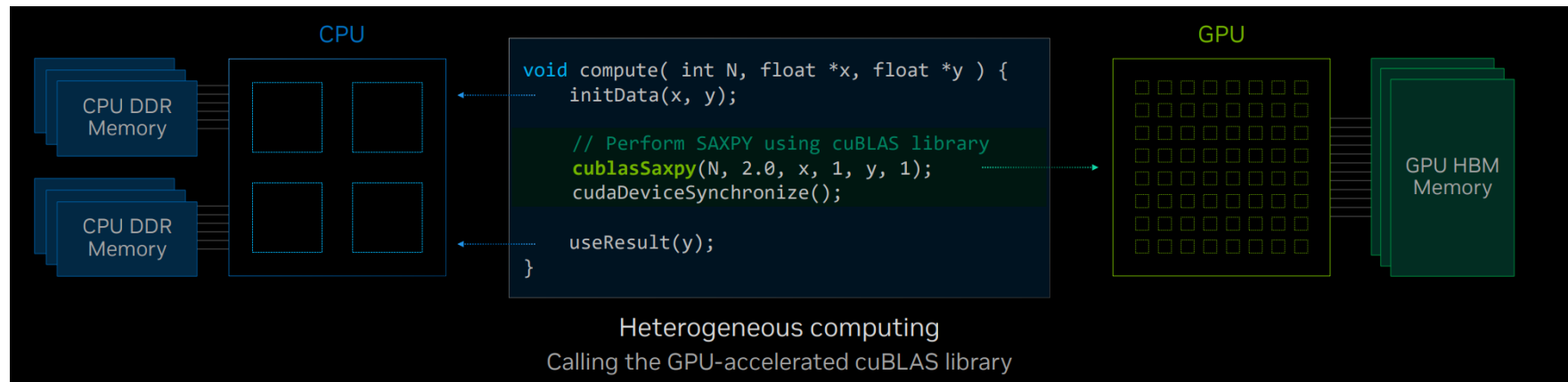
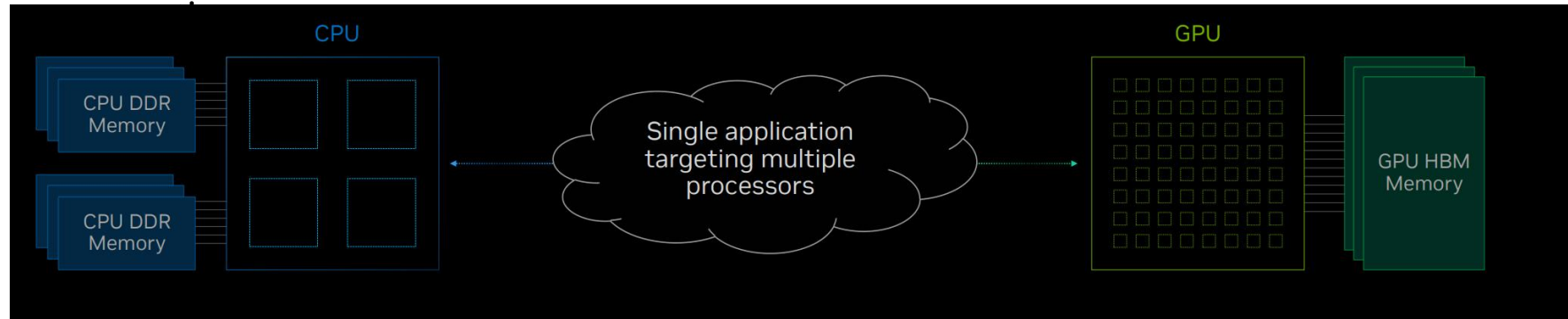
NumPy library implements fast computation in C & C++

Applications often combine components & libraries written in many different languages

왜 분산처리(병행처리)처리인가.??

Heterogeneous Computing

Combining processors of different types, each specializing in different types of

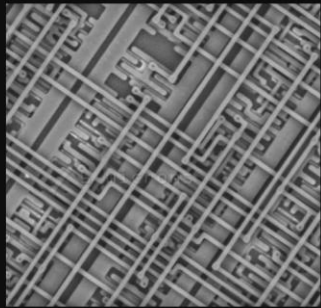


왜 분산처리(병행처리)처리인가.??

Third Phase of Moore's Law

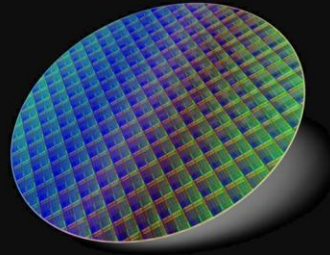
Dennard Scaling (until ~2007)

Doubling through smaller line size



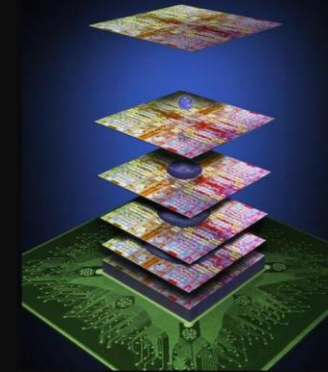
Single-Die Scaling (~2007 → Present)

Doubling through increasing core count

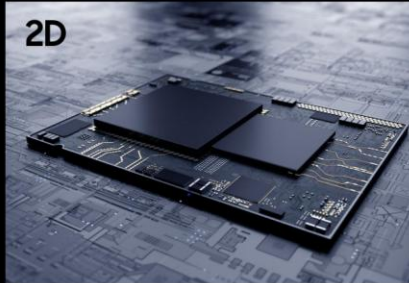


Multi-Die Scaling (Present → Future)

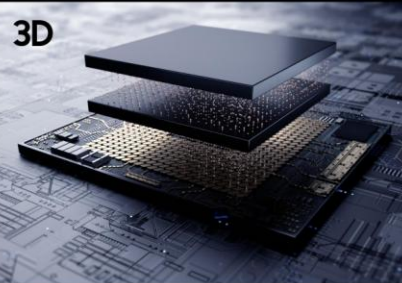
Doubling through combining dies



2D

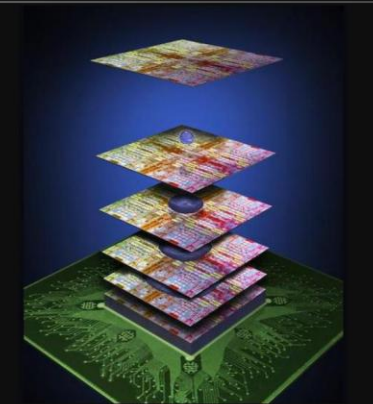


3D



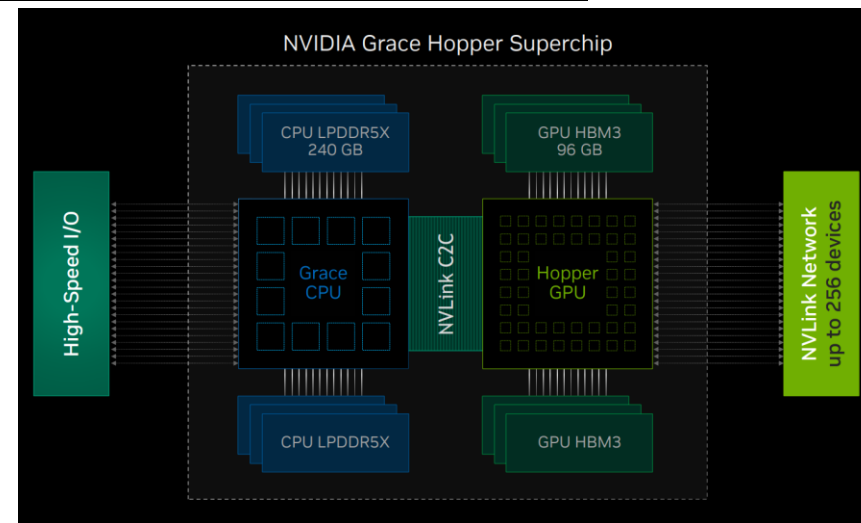
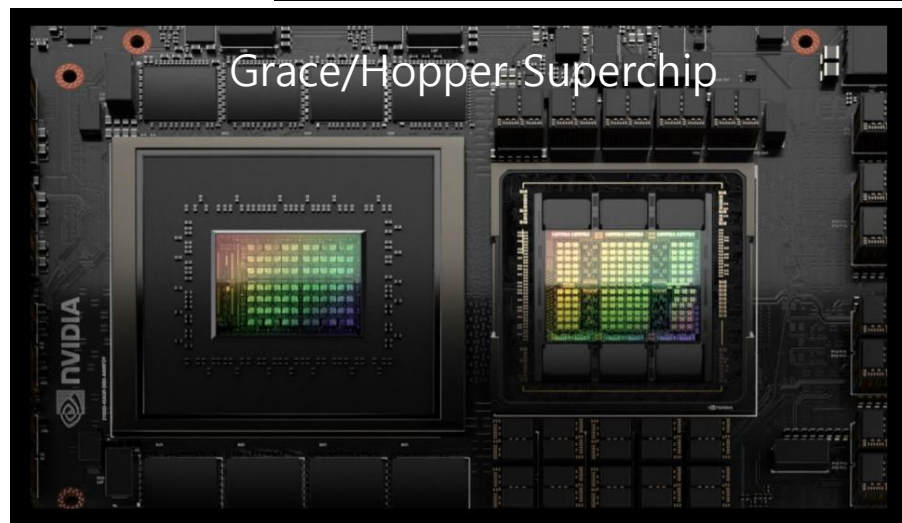
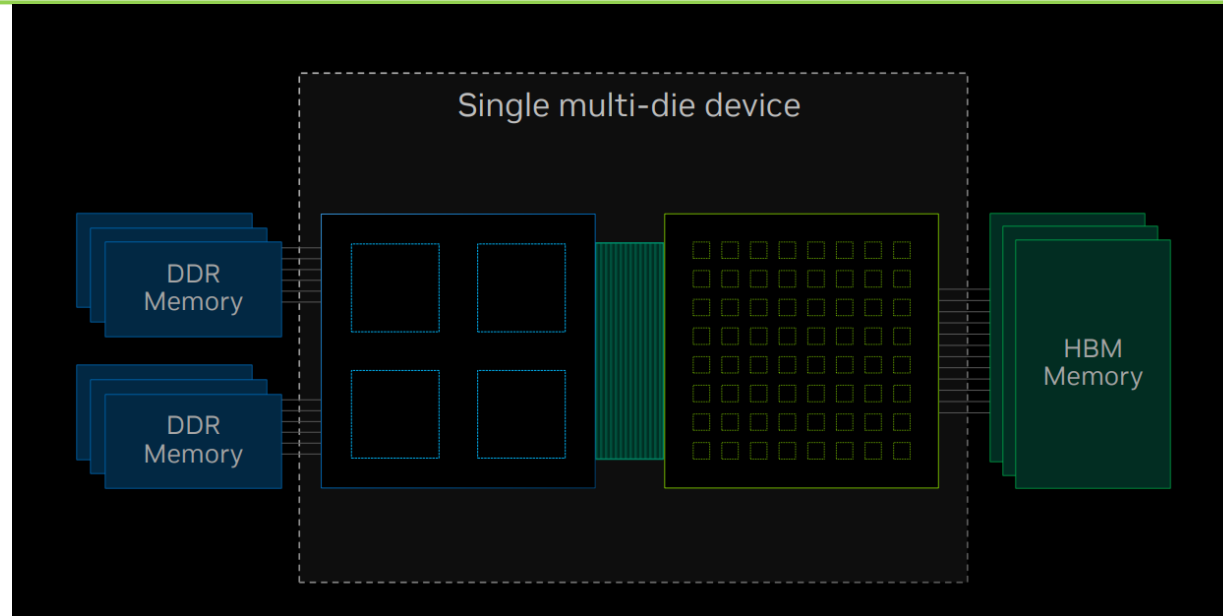
Multi-Die Scaling (Present → Future)

Doubling through combining dies



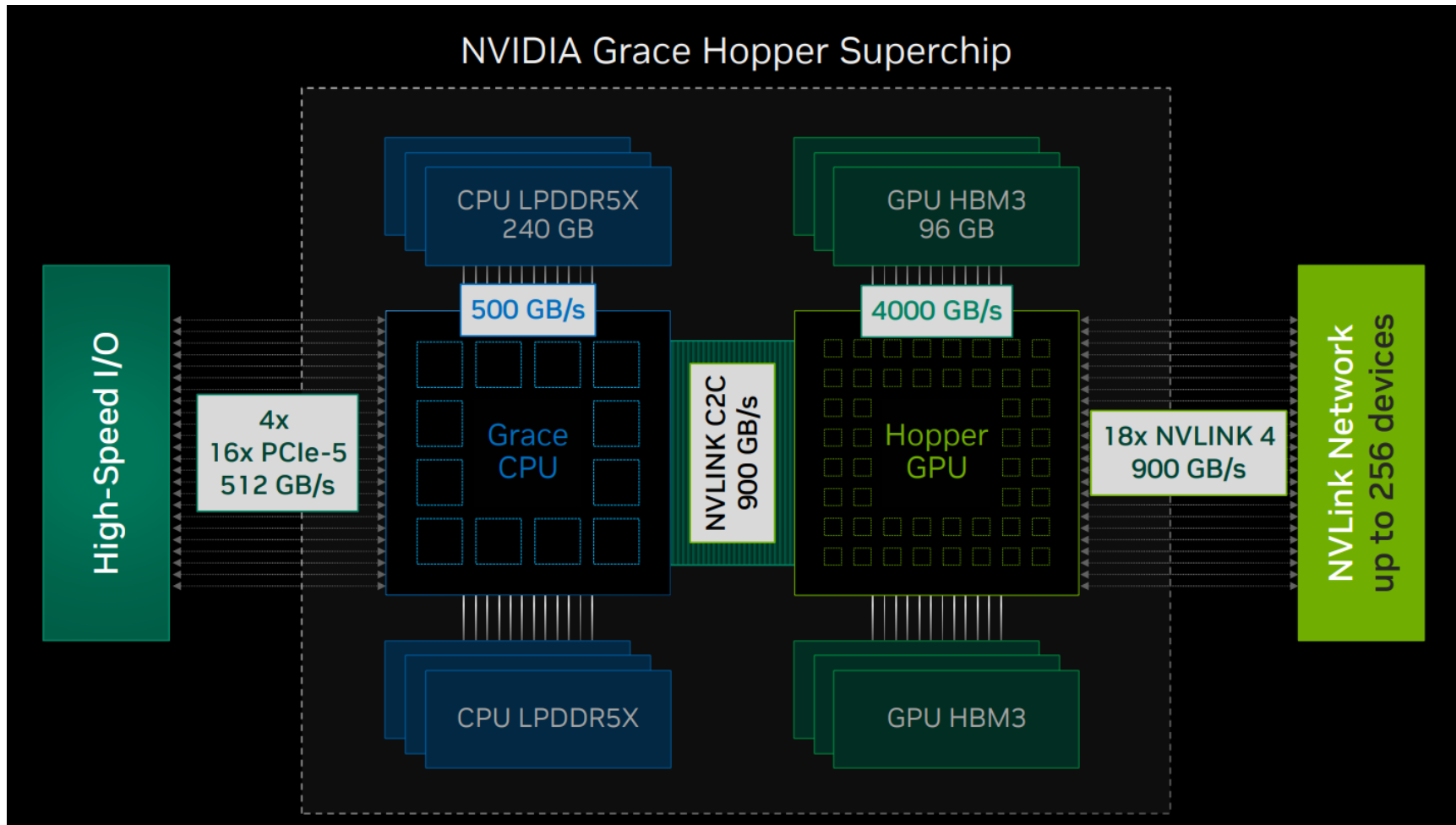
왜 분산처리(병행처리)처리인가.??

Multi-Die Heterogeneous Devices



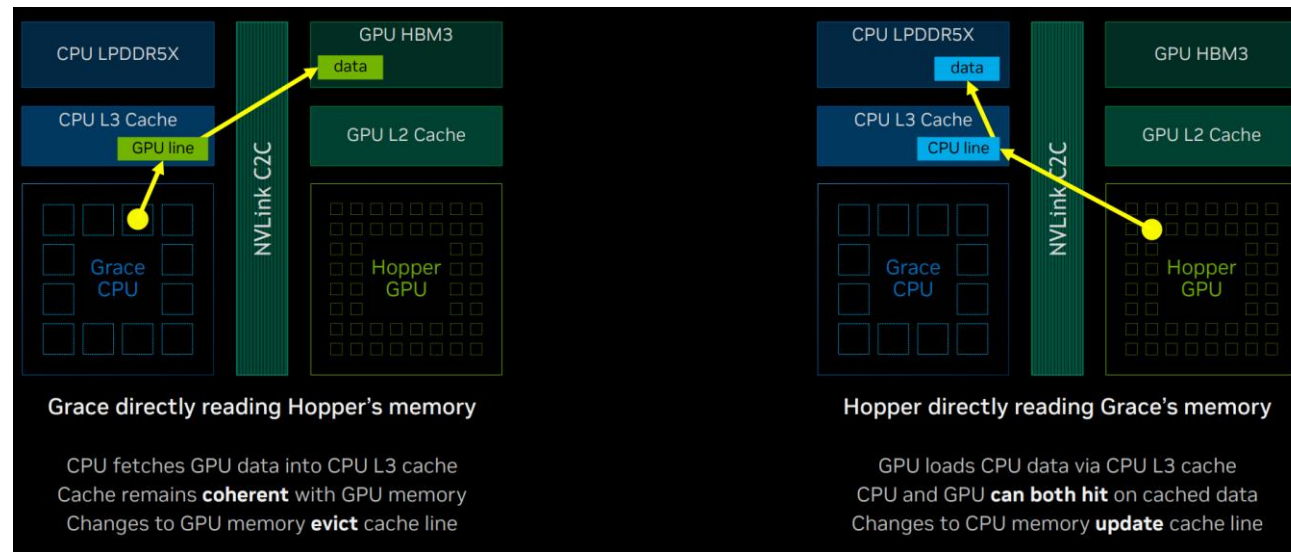
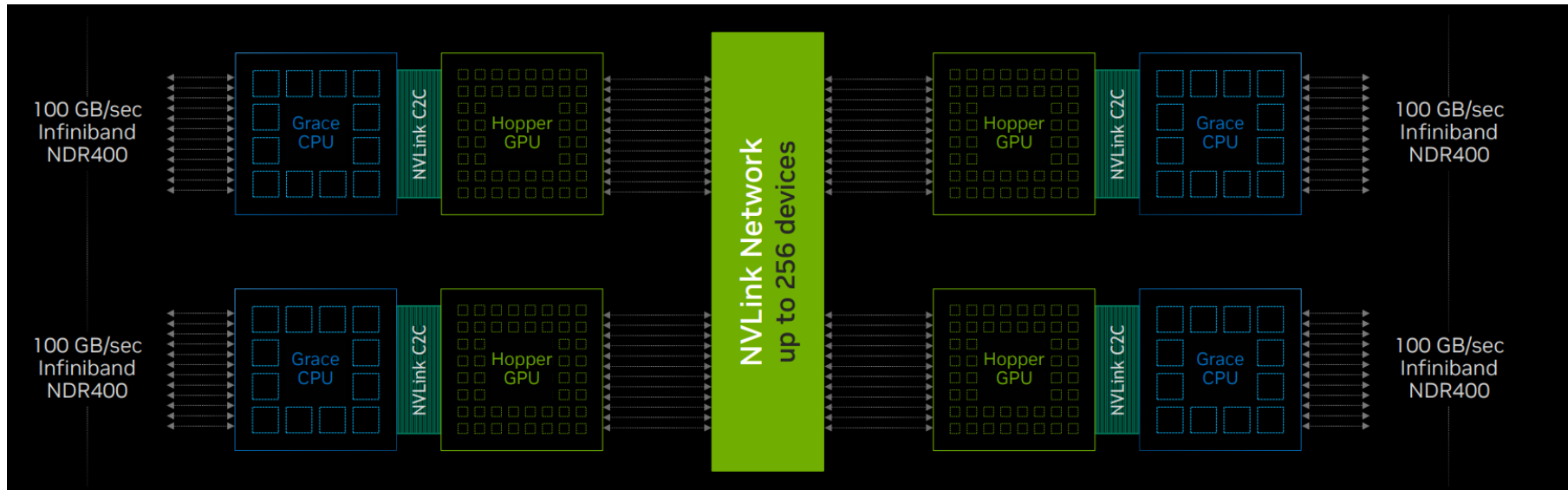
왜 분산처리(병행처리)처리인가.??

Grace/Hopper Superchip



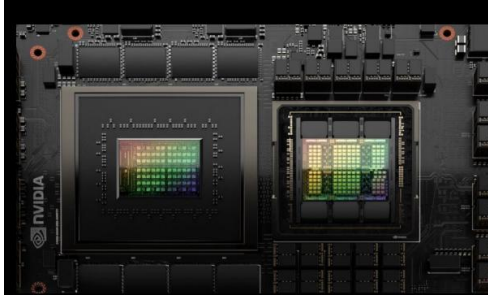
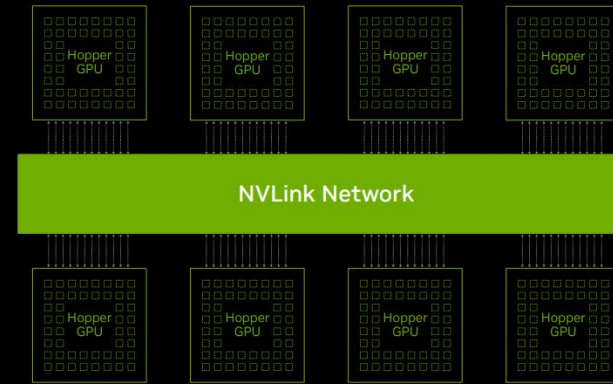
왜 분산처리(병행처리)처리인가.??

NVLink Connects Up To 256 Superchips



왜 분산처리(병행처리)처리인가.??

Multi-Chip Systems



Multi-die



Multi-chip



Multi-node

왜 분산처리(병행처리)처리인가.??

There Is No Future That Is Not Multi-Node

